
PCIe-6353 and PXIe-6353 User Manual

2026-06-16



Contents

PCIe-6353 and PXIe-6353 User Manual	6
Overview	7
Driver Support	7
System Overview	7
Signal Conditioning Options	10
Cables and Accessories	11
Theory of Operation	14
Finding Examples	17
Analog Input	20
Analog Input Range	21
Working Voltage Range	22
Analog Input Ground-Reference Settings	23
Configuring AI Ground-Reference Settings in Software	25
Multichannel Scanning Considerations	25
Analog Input Data Acquisition Methods	29
Getting Started with AI Applications in Software	30
Analog Input Timing Signals	31
Aggregate versus Single Channel Sample Rates	34
AI Sample Clock Signal	34
AI Sample Clock Timebase Signal	36
AI Convert Clock Signal	37
AI Convert Clock Timebase Signal	40
AI Hold Complete Event Signal	41
AI Start Trigger Signal	41
AI Reference Trigger Signal	43
AI Pause Trigger Signal	45
Analog Input Common Questions	47
Analog Output	50
AO Reference Selection	51
Analog and Digital Output Data Generation Methods	51
Minimizing Glitches on the Output Signal	53
Getting Started with AO Applications in Software	54

Analog Output Triggering.....	55
Analog Output Timing Signals	55
AO Start Trigger Signal	55
AO Pause Trigger Signal	57
AO Sample Clock Signal	59
AO Sample Clock Timebase Signal.....	61
Digital I/O	63
Digital Input Triggering.....	64
I/O Protection	64
Programmable Power-Up States	64
DI Change Detection	65
Digital Filtering.....	66
Getting Started with DIO Applications in Software.....	70
Digital Waveform Acquisition and Input Triggering	71
Digital Input Data Acquisition Methods.....	72
DI Sample Clock Signal.....	73
DI Sample Clock Timebase Signal	75
DI Start Trigger Signal	76
DI Reference Trigger Signal	79
DI Pause Trigger Signal	80
Digital Waveform Generation and Output Triggering	82
Digital Output Triggering	83
Analog and Digital Output Data Generation Methods	83
DO Sample Clock Signal	85
DO Sample Clock Timebase Signal	87
DO Start Trigger Signal	88
DO Pause Trigger Signal	89
Counters	92
Counter Timing Engine.....	93
Counter Input Applications	94
Counting Edges.....	94
Pulse-Width Measurement.....	96
Pulse Measurement.....	98
Comparing Pulse and Semi-Period Measurements	100
Semi-Period Measurement	101
Frequency Measurement	102

Choosing a Method for Measuring Frequency	106
Period Measurement	110
Position Measurement	111
Two-Signal Edge-Separation Measurement	114
Counter Output Applications	116
Simple Pulse Generation	117
Pulse Train Generation	118
Frequency Generation	124
Pulse Generation for ETS	125
Counter Timing Signals	126
Counter Triggering	134
Cascading Counters	135
Counter Signal Prescaling	136
Counter Signal Synchronization Modes	136
PFI	138
Digital Routing and Clock Generation	145
Clock Routing	145
PXIe Device Synchronization	147
PCIe Device Synchronization	148
Real-Time System Integration (RTSI)	148
PXI and PXI Express Clock and Trigger Signals	150
Data Transfer Methods	154
PXI Express Considerations	156
Triggering	157
Triggering with a Digital Source	157
Triggering with an Analog Source	158
APFI <0,1> Terminals	158
PCIe-6353 and PXIe-6353 Analog Input Channels	158
Analog Trigger Actions	159
Analog Trigger Types	159
Analog Trigger Accuracy	161
Unpacking the Kit	163
Safety Guidelines	164
PCIe-6353 and PXIe-6353 Pinout	165
Installing Software and Hardware	171
Installing the Software	171

Installing the PXIe Device into a Chassis	172
Installing the PCIe Device	173
PCI Express Device Disk Drive Power Connector	175
Real-Time System Integration (RTSI)	177
Configuring the Device in MAX	179
Self-Calibrating the Device in MAX	181
Configuring a Measurement	182
Connecting Signals	183
Connecting Analog Input Signals	183
Connecting Floating Signal Sources	184
Connecting Ground-Referenced Signal Sources	191
Connecting Analog Output Signals	195
Field Wiring Considerations	195
Connecting Digital I/O Signals	196
Connecting to Counter Signals	196
Connecting to PFI Input Signals	198
Connecting to the +5 V Power Source	198
Device Routing in MAX	199
Calibration	201
Cleaning	203
Maintaining Forced-Air Cooling for PCIe Devices	204
Maintaining Forced-Air Cooling for PXIe Devices	205

PCIe-6353 and PXIe-6353 User Manual

The PCIe-6353 and PXIe-6353 User Manual provides detailed descriptions of the product functionality and the step by step processes for use.

Looking for Something Else?

For information not found in the User Manual for your product, such as specifications and API reference, browse *Related Information*.

Related information:

- [PCIe-6353 Specifications](#)
- [PXIe-6353 Specifications](#)
- [PCIe-6353 Dimensional Drawings](#)
- [DAQ Multifunction I/O \(MIO\) and Simultaneous Multifunction I/O \(SMIO\) Devices Calibration Procedure](#)
- [NI-DAQmx User Manual](#)
- [NI-DAQmx C Reference](#)
- [NI-DAQmx VI and Function Reference](#)
- [NI-DAQmx .NET Framework 4.0 API Reference](#)
- [NI-DAQmx .NET Framework 4.5 API Reference](#)
- [NI-DAQmx Properties](#)
- [NI-DAQmx Python Reference](#)
- [NI Driver and Development Software Compatibility](#)
- [LabVIEW User Manual](#)
- [Software and Driver Downloads](#)
- [Release Notes](#)
- [License Setup and Activation](#)
- [Product Certifications](#)
- [Letter of Volatility](#)
- [Discussion Forums](#)
- [NI Learning Center](#)
- [NI Support](#)

PCIe-6353 and PXIe-6353 Overview

32 AI, (16-Bit, 1.25 MSa/s), 4 AO (2.86 MSa/s), 48 DIO Multifunction I/O Device

This product includes the following features:

- 32 SE/16 DIFF AI channels; 1.25 MSa/s single-channel, 1 MSamples/s multichannel
- Four 16-bit AO channels; 2.86 MSa/s single channel, 1.25 MSa/s multichannel
- 48 5 V TTL DIO lines; 32 hardware-timed up to 10 MHz
- Four 32-bit, 100 MHz counters
- Two 68-pin female VHDCI connectors

PCIe-6353 and PXIe-6353 Driver Support

NI-DAQmx supports the PCIe-6353 and PXIe-6353.



Note The earliest NI-DAQmx support version for the NI PCIe-6353 is NI-DAQmx 9.0.



Note The earliest NI-DAQmx support version for the NI PXIe-6353 is NI-DAQmx 25.5.

Components of a PCIe-6353 and PXIe-6353 System

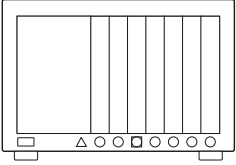
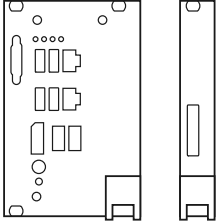
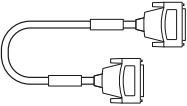
PCIe-6353 and PXIe-6353 instruments are designed for use in a system that requires hardware, drivers, and software for any application. Use these minimum required system components as a starting point for building your system.

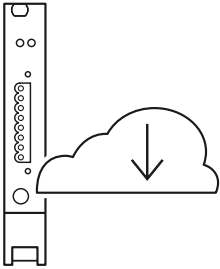
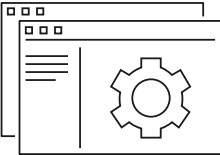


Note A system and the surrounding environment must meet any requirements defined in the relevant specifications documents.

Table 1. Minimum Required PCIe-6353 and PXIe-6353 System Components

Component	Description and Recommendations
PXI chassis or PC with PCIe slot	A PXI chassis or PC with PCIe support encloses your device

Component	Description and Recommendations
	and supplies power device functions.  Note Choose the enclosure that corresponds to your device form factor and bus type.
PXI controller or PXI remote control module 	If you are using a PXI chassis, you can install a PXI controller or a PXI remote control (MXI) module in the same chassis with your device, depending on your system requirements. These components interface with your instrument using NI device drivers.
Additional instruments 	PCIe-6353 and PXIe-6353 applications typically involve additional sensors, transducers, or signal conditioning hardware. For additional details, refer to the sections following this table.
Cables and accessories 	Cables and accessories allow connectivity to/from your instrument for measurements. Visit <i>63xx Models: DAQ Multifunction I/O Cable and Accessory Compatibility</i> on ni.com for a complete list of cables and accessories.
NI-DAQmx driver application software	Instrument driver software that provides functions to interact or execute measurements with the PCIe-6353 and PXIe-6353.  Note For optimal performance, use the most

Component	Description and Recommendations
	recent version of the driver software.
NI applications 	NI-DAQmx offers driver support for the following applications: • InstrumentStudio • LabVIEW • LabWindows/CVI • C/C++ • .NET • Python

Signal Conditioning

Most computer-based measurement systems involve plug-in data acquisition (DAQ) devices with some form of signal conditioning. Sensors and transducers usually require signal conditioning before a measurement system can effectively and accurately acquire the signal. The front-end signal conditioning system can include functions such as signal amplification, attenuation, filtering, electrical isolation, simultaneous sampling, and multiplexing. In addition, many transducers require excitation currents or voltages, bridge completion, linearization, or high amplification for proper and accurate operation.

Sensors and Transducers

Sensors generate electrical signals to measure physical phenomena, such as temperature, force, sound, or light. Strain gauges, thermocouples, thermistors, angular encoders, linear encoders, and resistance temperature detectors (RTDs) are commonly used sensors.

To measure signals from these various transducers, you must convert them into a form that a DAQ device can accept. For example, the output voltage of most thermocouples is very small and susceptible to noise. You may need to amplify or filter the thermocouple output before digitizing it. The manipulation of signals to prepare them for digitizing is called signal conditioning.

Related information:

- [63xx Models : DAQ Multifunction I/O Cable and Accessory Compatibility](#)
- [Sensor Fundamentals](#)

Signal Conditioning Options

SCXI

SCXI is a front-end signal conditioning and switching system for various measurement devices, including X Series devices. An SCXI system consists of a rugged chassis that houses shielded signal conditioning modules that amplify, filter, isolate, and multiplex analog signals from thermocouples or other transducers. SCXI is designed for large measurement systems or systems requiring high-speed acquisition.

System features include the following:

- Modular architecture—Choose your measurement technology
- Expandability—Expand your system to 3,072 channels
- Integration—Combine analog input, analog output, digital I/O, and switching into a single, unified platform
- High bandwidth—Acquire signals at high rates
- Connectivity—Select from SCXI modules with thermocouple connectors or terminal blocks

SCC

SCC is a front-end signal conditioning system for PCIe-6353 and PXIe-6353 plug-in data acquisition devices. An SCC system consists of a shielded carrier that holds up to 20 single- or dual-channel SCC modules for conditioning thermocouples and other transducers. SCC is designed for small measurement systems where you need only a few channels of each signal type, or for portable applications. SCC systems also offer

the most comprehensive and flexible signal connectivity options.

System features include the following:

- Modular architecture—Select your measurement technology on a per-channel basis
- Small-channel systems—Condition up to 16 analog input and eight digital I/O lines
- Low-profile/portable—Integrates well with other laptop computer measurement technologies
- Connectivity—Incorporates panellet technology to offer custom connectivity to thermocouple, BNC, LEMO™ (B Series), and MIL-Spec connectors

PCI Express Considerations for SCC



Note PCI Express users should consider the power limits on certain SCC modules without an external power supply. Refer to the device specifications, and the *PCI Express Supplemental Power Connector* topic for information about power limits and increasing the current the device can supply on the +5 V terminal.

Cables and Accessories



Notice For compliance with Electromagnetic Compatibility (EMC) requirements, operate the PCIe-6353 and PXIe-6353 with shielded cables and accessories. If unshielded cables or accessories are used, the EMC specifications are no longer guaranteed unless all unshielded cables and/or accessories are installed in a shielded enclosure with properly designed and shielded input/output ports.

NI recommends using the following cables and accessories with the PCIe-6353 and PXIe-6353:

Table 2. Cables and Accessories

Accessory Description	Description	Part Number	Notes
SHC68-68-EPM Multifunction Cable,	Shielded. 0.5 m, 1 m, 2 m, 5 m, and 10 m	192061-0R5/01/02/05/10	The SHC68-C68-EPM is intended for custom

Accessory Description	Description	Part Number	Notes
68 D-Type to 68 VHDCI Offset	lengths. Recommended		breakout fixtures that use the VHDCI 0.8 mm connector. Accessories listed in this manual use a SCSI 0.050 D-Type connector and will not work with this cable. For use only with Connector 0 on the NI 6345, 6355, 6357, 6365, or 6375.
SCB-68A Screw Terminal Block with Temperature Sensor	Noise rejecting, shielded	782536-01	
TB-2706 Screw Terminal Block	Front mount	779183-01	Uses Connector 0 of your PXI Express device and makes Connector 1 unavailable. Not for use with the NI 6345, 6355, 6357, 6365, or 6375.
BNC-2110 BNC Terminal Block	Noise rejecting, shielded	777643-01	
BNC-2120 BNC Terminal Block	Noise rejecting, shielded, with function generator and encoder	777960-01	
BNC-2115 BNC Terminal Block	Noise rejecting, shielded connector block for extended I/O.	777807-01	For use with only connectors 1, 2, or 3 of the NI 6345, 6355, 6357, 6365, or 6375 or connector 1 of the NI 6349.

Accessory Description	Description	Part Number	Notes
BNC-2090A BNC Terminal Block	Rack-mount	779556-01	
SHC68-68 Twisted Pair Cable	Basic shielding. 0.5 m, 1 m, 2 m lengths	191945-0R5/01/02	
RC68-68 Ribbon Cable, 68 D-Type to 68 VHDCI Offset	0.25 m, 0.5 m, and 1 m lengths	187252-0R25/0R5/01	
CB-68LP Screw Terminal Block	Unshielded, vertical mount	777145-01	
	Unshielded, right-angle power	777145-02	
TBX-68 Screw Terminal Block	DIN rail mount, unshielded	777141-01	
PXI Slot Blocker	Set of 5	199198-01	



Note Visit *DAQ Multifunction I/O Cable and Accessory Compatibility* for more information about supported cables and accessories for the PCIe-6353 and PXIe-6353.

BNC Accessories

You can use one BNC accessory on connector 0 of any NI 63xx devices. An additional BNC accessory may be used on connector 1. Some devices support only specific BNC accessories for connectors other than connector 0.

- The NI 6345, 6355, 6357, 6365, and 6375 support only the BNC-2115 on connectors 1, 2, and 3.
- The NI 6349 support only the BNC-2115 on connector 1.



Note You can use the SHC68-68-EPM shielded cable to connect the PCIe-6353 and PXIe-6353 to a BNC terminal block. This cable can also be used with breakout fixtures that use the VHDCI 0.8 mm connector.

Custom Cabling and Connectivity

Note the following guidelines when developing custom cables:

- For AI signals, use shielded, twisted-pair wires for each AI pair of differential inputs. Connect the shield for each signal pair to the ground reference at the source.
- Route the analog lines separately from the digital lines.
- When using a cable shield, use separate shields for the analog and digital sections of the cable. To prevent noise when using a cable shield, use separate shields for the analog and digital sections of the cable.



Note Visit *NI DAQ Device Custom Cables, Replacement Connectors, and Screws* for more information about the connectors used for DAQ devices.

RTSI Cables (PCI Express Devices)

Use RTSI cables to connect timing and synchronization signals among PCI/PCI Express devices, such as X Series, M Series, CAN, and other measurement, vision, and motion devices. PXI devices use PXI backplane signals for timing and synchronization and do not require timing and synchronization cables.

Related information:

- [NI DAQ Device Custom Cables, Replacement Connectors, and Screws](#)
- [63xx Models : DAQ Multifunction I/O Cable and Accessory Compatibility](#)

PCIe-6353 and PXIe-6353 Theory of Operation

The PCIe-6353 and PXIe-6353 combines NI-STC3 timing and synchronization technology with simultaneous sampling of multiple functions including analog I/O, digital I/O, counters and triggering.

The PCIe-6353 and PXIe-6353 is designed for maximum throughput applications with low latency. NI-STC3 technology provides four 32-bit counters and a 100 MHz timebase. Each I/O subsystem features independent sample clocks and triggers that let you perform hardware-timed acquisitions with improved timing and triggering capability. Use the retrigger options to automatically re-arm the trigger for the next

acquisition, and the dedicated clock to generate or acquire hardware-timed digital waveforms.

The 100 MHz timebase drives all digital circuitry. This timebase is used as the reference to generate clock frequencies and latch digital edges.

Eight DMA channels stream data directly between the PCIe-6353 and PXIe-6353 and PC memory without CPU interaction or additional programming and provide parallel data streams for each subsystem.

Each 32-bit counter has a 127-sample FIFO and can perform buffered counter operations, such as event counting or generating a PWM output, at fast rates.

The NI-DAQmx driver is multithreaded to develop applications that segment processing of your measurement and generation tasks into separate threads. You can log data to TDMS files at rates above 1 GB/s.

For high-channel-count applications, connect multiple DAQ modules with a Real-Time System Integration (RTSI) cable or synchronize over the timing and triggering buses on the PXIe backplane.

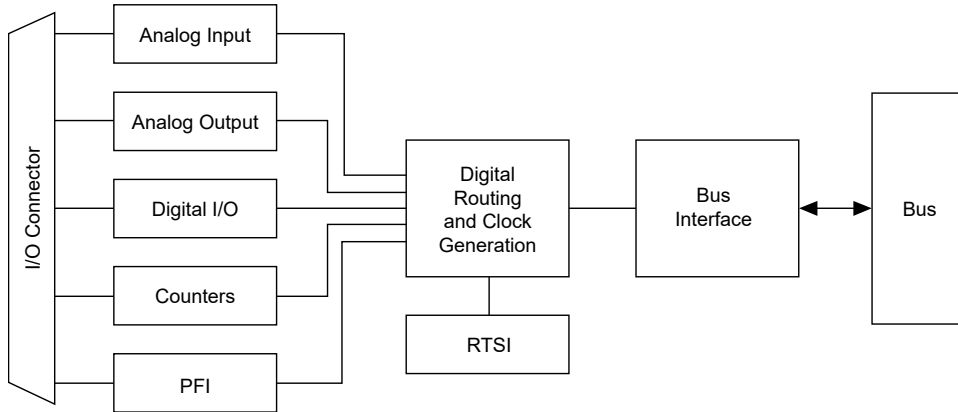
PCIe-6353 and PXIe-6353 Block Diagram

DAQ hardware digitizes signals, performs D/A conversions to generate analog output signals, and measures and controls digital I/O signals.



Note The following figure features components common to all NI 63xx devices. Some individual models do not support specific capabilities.

Figure 1. General PCIe-6353 and PXIe-6353 Device Block Diagram



DAQ-STC3

The DAQ-STC3 and DAQ-6202 implement a high-performance digital engine for X Series data acquisition hardware. Some key features of this engine include the following:

- Flexible AI and AO sample and convert timing
- Many triggering modes
- Independent AI, AO, DI, DO, and counter FIFOs
- Generation and routing of RTSI signals for multi-device synchronization
- Generation and routing of internal and external timing signals
- Four flexible 32-bit counter/timer modules with hardware gating
- Digital waveform acquisition and generation
- Static DIO signals
- True 5 V high current drive DO
- DI change detection
- DO watchdog timers
- PLL for clock synchronization
- Seamless interface to signal conditioning accessories
- PCI Express/PXI Express interface
- Independent scatter-gather DMA controllers for all acquisition and generation functions

Finding Examples

NI-DAQmx includes several example applications that demonstrate the functionality of your device and can serve as interactive tools, programming models, and building blocks for your own applications. Access examples depending on your programming environment and the driver APIs you use to control your device.

NI Example Finder

The NI Example Finder is a utility that organizes examples into categories and allows you to browse and search installed examples. For example, search for "DAQmx" to locate all NI multifunction DAQ examples. You can see descriptions and compatible hardware models for each example or see all the examples compatible with one particular hardware model.

To locate examples using the NI Example Finder within LabVIEW or LabWindows/CVI, select **Help » Find Examples** and navigate to **Hardware Input and Output » DAQmx**.

Installed Example Locations

The installation location for NI-DAQmx example programs differs by programming language and development environment. Refer to the following table for information about example program installation locations.

Table 3. Installed NI-DAQmx Example Locations

Software Application	Installed Example Locations
LabVIEW	Help » Find Examples
LabWindows/ CVI	C:\Users\Public\Documents\National Instruments\CVI\Samples\DAQmx
SignalExpress	Program Files\National Instruments\SignalExpress\Examples
ANSI C	Users\Public\Public Documents\National Instruments\NI-DAQ\Examples\DAQmx ANSI C.
.NET 4.0	Users\Public\Public Documents\National Instruments\NI-

Software Application	Installed Example Locations
	DAQ\Examples\DotNET 4.0
.NET 4.5	Users\Public\Public Documents\National Instruments\NI-DAQ\Examples\DotNET 4.5

Common Example Programs

The following NI-DAQmx example programs demonstrate common functions and operations.

The PCIe-6353 and PXIe-6353 uses the NI-DAQmx driver. NI-DAQmx includes a collection of programming examples to help you get started developing an application. You can modify example code and save it in an application. You can use examples to develop a new application or add example code to an existing application.

- **Analog Input Example Programs**—Demonstrates how to acquire analog measurements. Options include continuous or finite acquisitions, software or hardware timing, and simultaneous sampling. You can perform analog input applications through DMA or programmed I/O data transfer mechanisms. Some of the applications also use start and reference pause triggers.
- **Analog Output Example Programs**—Demonstrates how to generate voltage and waveform data. Options include on-demand, continuous or finite output, waveforms, and hardware timing. You can perform these generations through programmed I/O or DMA data transfer mechanisms. Some of the applications also use start triggers and pause triggers.
- **Counter Input Example Programs**—Demonstrates how to acquire counter input data. Options include finite or continuous acquisitions, and reading edge counts, position, pulse-width, or frequency data.
- **Counter Output Example Programs**—Demonstrates how to generate one or more digital pulses using a counter output. Options include finite or continuous output.
- **Digital Input Example Programs**—Demonstrates how to acquire digital input data. Options include continuous or finite output, software timing, change detection, and pipelined sample clock mode.
- **Digital Output Example Programs**—Demonstrates how to generate digital output

data. Options include finite or continuous output, software timing, sample clock, and waveform generation.

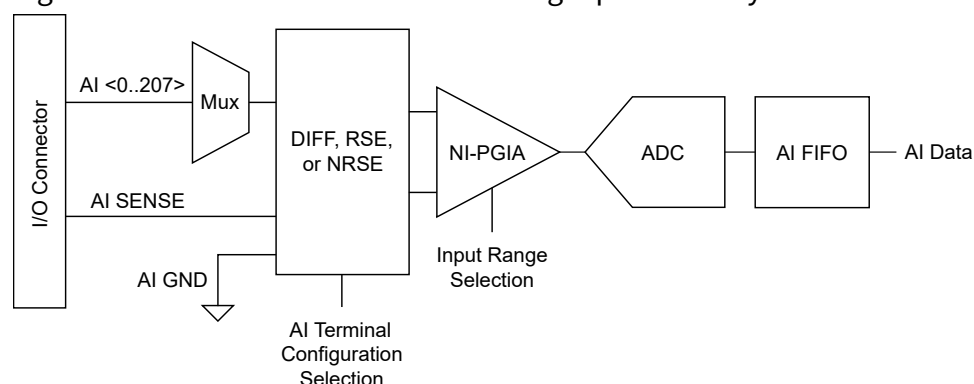
- **Synchronization Example Programs**—Demonstrates how to continuously acquire data. Options include acquiring buffered voltage and thermocouple measurements in a single task, and acquiring waveform data from two modules.

Analog Input for PCIe-6353 and PXIe-6353 Devices

PCIe-6353 and PXIe-6353 devices can be configured for single-ended and differential analog input measurements.

The following figure shows the analog input circuitry of PCIe-6353 and PXIe-6353 devices.

Figure 2. PCIe-6353 and PXIe-6353 Analog Input Circuitry



The main blocks featured in the PCIe-6353 and PXIe-6353 device analog input circuitry are as follows:

- **I/O Connector**—You can connect analog input signals to the PCIe-6353 and PXIe-6353 device through the I/O connector. The proper way to connect analog input signals depends on the analog input ground-reference settings.
- **Mux**—Each PCIe-6353 and PXIe-6353 device has one analog-to-digital converter (ADC). The multiplexers (mux) route one AI channel at a time to the ADC through the NI-PGIA.
- **Ground-Reference Settings**—The analog input ground-reference settings circuitry selects between differential, referenced single-ended, and non-referenced single-ended input modes. Each AI channel can use a different mode.
- **Instrumentation Amplifier (NI-PGIA)**—The NI programmable gain instrumentation amplifier (NI-PGIA) is a measurement and instrument class amplifier that minimizes settling times for all input ranges. The NI-PGIA can amplify or attenuate

an AI signal to ensure that you use the maximum resolution of the ADC.

PCIe-6353 and PXIe-6353 devices use the NI-PGIA to deliver high accuracy even when sampling multiple channels with small input ranges at fast rates.

PCIe-6353 and PXIe-6353 devices can sample channels in any order, and you can individually program each channel in a sample with a different input range.

- **ADC**—The analog-to-digital converter (ADC) digitizes the AI signal by converting the analog voltage into a digital number.
- **AI FIFO**—PCIe-6353 and PXIe-6353 devices can perform both single and multiple A/D conversions of a fixed or infinite number of samples. A large first-in-first-out (FIFO) buffer holds data during AI acquisitions to ensure that no data is lost. PCIe-6353 and PXIe-6353 devices can handle multiple A/D conversion operations with DMA or programmed I/O.

Analog Input Range for PCIe-6353 and PXIe-6353 Devices

Input range refers to the set of input voltages that an analog input channel can digitize with the specified accuracy. The NI-PGIA amplifies or attenuates the AI signal depending on the input range. You can individually program the input range of each AI channel on your PCIe-6353 and PXIe-6353 device.

The input range affects the resolution of the PCIe-6353 and PXIe-6353 device for an AI channel. Resolution refers to the voltage of one ADC code. For example, a 16-bit ADC converts analog inputs into one of 65,536 ($= 2^{16}$) codes—that is, one of 65,536 possible digital values. These values are spread fairly evenly across the input range. So, for an input range of -10 V to 10 V, the voltage of each code of a 16-bit ADC is:

$$\frac{10\text{V} - (-10\text{V})}{2^{16}} = 305\mu\text{V}$$

PCIe-6353 and PXIe-6353 devices use a calibration method that requires some codes (typically about 5% of the codes) to lie outside of the specified range. This calibration method improves absolute accuracy, but it increases the nominal resolution of input ranges by about 5% over what the formula shown above would indicate.

Choose an input range that matches the expected input range of your signal. A large

input range can accommodate a large signal variation, but reduces the voltage resolution. Choosing a smaller input range improves the voltage resolution, but may result in the input signal going out of range.

For more information about setting ranges, refer to the *NI-DAQmx Help*.

The following table shows the supported input ranges and resolutions.

Input Range	Nominal Resolution Assuming 5% Over Range
-10 V to 10 V	320 μ V
-5 V to 5 V	160 μ V
-2 V to 2 V	64 μ V
-1 V to 1 V	32 μ V
-500 mV to 500 mV	16 μ V
-200 mV to 200 mV	6.4 μ V
-100 mV to 100 mV	3.2 μ V

Related information:

- [NI-DAQmx User Manual](#)

Working Voltage Range

On most X Series devices, the PGIA operates normally by amplifying signals of interest while rejecting common-mode signals under certain conditions. If any of these conditions are exceeded, the input voltage is clamped until the fault condition is removed.

- The common-mode voltage (V_{cm}), which is equivalent to subtracting AI GND from AI $\langle 0..x \rangle^-$, must be less than ± 10 V. This V_{cm} is a constant for all range selections.
- The signal voltage (V_s), which is equivalent to subtracting AI $\langle 0..x \rangle^+$ from AI $\langle 0..x \rangle^-$, must be less than or equal to the range selection of the given channel. If V_s is greater than the range selected, the signal clips and information are lost.
- The total working voltage of the positive input, which is equivalent to $(V_{cm} + V_s)$, or

subtracting AI <0..x> GND from AI <0..x>+, must be less than ± 11 V.

Analog Input Ground-Reference Settings

PCIe-6353 and PXIe-6353 devices support the following analog input ground-reference settings:

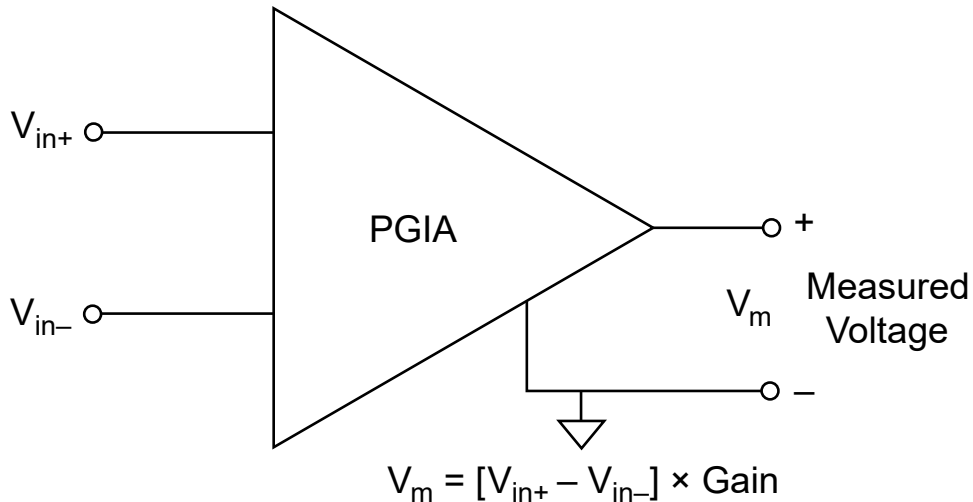
- Differential mode—In DIFF mode, the device measures the difference in voltage between two AI signals.
- Referenced single-ended mode—In RSE mode, the device measures the voltage of an AI signal relative to AI GND.
- Non-referenced single-ended mode—In NRSE mode, the device measures the voltage of an AI signal relative to one of the AI SENSE inputs specific for that channel.

The AI ground-reference setting determines how you should connect your AI signals to the MIO X Series device. Refer to the *Connecting Analog Input Signals* section for more information.

Ground-reference settings are programmed on a per-channel basis. For example, you might configure the device to scan 12 channels—four differentially-configured channels and eight single-ended channels.

PCIe-6353 and PXIe-6353 devices implement the different analog input ground-reference settings by routing different signals to the NI-PGIA. The NI-PGIA is a differential amplifier. That is, the NI-PGIA amplifies (or attenuates) the difference in voltage between its two inputs. The NI-PGIA drives the ADC with this amplified voltage. The amount of amplification (the gain), is determined by the analog input range, as shown in the following figure.

Figure 3. PCIe-6353 and PXIe-6353 Device NI-PGIA



The following table shows how signals are routed to the NI-PGIA on PCIe-6353 and PXIe-6353 devices.

Table 4. Signals Routed to the NI-PGIA on PCIe-6353 and PXIe-6353 Devices

AI Ground-Reference Settings	Signals Routed to the Positive Input of the NI-PGIA (V _{in+})	Signals Routed to the Negative Input of the NI-PGIA (V _{in-})
RSE	AI <0..31>	AI GND
NRSE	AI <0..15>	AI SENSE
	AI <16..31>	AI SENSE 2
DIFF	AI <0..7>	AI <8..15>
	AI <16..23>	AI <24..31>

For differential measurements, AI 0 and AI 8 are the positive and negative inputs of differential analog input channel 0. Similarly, the following signal pairs also form differential input channels: AI <1,9>, AI <2,10>, AI <3,11>, AI <4,12>, AI <5,13>, AI <6,14>, AI <7,15>, AI <16,24>, AI <17,25>, AI <18,26>, AI <19,27>, AI <20,28>, AI <21,29>, AI <22,30>, AI <23,31> and so on.

The maximum input voltages rating of AI signals with respect to ground (and for signal pairs in differential mode with respect to each other) are listed in the device specifications. Exceeding the maximum input voltage of AI signals distorts the measurement results. Exceeding the maximum input voltage rating can also damage the device and the computer. NI is *not* liable for any damage resulting from such signal

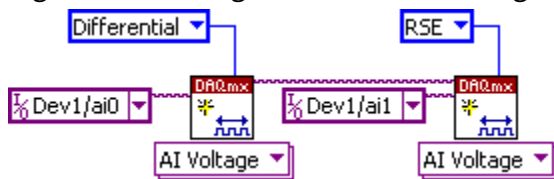
connections. AI ground-reference setting is sometimes referred to as AI terminal configuration.

Configuring AI Ground-Reference Settings in Software

You can program channels on an PCIe-6353 and PXIe-6353 device to acquire with different ground references.

To enable multimode scanning in LabVIEW, use the NI-DAQmx Create Virtual Channel.vi of the NI-DAQmx API. You must use a new VI for each channel or group of channels configured in a different input mode. In the following figure, channel 0 is configured in differential mode, and channel 1 is configured in RSE mode.

Figure 4. Enabling Multimode Scanning in LabVIEW



To configure the input mode of your voltage measurement using the DAQ Assistant, use the Terminal Configuration drop-down list. Refer to the *DAQ Assistant Help* for more information.

To configure the input mode of your voltage measurement using the NI-DAQmx C API, set the terminalConfig property. Refer to the *NI-DAQmx C Reference Help* for more information.

Multichannel Scanning Considerations

PCIe-6353 and PXIe-6353 devices can scan multiple channels at high rates and digitize the signals accurately. However, you should consider several issues when designing your measurement system to ensure the high accuracy of your measurements.

In multichannel scanning applications, accuracy is affected by settling time. When your PCIe-6353 and PXIe-6353 device switches from one AI channel to another AI channel, the device configures the NI-PGIA with the input range of the new channel. The NI-PGIA then amplifies the input signal with the gain for the new input range.

Settling time refers to the time it takes the NI-PGIA to amplify the input signal to the desired accuracy before it is sampled by the ADC. To determine your device settling time, refer to the device specifications.

PCIe-6353 and PXIe-6353 devices are designed to have fast settling times. However, several factors can increase the settling time which decreases the accuracy of your measurements. To ensure fast settling times, you should do the following (in order of importance):

1. Use Low Impedance Sources—To ensure fast settling times, your signal sources should have an impedance of $<1\text{ k}\Omega$. Large source impedances increase the settling time of the NI-PGIA, and so decrease the accuracy at fast scanning rates.

Settling times increase when scanning high-impedance signals due to a phenomenon called charge injection. Multiplexers contain switches, usually made of switched capacitors. When one of the channels, for example channel 0, is selected in a multiplexer, those capacitors accumulate charge. When the next channel, for example channel 1, is selected, the accumulated charge leaks backward through channel 1. If the output impedance of the source connected to channel 1 is high enough, the resulting reading of channel 1 can be partially affected by the voltage on channel 0. This effect is referred to as ghosting.

If your source impedance is high, you can decrease the scan rate to allow the NI-PGIA more time to settle. Another option is to use a voltage follower circuit external to your DAQ device to decrease the impedance seen by the DAQ device. Refer to the document, [Eliminate Ghosting on Adjacent Input Channels by Decreasing Source Impedance](#), by going to ni.com/info and entering the Info Code rdbbis.

2. Use Short High-Quality Cabling—Using short high-quality cables can minimize several effects that degrade accuracy including crosstalk, transmission line effects, and noise. The capacitance of the cable can also increase the settling time.

National Instruments recommends using individually shielded, twisted-pair wires that are 2 m or less to connect AI signals to the device. Refer to the [Connecting Analog Input Signals](#) section for more information.

3. Carefully Choose the Channel Scanning Order

- Avoid Switching from a Large to a Small Input Range—Switching from a channel with a large input range to a channel with a small input range can greatly increase the settling time.

Suppose a 4 V signal is connected to channel 0 and a 1 mV signal is connected to channel 1. The input range for channel 0 is -10 V to 10 V and the input range of channel 1 is -200 mV to 200 mV.

When the multiplexer switches from channel 0 to channel 1, the input to the NI-PGIA switches from 4 V to 1 mV. The approximately 4 V step from 4 V to 1 mV is 1,000% of the new full-scale range. For a 16-bit device to settle within 0.0015% (15 ppm or 1 LSB) of the ± 200 mV full-scale range on channel 1, the input circuitry must settle to within 0.000031% (0.31 ppm or 1/50 LSB) of the ± 10 V range. Some devices can take many microseconds for the circuitry to settle this much.

To avoid this effect, you should arrange your channel scanning order so that transitions from large to small input ranges are infrequent.

In general, you do not need this extra settling time when the NI-PGIA is switching from a small input range to a larger input range.

- Insert Grounded Channel between Signal Channels—Another technique to improve settling time is to connect an input channel to ground. Then insert this channel in the scan list between two of your signal channels. The input range of the grounded channel should match the input range of the signal after the grounded channel in the scan list.

Consider again the example above where a 4 V signal is connected to channel 0 and a 1 mV signal is connected to channel 1. Suppose the input range for channel 0 is -10 V to 10 V and the input range of channel 1 is -200 mV to 200 mV.

You can connect channel 2 to AI GND (or you can use the internal ground; refer to Internal Channels in the NI-DAQmx Help). Set the input range of channel 2 to -200 mV to 200 mV to match channel 1. Then scan channels in the order: 0, 2, 1.

Inserting a grounded channel between signal channels improves settling time because the NI-PGIA adjusts to the new input range setting faster when the

input is grounded.

- **Minimize Voltage Step between Adjacent Channels**—When scanning between channels that have the same input range, the settling time increases with the voltage step between the channels. If you know the expected input range of your signals, you can group signals with similar expected ranges together in your scan list.

For example, suppose all channels in a system use a -5 V to 5 V input range. The signals on channels 0, 2, and 4 vary between 4.3 V and 5 V. The signals on channels 1, 3, and 5 vary between -4 V and 0 V. Scanning channels in the order 0, 2, 4, 1, 3, 5 produces more accurate results than scanning channels in the order 0, 1, 2, 3, 4, 5.

4. **Avoid Scanning Faster Than Necessary**—Designing your system to scan at slower speeds gives the NI-PGIA more time to settle to a more accurate level. Here are two examples to consider:

- **Example 1**—Averaging many AI samples can increase the accuracy of the reading by decreasing noise effects. In general, the more points you average, the more accurate the final result. However, you may choose to decrease the number of points you average and slow down the scanning rate.

Suppose you want to sample 10 channels over a period of 20 ms and average the results. You could acquire 500 points from each channel at a scan rate of 250 kS/s. Another method would be to acquire 1,000 points from each channel at a scan rate of 500 kS/s. Both methods take the same amount of time. Doubling the number of samples averaged (from 500 to 1,000) decreases the effect of noise by a factor of 1.4 (the square root of 2). However, doubling the number of samples (in this example) decreases the time the NI-PGIA has to settle from 4 μ s to 2 μ s. In some cases, the slower scan rate system returns more accurate results.

- **Example 2**—If the time relationship between channels is not critical, you can sample from the same channel multiple times and scan less frequently. For example, suppose an application requires averaging 100 points from channel 0 and averaging 100 points from channel 1. You could alternate reading between channels—that is, read one point from channel 0, then one point from channel 1, and so on. You also could read all 100 points from channel 0 then

read 100 points from channel 1. The second method switches between channels much less often and is affected much less by settling time.

Related reference:

- [Connecting Analog Input Signals](#)

Analog Input Data Acquisition Methods

When performing analog input measurements, you either can perform software-timed or hardware-timed acquisitions.

Software-timed Acquisitions

With a software-timed acquisition, software controls the rate of the acquisition. Software sends a separate command to the hardware to initiate each ADC conversion. In NI-DAQmx, software-timed acquisitions are referred to as having on-demand timing. Software-timed acquisitions are also referred to as immediate or static acquisitions and are typically used for reading a single sample of data.

Hardware-Timed Acquisitions

With hardware-timed acquisitions, a digital hardware signal (AI Sample Clock) controls the rate of the acquisition. This signal can be generated internally on your device or provided externally.

Hardware-timed acquisitions have several advantages over software-timed acquisitions:

- The time between samples can be much shorter.
- The timing between samples is deterministic.
- Hardware-timed acquisitions can use hardware triggering.

The PCIe-6353 and PXIe-6353 supports buffered and hardware-timed single point (HWTSP) hardware-timed operations.

A buffer is a temporary storage in computer memory for to-be-transferred samples. Data is moved from the DAQ device's onboard FIFO memory to a PC buffer using DMA

before it is transferred to application memory. Buffered acquisition typically allow for much faster transfer rates because data is moved in large blocks.

One property of buffered I/O operations is the sample mode. The sample mode can be either finite or continuous:

- Finite sample mode acquisition refers to the acquisition of a specific, predetermined number of data samples. Once the specified number of samples has been read in, the acquisition stops. If you use a reference trigger, you must use finite sample mode.
- Continuous acquisition refers to the acquisition of an unspecified number of samples. Instead of acquiring a set number of data samples and stopping, a continuous acquisition continues until you stop the operation. Continuous acquisition is also referred to as double-buffered or circular-buffered acquisition.

If data cannot be transferred across the bus fast enough, the FIFO becomes full. New acquisitions overwrite data in the FIFO before it can be transferred to host memory, which causes the device to generate an error. With continuous operations, if the user program does not read data out of the PC buffer fast enough to keep up with the data transfer, the buffer could reach an overflow condition, causing an error to be generated.

Typically, HWTSP operations are used to write single samples at known time intervals. While buffered operations are optimized for high throughput, HWTSP operations are optimized for low latency and low jitter. In addition, HWTSP can notify software if it falls behind hardware. These features make HWTSP ideal for real time control applications. HWTSP operations, in conjunction with the wait for next sample clock function, provide tight synchronization between the software layer and the hardware layer. Refer to the NI-DAQmx Hardware-Timed Single Point Lateness Checking document for more information. To access this document, go to ni.com/info and enter the Info Code daqhwtsp.

Getting Started with AI Applications in Software

You can use the PCIe-6353 and PXIe-6353 device in the following analog input applications:

- Single-point analog input (on demand)
- Finite analog input
- Continuous analog input
- Hardware-timed single point

You can perform these applications through DMA or programmed I/O data transfer mechanisms. Some of the applications also use start, reference, and pause triggers.



Note For more information about programming analog input applications and triggers in software, refer to the *NI-DAQmx Help*.

X Series devices use the NI-DAQmx driver. NI-DAQmx includes a collection of programming examples to help you get started developing an application. You can modify example code and save it in an application. You can use examples to develop a new application or add example code to an existing application.

To locate examples, refer to the document *Where Are NI-DAQmx Examples Installed in Windows?* by going to ni.com/info and entering the Info Code `daqmxexp`. For additional examples, refer to ni.com/examples.

Related reference:

- [Finding Examples](#)

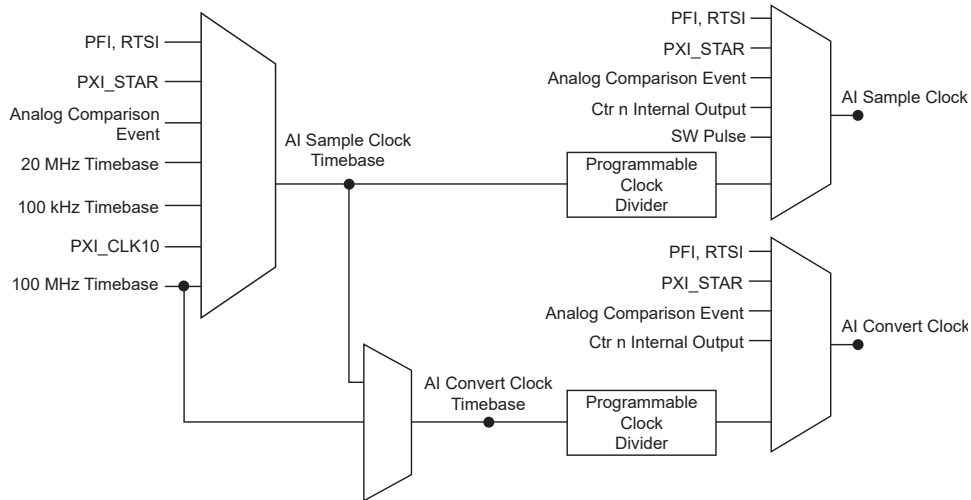
Related information:

- [NI-DAQmx User Manual](#)

Analog Input Timing Signals

To provide timing functionality, PCIe-6353 and PXIe-6353 devices have a flexible timing engine. The following figure summarizes all of the timing options provided by the analog input timing engine. Also refer to the *Clock Routing* topic.

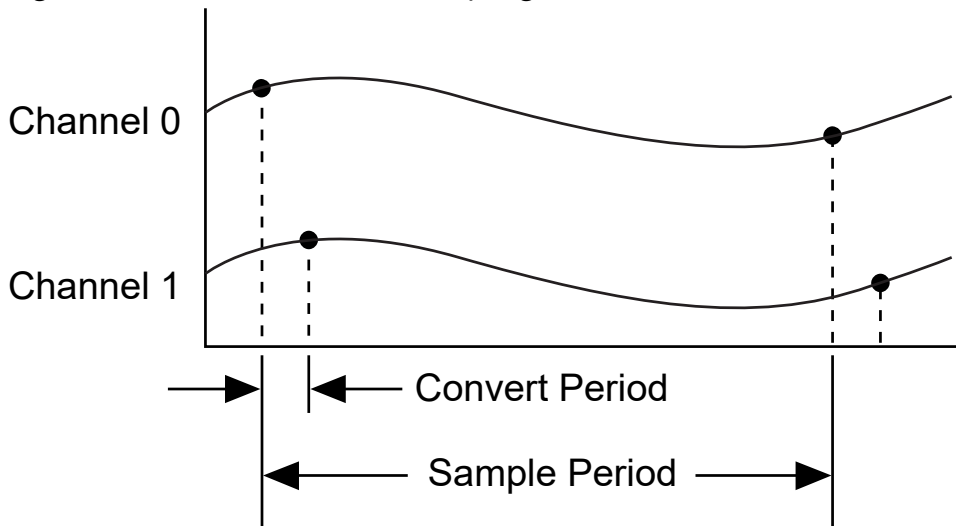
Figure 5. Analog Input Timing Options



PCIe-6353 and PXIe-6353 devices use AI Sample Clock (ai/SampleClock) and AI Convert Clock (ai/ConvertClock) to perform interval sampling. As the following figure shows, AI Sample Clock controls the sample period, which is determined by the following equation:

$$1/\text{Sample Period} = \text{Sample Rate}$$

Figure 6. MIO X Series Interval Sampling

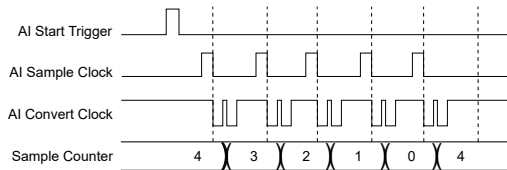


AI Convert Clock controls the Convert Period, which is determined by the following equation:

$$1/\text{Convert Period} = \text{Convert Rate}$$

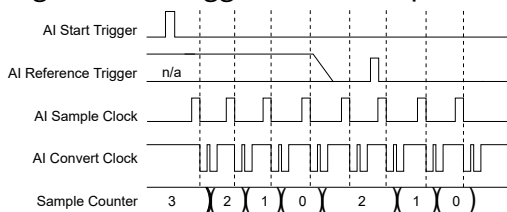
Posttriggered data acquisition allows you to view only data that is acquired after a trigger event is received. A typical posttriggered DAQ sequence is shown in the following figure. The sample counter is loaded with the specified number of posttrigger samples, in this example, five. The value decrements with each pulse on AI Sample Clock, until all desired samples have been acquired.

Figure 7. Posttriggered Data Acquisition Example



Pretriggered data acquisition allows you to view data that is acquired before the trigger of interest, in addition to data acquired after the trigger. The following figure shows a typical pretriggered DAQ sequence. AI Start Trigger (ai/StartTrigger) can be either a hardware or software signal. If AI Start Trigger is set up to be a software start trigger, an output pulse appears on the ai/StartTrigger line when the acquisition begins. When the AI Start Trigger pulse occurs, the sample counter is loaded with the number of pretriggered samples, in this example, four. The value decrements with each pulse on AI Sample Clock. The sample counter is then loaded with the number of posttriggered samples, in this example, three.

Figure 8. Pretriggered Data Acquisition Example



If an AI Reference Trigger (ai/ReferenceTrigger) pulse occurs before the specified number of pretrigger samples are acquired, the trigger pulse is ignored. Otherwise, when the AI Reference Trigger pulse occurs, the sample counter value decrements until the specified number of posttrigger samples have been acquired.

PCIe-6353 and PXIe-6353 devices feature the following analog input timing signals:

- *AI Sample Clock Signal**
- *AI Sample Clock Timebase Signal*
- *AI Convert Clock Signal**

- *AI Convert Clock Timebase Signal*
- *AI Hold Complete Event Signal*
- *AI Start Trigger Signal**
- *AI Reference Trigger Signal**
- *AI Pause Trigger Signal**

Signals with an * support digital filtering. Refer to the *PFI Filters* topic for more information.

Aggregate versus Single Channel Sample Rates

PCIe-6353 and PXIe-6353 devices are characterized with maximum single channel and maximum aggregate sample rates. The maximum single channel rate is the fastest you can acquire data on the device from a single channel and still achieve accurate results. The maximum aggregate sample rate is the fastest you can acquire on multiple channels and still achieve accurate results.

AI Sample Clock Signal

Use the AI Sample Clock (ai/SampleClock) signal to initiate a set of measurements. Your MIO X Series device samples the AI signals of every channel in the task once for every AI Sample Clock. A measurement acquisition consists of one or more samples.

You can specify an internal or external source for AI Sample Clock. You can also specify whether the measurement sample begins on the rising edge or falling edge of AI Sample Clock.



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Using an Internal Source

One of the following internal signals can drive AI Sample Clock:

- Counter *n* Internal Output
- AI Sample Clock Timebase (divided down)

- A pulse initiated by host software
- Change Detection Event
- Counter n Sample Clock
- AO Sample Clock (ao/SampleClock)
- DI Sample Clock (di/SampleClock)
- DO Sample Clock (do/SampleClock)

A programmable internal counter divides down the sample clock timebase.

Several other internal signals can be routed to AI Sample Clock through internal routes. Refer to *Device Routing in MAX* in the *NI-DAQmx Help*.

Using an External Source

Use one of the following external signals as the source of AI Sample Clock:

- PFI <0..15>
- RTSI <0..7>
- PXI_STAR
- PXIe_DSTAR<A,B>
- Analog Comparison Event (an analog trigger)

Routing AI Sample Clock Signal to an Output Terminal

You can route AI Sample Clock out to any PFI <0..15>, RTSI <0..7>, or PXIe_DSTAR terminal. This pulse is always active high.

All PFI terminals are configured as inputs by default.

Related concepts:

- [Device Routing in MAX](#)

Other Timing Requirements

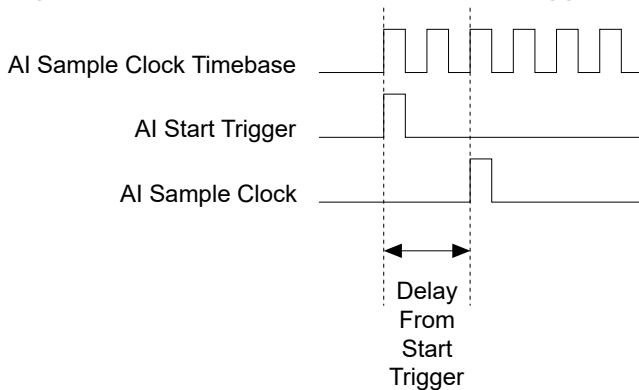
Your DAQ device only acquires data during an acquisition. The device ignores AI Sample Clock when a measurement acquisition is not in progress. During a measurement acquisition, you can cause your DAQ device to ignore AI Sample Clock

using the AI Pause Trigger signal.

A counter/timing engine on your device internally generates AI Sample Clock unless you select some external source. AI Start Trigger starts this counter and either software or hardware can stop it once a finite acquisition completes. When using the AI timing engine, you can also specify a configurable delay from AI Start Trigger to the first AI Sample Clock pulse. By default, this delay is set to four ticks of the AI Sample Clock Timebase signal.

When using an externally generated AI Sample Clock, you must ensure the clock signal is consistent with respect to the timing requirements of AI Convert Clock. Failure to do so may result in a scan overrun and will cause an error. Refer to the *AI Convert Clock Signal* topic for more information about the timing requirements between AI Convert Clock and AI Sample Clock. The following figure shows the relationship of AI Sample Clock to AI Start Trigger.

Figure 9. AI Sample Clock and AI Start Trigger



Related concepts:

- [Device Routing in MAX](#)

AI Sample Clock Timebase Signal

You can route any of the following signals to be the AI Sample Clock Timebase (ai/SampleClockTimebase) signal:



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe

backplane.

- 100 MHz Timebase (default)
- 20 MHz Timebase
- 100 kHz Timebase
- PFI <0..15>
- RTSI <0..7>
- PXI_STAR
- PXIe_DSTAR<A,B>
- PXI_CLK10
- Analog Comparison Event (an analog trigger)

AI Sample Clock Timebase is not available as an output on the I/O connector. AI Sample Clock Timebase is divided down to provide one of the possible sources for AI Sample Clock. You can configure the polarity selection for AI Sample Clock Timebase as either rising or falling edge, except on 100 MHz Timebase or 20 MHz Timebase.

AI Convert Clock Signal

Use the AI Convert Clock (ai/ConvertClock) signal to initiate a single A/D conversion on a single channel. A sample (controlled by the AI Sample Clock) consists of one or more conversions. You can specify either an internal or external signal as the source of AI Convert Clock. You can also specify whether the measurement sample begins on the rising edge or falling edge of AI Convert Clock.

With NI-DAQmx, the driver chooses the fastest conversion rate possible based on the speed of the A/D converter and adds 10 μ s of padding between each channel to allow for adequate settling time. This scheme enables the channels to approximate simultaneous sampling and still allow for adequate settling time. If the AI Sample Clock rate is too fast to allow for this 10 μ s of padding, NI-DAQmx chooses the conversion rate so that the AI Convert Clock pulses are evenly spaced throughout the sample.

To explicitly specify the conversion rate, use AI Convert Clock Rate DAQmx Timing property node or function.



Notice Setting the conversion rate higher than the maximum rate specified for your device will result in errors.

Using an Internal Source

One of the following internal signals can drive AI Convert Clock:

- AI Convert Clock Timebase (divided down)
- Counter n Internal Output
- Change Detection Event
- Counter n Sample Clock
- AO Sample Clock (ao/SampleClock)
- DI Sample Clock (di/SampleClock)
- DO Sample Clock (do/SampleClock)

A programmable internal counter divides down the AI Convert Clock Timebase to generate AI Convert Clock. The counter is started by AI Sample Clock and continues to count down to zero, produces an AI Convert Clock, reloads itself, and repeats the process until the sample is finished. It then reloads itself in preparation for the next AI Sample Clock pulse.

Several other internal signals can be routed to AI Convert Clock through internal routes. Refer to *Device Routing in MAX* in the *NI-DAQmx Help* for more information.



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Using an External Source

Use one of the following external signals as the source of AI Convert Clock:

- PFI <0..15>
- PXI_STAR
- PXIe_DSTAR<A,B>
- RTSI <0..7>
- Analog Comparison Event (an analog trigger)

Routing AI Convert Clock Signal to an Output Terminal

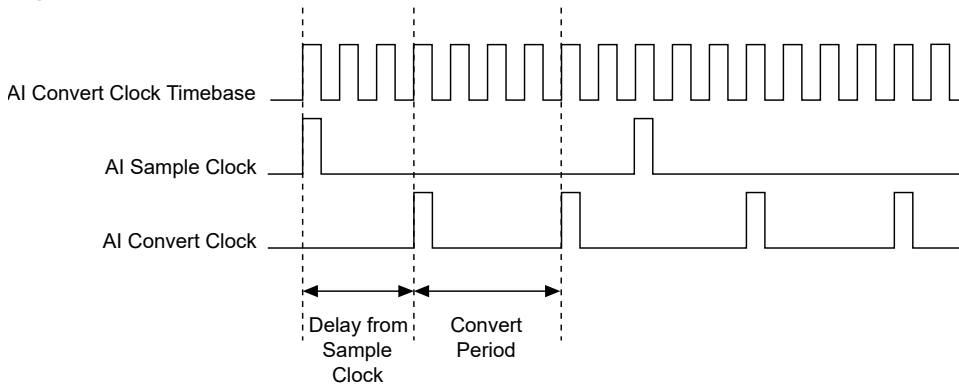
You can route AI Convert Clock (as an active low signal) out to any PFI <0..15>, RTSI <0..7>, or PXIe_DSTARC terminal.

All PFI terminals are configured as inputs by default.

Using a Delay from Sample Clock to Convert Clock

When using the AI timing engine to generate your Convert Clock, you can also specify a configurable delay from AI Sample Clock to the first AI Convert Clock pulse within the sample. By default, this delay is three ticks of AI Convert Clock Timebase. The following figure shows the relationship of AI Sample Clock to AI Convert Clock.

Figure 10. AI Sample Clock and AI Convert Clock



Other Timing Requirements

The sample and conversion level timing of MIO X Series devices work such that some clock signals are gated off unless the proper timing requirements are met. For example, the device ignores both AI Sample Clock and AI Convert Clock until it receives a valid AI Start Trigger signal. Similarly, the device ignores all AI Convert Clock pulses until it recognizes an AI Sample Clock pulse. Once the device receives the correct number of AI Convert Clock pulses, it ignores subsequent AI Convert Clock pulses until it receives another AI Sample Clock. However, after the device recognizes an AI Sample Clock pulse, it causes an error if it receives an AI Sample Clock pulse before the correct number of AI Convert Clock pulses are received.

The following figures show timing sequences for a four-channel acquisition (using AI channels 0, 1, 2, and 3) and demonstrate proper and improper sequencing of AI

Sample Clock and AI Convert Clock.

Figure 11. Scan Overrun Condition; AI Sample Clock Too Fast For Convert Clock Causes an Error

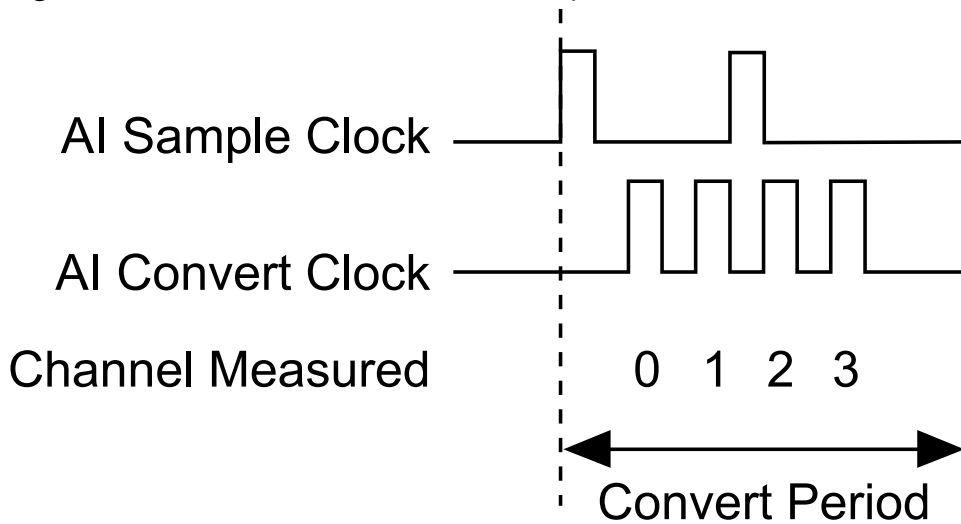


Figure 12. AI Convert Clock Too Fast For AI Sample Clock; AI Convert Clock Pulses Are Ignored

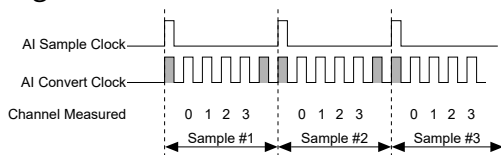


Figure 13. AI Sample Clock and AI Convert Clock Improperly Matched; Leads to Aperiodic Sampling

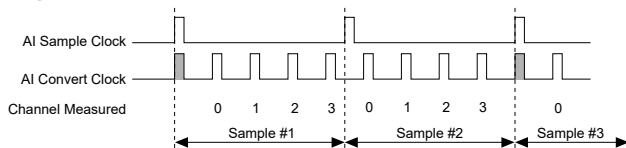
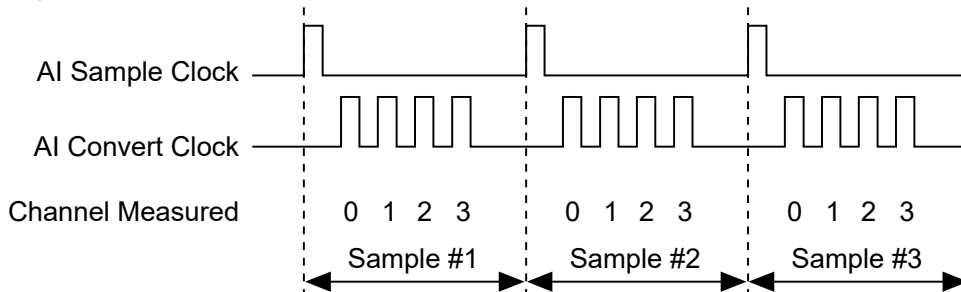


Figure 14. AI Sample Clock and AI Convert Clock Properly Matched



Related concepts:

- [Device Routing in MAX](#)

AI Convert Clock Timebase Signal

The AI Convert Clock Timebase (ai/ConvertClockTimebase) signal is divided down to

provide one of the possible sources for AI Convert Clock. Use one of the following signals as the source of AI Convert Clock Timebase:

- AI Sample Clock Timebase
- 100 MHz Timebase

AI Convert Clock Timebase is not available as an output on the I/O connector.

AI Hold Complete Event Signal

The AI Hold Complete Event (ai/HoldCompleteEvent) signal generates a pulse after each A/D conversion begins. You can route AI Hold Complete Event out to any PFI <0..15>, RTSI <0..7>, or PXIe_DSTARC terminal.



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

The polarity of AI Hold Complete Event is software-selectable, but is typically configured so that a low-to-high leading edge can clock external AI multiplexers indicating when the input signal has been sampled and can be removed.

AI Start Trigger Signal

Use the AI Start Trigger (ai/StartTrigger) signal to begin a measurement acquisition. A measurement acquisition consists of one or more samples. If you do not use triggers, begin a measurement with a software command.

Once the acquisition begins, configure the acquisition to stop:

- When a certain number of points are sampled (in finite mode)
- After a hardware reference trigger (in finite mode)
- With a software command (in continuous mode)

An acquisition that uses a start trigger (but not a reference trigger) is sometimes referred to as a posttriggered acquisition.

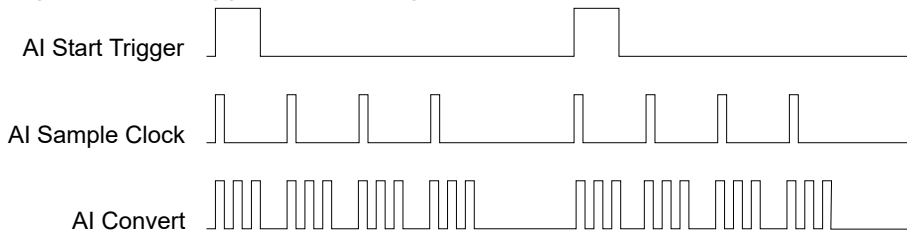


Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Retriggerable Analog Input

The AI Start Trigger is configurable as retriggerable. When the AI Start Trigger is configured as retriggerable, the timing engine generates the sample and convert clocks for the configured acquisition in response to each pulse on an AI Start Trigger signal. The timing engine ignores the AI Start Trigger signal while the clock generation is in progress. After the clock generation is finished, the counter waits for another Start Trigger to begin another clock generation. The following figure shows a retriggerable analog input with three AI channels and four samples per trigger.

Figure 15. Retriggerable Analog Input



Note Waveform information from LabVIEW does not reflect the delay between triggers. They are treated as a continuous acquisition with constant t_0 and dt information.

Reference triggers are not retriggerable.

Using a Digital Source

To use AI Start Trigger with a digital source, specify a source and an edge. The source can be any of the following signals:

- Counter n Internal Output
- Change Detection Event
- AO Start Trigger (ao/StartTrigger)
- DI Start Trigger (di/StartTrigger)
- DO Start Trigger (do/StartTrigger)
- PFI <0..15>

- PXI_STAR
- PXIe_DSTAR<A,B>
- RTSI <0..7>

The source can also be one of several other internal signals on your DAQ device. Refer to *Device Routing in MAX* in the *NI-DAQmx Help* for more information.

You can also specify whether the measurement acquisition begins on the rising edge or falling edge of AI Start Trigger.

Using an Analog Source

When you use an analog trigger source, the acquisition begins on the first rising edge of the Analog Comparison Event signal.

Routing AI Start Trigger to an Output Terminal

You can route AI Start Trigger out to any PFI <0..15>, RTSI <0..7>, or PXIe_DSTARC terminal. The output is an active high pulse. All PFI terminals are configured as inputs by default.

The device also uses AI Start Trigger to initiate pretriggered DAQ operations. In most pretriggered applications, a software trigger generates AI Start Trigger. Refer to the *AI Reference Trigger Signal* topic for a complete description of the use of AI Start Trigger and AI Reference Trigger in a pretriggered DAQ operation.

Related concepts:

- [Device Routing in MAX](#)

AI Reference Trigger Signal

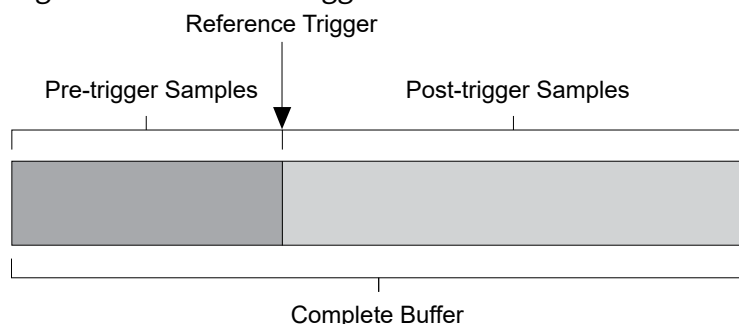
Use AI Reference Trigger (ai/ReferenceTrigger) signal to stop a measurement acquisition. To use a reference trigger, specify a buffer of finite size and a number of pretrigger samples (samples that occur before the reference trigger). The number of posttrigger samples (samples that occur after the reference trigger) desired is the buffer size minus the number of pretrigger samples.

Once the acquisition begins, the DAQ device writes samples to the buffer. After the DAQ device captures the specified number of pretrigger samples, the DAQ device begins to look for the reference trigger condition. If the reference trigger condition occurs before the DAQ device captures the specified number of pretrigger samples, the DAQ device ignores the condition.

If the buffer becomes full, the DAQ device continuously discards the oldest samples in the buffer to make space for the next sample. This data can be accessed (with some limitations) before the DAQ device discards it. Refer to the document, *Can a Pretriggered Analog Acquisition be Continuous?*, for more information. To access this document, go to ni.com/info and enter the Info Code `rdcanq`.

When the reference trigger occurs, the DAQ device continues to write samples to the buffer until the buffer contains the number of posttrigger samples desired. The following figure shows the final buffer.

Figure 16. Reference Trigger Final Buffer



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Using a Digital Source

To use AI Reference Trigger with a digital source, specify a source and an edge. The source can be any of the following signals:

- PFI <0..15>
- PXI_STAR
- PXIe_DSTAR<A,B>
- RTSI <0..7>

- Change Detection Event
- Counter n Internal Output
- DI Reference Trigger (di/ReferenceTrigger)
- AO Start Trigger (ao/StartTrigger)
- DO Start Trigger (do/StartTrigger)

The source can also be one of several internal signals on your DAQ device. Refer to Device Routing in MAX in the *NI-DAQmx Help* for more information.

You can also specify whether the measurement acquisition stops on the rising edge or falling edge of AI Reference Trigger.

Using an Analog Source

When you use an analog trigger source, the acquisition stops on the first rising edge of the Analog Comparison Event signal.

Routing AI Reference Trigger Signal to an Output Terminal

You can route AI Reference Trigger out to any PFI <0..15>, RTSI <0..7>, PXI_TRIG <0..7>, or PXIe_DSTARC terminal.

All PFI terminals are configured as inputs by default.

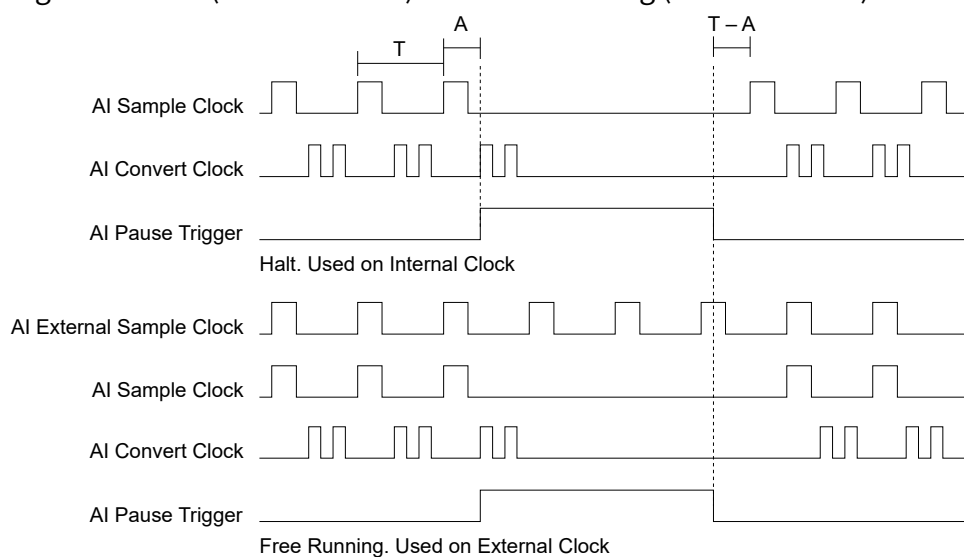
Related concepts:

- [Device Routing in MAX](#)

AI Pause Trigger Signal

Use the AI Pause Trigger (ai/PauseTrigger) signal to pause and resume a measurement acquisition. The internal sample clock pauses while the external trigger signal is active and resumes when the signal is inactive. You can program the active level of the pause trigger to be high or low, as shown in the following figure. In the figure, T represents the period, and A represents the unknown time between the clock pulse and the posttrigger.

Figure 17. Halt (Internal Clock) and Free Running (External Clock)



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Using a Digital Source

To use AI Pause Trigger, specify a source and a polarity. The source can be one of the following signals:

- PFI <0..15>
- PXI_STAR
- PXIe_DSTAR<A,B>
- RTSI <0..7>
- Counter *n* Internal Output
- Counter *n* Gate
- AO Pause Trigger (ao/PauseTrigger)
- DI Pause Trigger (di/PauseTrigger)
- DO Pause Trigger (do/PauseTrigger)

The source can also be one of several other internal signals on your DAQ device. Refer to *Device Routing in MAX* in the *NI-DAQmx Help* for more information.

Using an Analog Source

When you use an analog trigger source, the internal sample clock pauses when the Analog Comparison Event signal is low and resumes when the signal goes high (or vice versa).

Routing AI Pause Trigger Signal to an Output Terminal

You can route AI Pause Trigger out to any PFI <0..15>, RTSI <0..7>, PXI_STAR, or PXIe_DSTARC terminal.



Note Pause triggers are only sensitive to the level of the source, not the edge.

Related concepts:

- [Device Routing in MAX](#)

Analog Input Common Questions

This section contains common questions about X Series devices. If your questions are not answered here, refer to ni.com/support.

I am seeing crosstalk, or ghost voltages, when sampling multiple channels. What does this mean?

You may be experiencing a phenomenon called charge injection, which occurs when you sample a series of high-output impedance sources with a multiplexer. Multiplexers contain switches, usually made of switched capacitors. When a channel, for example AI 0, is selected in a multiplexer, those capacitors accumulate charge. When the next channel, for example AI 1, is selected, the accumulated current (or charge) leaks backward through channel 1. If the output impedance of the source connected to AI 1 is high enough, the resulting reading can somewhat affect the voltage in AI 0. To circumvent this problem, use a voltage follower that has operational amplifiers (op-amps) with unity gain for each high-impedance source before connecting to an X Series device. Otherwise, you must decrease the sample rate for each channel.

Another common cause of channel crosstalk is due to sampling among multiple channels at various gains. In this situation, the settling times can increase. For more information about charge injection and sampling channels at different gains, refer to the *Multichannel Scanning Considerations* topic.

I am using my device in differential analog input ground-reference mode and I have connected a differential input signal, but my readings are random and drift rapidly. What is wrong?

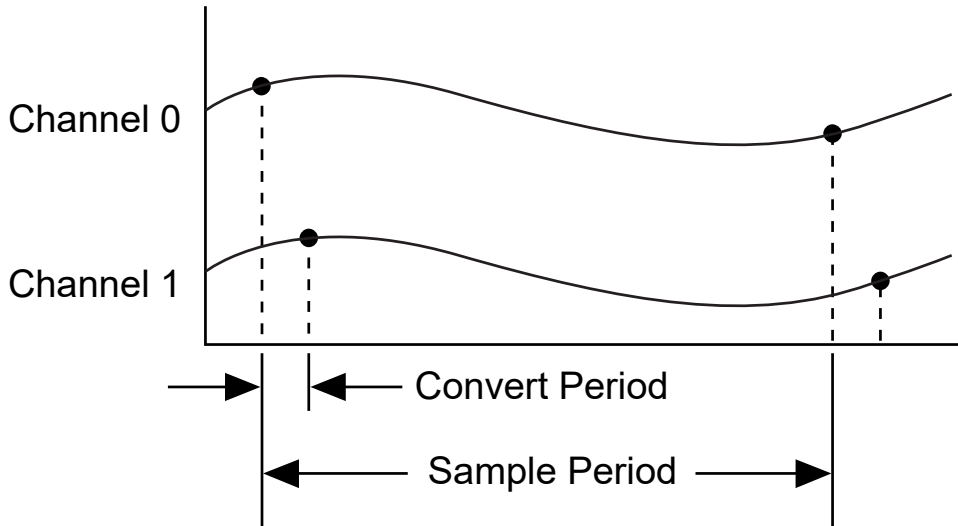
In DIFF mode, if the readings from the DAQ device are random and drift rapidly, you should check the ground-reference connections. The signal can be referenced to a level that is considered floating with reference to the device ground reference. Even if you are in DIFF mode, you must still reference the signal to the same ground level as the device reference. There are various methods of achieving this reference while maintaining a high common-mode rejection ratio (CMRR). These methods are outlined in the *Connecting Analog Input Signals* topic.

AI GND is an AI common signal that routes directly to the ground connection point on the devices. You can use this signal if you need a general analog ground connection point to the device. Refer to the *When to Use Differential Connections with Ground-Referenced Signal Sources* section of the *Connecting Ground-Referenced Signal Sources* topic for more information.

How can I use the AI Sample Clock and AI Convert Clock signals on an MIO X Series device to sample the AI channel(s)?

MIO X Series devices use AI Sample Clock (ai/SampleClock) and AI Convert Clock (ai/ConvertClock) to perform interval sampling. As the following figure shows, AI Sample Clock controls the sample period, which is determined by the following equation: $1/\text{sample period} = \text{sample rate}$

Figure 18. AI Sample Clock and AI Convert Clock



AI Convert Clock controls the convert period, which is determined by the following equation: $1/\text{convert period} = \text{convert rate}$

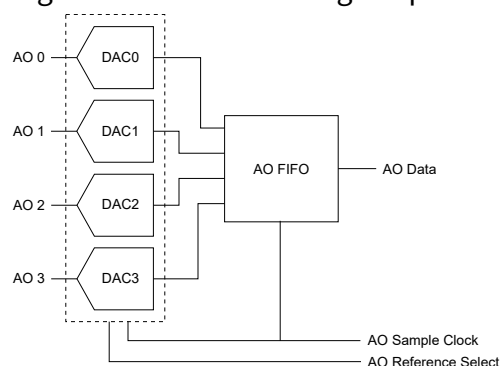
This method allows multiple channels to be sampled relatively quickly in relationship to the overall sample rate, providing a nearly simultaneous effect with a fixed delay between channels.

Analog Output for PCIe-6353 and PXIe-6353 Devices

PCIe-6353 and PXIe-6353 devices that support analog output have either two or four AO channels. AO channels are controlled by a single clock and are capable of waveform generation.

The following figure shows the analog output circuitry of PCIe-6353 and PXIe-6353 devices.

Figure 19. X Series Analog Output Circuitry for PCIe-6353 and PXIe-6353 Devices



The main blocks featured in the PCIe-6353 and PXIe-6353 analog output circuitry are as follows:

- **DACs**—Digital-to-analog converters (DACs) convert digital codes to analog voltages.
- **AO FIFO**—The AO FIFO enables analog output waveform generation. It is a first-in-first-out (FIFO) memory buffer between the computer and the DACs. It allows you to download the points of a waveform to your X Series device without host computer interaction.
- **AO Sample Clock**—The AO Sample Clock signal reads a sample from the DAC FIFO and generates the AO voltage.
- **AO Reference Selection**—The AO reference selection signal allows you to change the range of the analog outputs.

AO Reference Selection

AO reference selection allows you to set the analog output range. The analog output range describes the set of voltages the device can generate. The digital codes of the DAC are spread evenly across the analog output range. Smaller analog output ranges have better resolution and greater accuracy because the voltage output difference between two consecutive codes is smaller.

The analog output range of a device is all of the voltages between -AO Reference and +AO Reference.

The number of analog output channels (AO <0..1> or AO <0..3>) and possible AO reference settings vary depending on the device model. For the PCIe-6353 and PXIe-6353 device, the AO reference of each analog output can be individually set to one of the following:

- 10 V
- 5 V
- APFI <0,1>

You can connect an external signal to APFI <0,1> to provide the AO reference. The AO reference can be a positive or negative voltage. If AO reference is a negative voltage, the polarity of the AO output is inverted. The valid ranges of APFI <0,1> are listed in the device specifications.

You can use one of the AO signals as the AO reference for a different AO signal. However, you must externally connect this channel to APFI 0 or APFI 1.



Note When using an external reference, the output signal is not calibrated in software. You can generate a value and measure the voltage offset to calibrate your output in software.

Analog and Digital Output Data Generation Methods

You can use software-timed or hardware-timed data generations methods when performing an analog output or digital waveform operation.

Software-Timed Generations

With a software-timed generation, software controls the rate at which data is generated. Software sends a separate command to the hardware to initiate each update. In NI-DAQmx, software-timed generations are referred to as on-demand timing. Software-timed generations are also referred to as immediate or static operations. They are typically used for writing a single value out, such as a constant digital value.

Hardware-Timed Generations

With a hardware-timed generation, a digital hardware signal controls the rate of the generation. This signal can be generated internally on your device or provided externally.

Hardware-timed generations have several advantages over software-timed generations:

- The time between samples can be much shorter.
- The timing between samples can be deterministic.
- Hardware-timed acquisitions can use hardware triggering.

PCIe-6353 and PXIe-6353 devices support buffered hardware-timed operations and hardware-timed single point operations.

A buffer is a temporary storage in computer memory for to-be-transferred samples. Data is moved from a PC buffer to the DAQ device's onboard FIFO using DMA before it is written to the output lines one sample at a time. Buffered generation typically allow for much faster transfer rates than non-buffered acquisitions because data is moved in large blocks, rather than one point at a time.

One property of buffered I/O operations is the sample mode. The sample mode can be either finite or continuous:

- Finite sample mode acquisition refers to the acquisition of a specific, predetermined number of data samples. Once the specified number of samples has been written out, the generation stops.
- Continuous generation refers to the generation of an unspecified number of

samples. Instead of generating a set number of data samples and stopping, a continuous generation continues until you stop the operation. There are several different methods of continuous generation that control what data is written.

These methods are regeneration, FIFO regeneration and non-regeneration modes:

- Regeneration is the repetition of the data that is already in the buffer. Standard regeneration is when data from the PC buffer is continually downloaded to the FIFO to be written out. New data can be written to the PC buffer at any time without disrupting the output. Use the NI-DAQmx write property `regenMode` to allow (or not allow) regeneration. The NI-DAQmx default is to allow regeneration.
- With FIFO regeneration, the entire buffer is downloaded to the FIFO and regenerated from there. Once the data is downloaded, new data cannot be written to the FIFO. To use FIFO regeneration, the entire buffer must fit within the FIFO size. The advantage of using FIFO regeneration is that it does not require communication with the main host memory once the operation is started, thereby preventing any problems that may occur due to excessive bus traffic. Use the NI-DAQmx `UseOnlyOnBoardMemory` DO channel property to enable or disable FIFO regeneration.
- With non-regeneration, old data is not repeated. New data must be continually written to the buffer. If the program does not write new data to the buffer at a fast enough rate to keep up with the generation, the buffer underflows and causes an error.

Hardware-timed single point (HWTSP) operations are typically used to write single samples at known time intervals. While buffered operations are optimized for high throughput, HWTSP operations are optimized for low latency and low jitter. In addition, HWTSP can notify software if it falls behind hardware. These features make HWTSP ideal for real time control applications. HWTSP operations, in conjunction with the wait for next sample clock function, provide tight synchronization between the software layer and the hardware layer. Refer to the NI-DAQmx Hardware-Timed Single Point Lateness Checking document for more information. To access this document, go to ni.com/info and enter the Info Code `daqhwtsp`.

Minimizing Glitches on the Output Signal

When you use a DAC to generate a waveform, you may observe glitches on the output signal. These glitches are normal; when a DAC switches from one voltage to another, it

produces glitches due to released charges. The largest glitches occur when the most significant bit of the DAC code changes. You can build a lowpass deglitching filter to remove some of these glitches, depending on the frequency and nature of the output signal. Visit ni.com/support for more information about minimizing glitches.

Getting Started with AO Applications in Software

You can use an X Series device in the following analog output applications:

- Single-point (on-demand) generation
- Finite generation
- Continuous generation
- Waveform generation

You can perform these generations through programmed I/O or DMA data transfer mechanisms. Some of the applications also use start triggers and pause triggers.



Note For more information about programming analog output applications and triggers in software, refer to the NI-DAQmx Help or the LabVIEW Help.

X Series devices use the NI-DAQmx driver. NI-DAQmx includes a collection of programming examples to help you get started developing an application. You can modify example code and save it in an application. You can use examples to develop a new application or add example code to an existing application.

Related reference:

- [Finding Examples](#)

Related information:

- [NI-DAQmx User Manual](#)

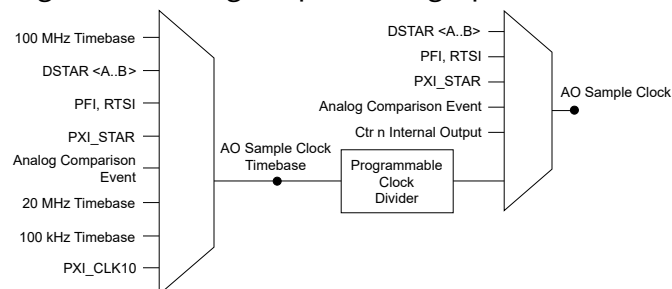
Analog Output Triggering

Analog output supports start and pause trigger actions. An analog or digital trigger can initiate these actions. To verify your device's triggering options, refer to the device specifications.

Analog Output Timing Signals

The following figure summarizes all of the timing options provided by the analog output timing engine.

Figure 20. Analog Output Timing Options



Analog output (waveform generation) timing signals support digital filtering (excluding the AO Sample Clock Timebase Signal, which does not support digital filtering). Refer to the *PFI Filters* topic for more information.

X Series devices feature the following analog output (waveform generation) timing signals. Signals with an * support digital filtering.

- AO Start Trigger Signal*
- AO Pause Trigger Signal*
- AO Sample Clock Signal*
- AO Sample Clock Timebase Signal

AO Start Trigger Signal

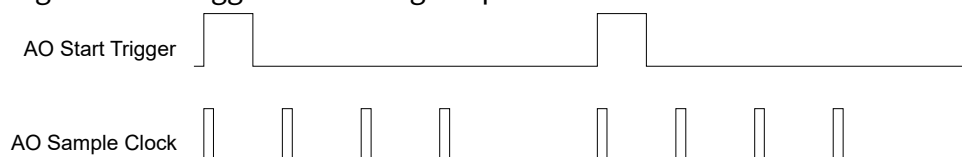
Use the AO Start Trigger (ao/StartTrigger) signal to initiate a waveform generation. If you do not use triggers, you can begin a generation with a software command.

Retriggerable AO

The AO Start Trigger is configurable as retriggerable. The timing engine generates the sample clock for the configured generation in response to each pulse on an AO Start Trigger signal.

The timing engine ignores the AO Start Trigger signal while the clock generation is in progress. After the clock generation is finished, the counter waits for another Start Trigger to begin another clock generation.

Figure 21. Retriggerable Analog Output



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Using a Digital Source

To use AO Start Trigger with a digital source, specify a source and an edge. The source can be one of the following signals:

- PFI <0..15>
- PXI_STAR
- PXIe_DSTAR<A,B>
- RTSI <0..7>
- A pulse initiated by host software
- AI Start Trigger (ai/StartTrigger)
- AI Reference Trigger (ai/ReferenceTrigger)
- Counter n Internal Output
- Counter n Internal Output
- Change Detection Event
- DI Start Trigger (di/StartTrigger)
- DI Reference Trigger (di/ReferenceTrigger)
- DO Start Trigger (do/StartTrigger)

The source can also be one of several other internal signals on your DAQ device. Refer to *Device Routing in MAX* in the *NI-DAQmx Help* for more information.

You can also specify whether the measurement acquisition begins on the rising edge or falling edge of AO Start Trigger.

Using an Analog Source

When you use an analog trigger source, the waveform generation begins on the first rising edge of the Analog Comparison Event signal.

Routing AO Start Trigger Signal to an Output Terminal

You can route AO Pause Trigger out to any PFI <0..15>, RTSI <0..7>, or PXIe_DSTARC terminal.



Note The output is an active high pulse. PFI terminals are configured as inputs by default.

Related concepts:

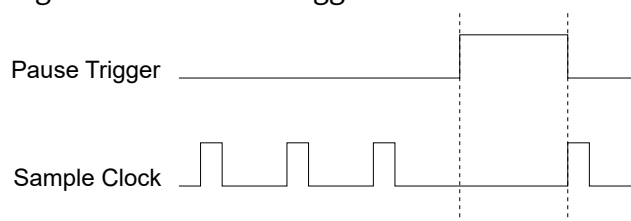
- [Device Routing in MAX](#)

AO Pause Trigger Signal

Use the AO Pause Trigger (ao/PauseTrigger) signal to mask off samples in a DAQ sequence. That is, when AO Pause Trigger is active, no samples occur. AO Pause Trigger does not stop a sample that is in progress. The pause does not take effect until the beginning of the next sample.

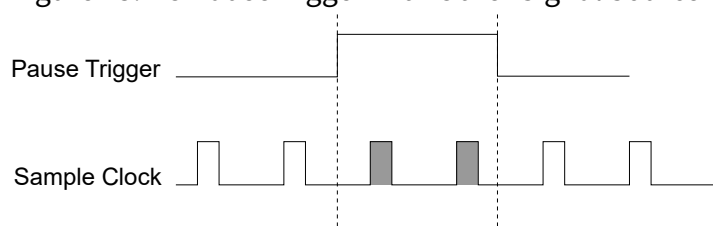
When you generate analog output signals, the generation pauses as soon as the pause trigger is asserted. If the source of your sample clock is the onboard clock, the generation resumes as soon as the pause trigger is deasserted, as shown in the following figure.

Figure 22. AO Pause Trigger with the Onboard Clock Source



If you are using any signal other than the onboard clock as the source of your sample clock, the generation resumes as soon as the pause trigger is deasserted and another edge of the sample clock is received, as shown in the following figure.

Figure 23. AO PauseTrigger with Other Signal Source



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Using a Digital Source

To use AO Pause Trigger, specify a source and a polarity. The source can be any of the following signals:

- PFI <0..15>
- PXI_STAR
- PXIe_DSTAR<A,B>
- RTSI <0..7>
- Counter *n* Internal Output
- Counter *n* Gate
- AI Pause Trigger (ai/PauseTrigger)
- AO Pause Trigger (ao/PauseTrigger)
- DO Pause Trigger (do/PauseTrigger)

The source can also be one of several other internal signals on your DAQ device. Refer

to *Device Routing in MAX* in the *NI-DAQmx Help* for more information.

You can also specify whether the samples are paused when AO Pause Trigger is at a logic high or low level.

Using an Analog Source

When you use an analog trigger source, the samples are paused when the Analog Comparison Event signal is at a high level.

Routing AO Pause Trigger Signal to an Output Terminal

You can route AO Pause Trigger out to any PFI <0..15>, RTSI <0..7>, PXI_STAR, or PXIe_DSTARC terminal.

Related concepts:

- [Device Routing in MAX](#)

AO Sample Clock Signal

Use the AO Sample Clock (ao/SampleClock) signal to initiate AO samples. Each sample updates the outputs of all of the DACs. You can specify an internal or external source for AO Sample Clock. You can also specify whether the DAC update begins on the rising edge or falling edge of AO Sample Clock.

Using an Internal Source

One of the following internal signals can drive AO Sample Clock:

- AO Sample Clock Timebase (divided down)
- Counter *n* Internal Output
- Change Detection Event
- Counter *n* Sample Clock
- AI Convert Clock (ai/ConvertClock)
- AI Sample Clock (ai/SampleClock)
- DI Sample Clock (di/SampleClock)
- DO Sample Clock (do/SampleClock)

A programmable internal counter divides down the AO Sample Clock Timebase signal.

Several other internal signals can be routed to AO Sample Clock through internal routes. Refer to Device Routing in MAX in the NI-DAQmx Help or the LabVIEW Help for more information.



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Using an External Source

Use one of the following external signals as the source of AO Sample Clock:

- PFI <0..15>
- PXI_STAR
- PXIe_DSTAR<A,B>
- RTSI <0..7>
- Analog Comparison Event (an analog trigger)

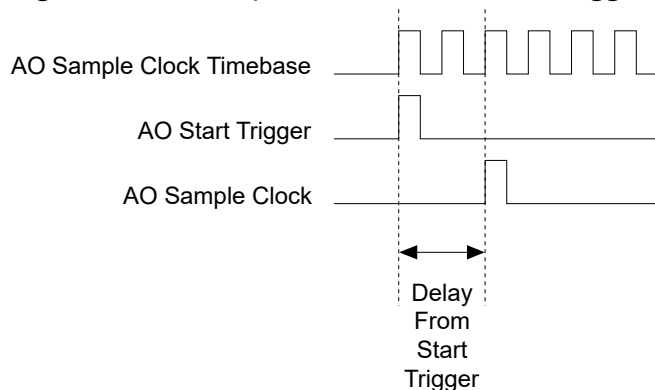
Routing AO Sample Clock Signal to an Output Terminal

You can route AO Sample Clock (as an active low signal) out to any PFI <0..15>, RTSI <0..7>, or PXIe_DSTARC terminal.

Other Timing Requirements

The AO timing engine on your device internally generates AO Sample Clock unless you select some external source. AO Start Trigger starts the timing engine and either the software or hardware can stop it once a finite generation completes. When using the AO timing engine, you can also specify a configurable delay from AO Start Trigger to the first AO Sample Clock pulse. By default, this delay is two ticks of AO Sample Clock Timebase. The following figure shows the relationship of AO Sample Clock to AO Start Trigger.

Figure 24. AO Sample Clock and AO Start Trigger

**Related concepts:**

- [Device Routing in MAX](#)

AO Sample Clock Timebase Signal

The AO Sample Clock Timebase (ao/SampleClockTimebase) signal is divided down to provide a source for AO Sample Clock.



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

You can route any of the following signals to be the AO Sample Clock Timebase signal:

- 100 MHz Timebase (default)
- 20 MHz Timebase
- 100 kHz Timebase
- PXI_CLK10
- Analog Comparison Event (an analog trigger)
- PFI <0..15>
- PXI_CLK10
- PXI_STAR
- PXIe_DSTAR<A,B>
- RTSI <0..7>

AO Sample Clock Timebase is not available as an output on the I/O connector.

You can use an external sample clock signal as AO Sample Clock Timebase signal by dividing the signal down in a DAQ device. You can also use it as AO Sample Clock signal without dividing the signal.

Digital I/O for PCIe-6353 and PXIe-6353 Devices

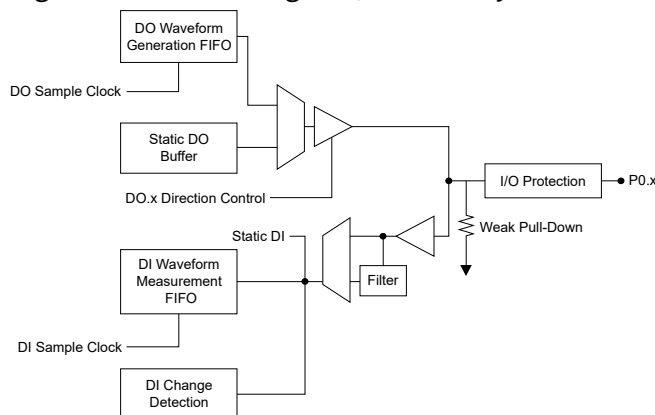
PCIe-6353 and PXIe-6353 devices contain up to 32 lines of bidirectional DIO signals on Port 0. In addition, PCIe-6353 and PXIe-6353 devices have up to 16 PFI signals that can function as static DIO signals.

PCIe-6353 and PXIe-6353 devices support the following DIO features on Port 0:

- Up to 32 lines of DIO
- Direction and function of each terminal individually controllable
- Static digital input and output
- High-speed digital waveform generation
- High-speed digital waveform acquisition
- DI change detection trigger/interrupt

The following figure shows the circuitry of one DIO line. Each DIO line is similar.

Figure 25. X Series Digital I/O Circuitry for PCIe-6353 and PXIe-6353 Devices



The DIO terminals are named P0.<0..31> on the PCIe-6353 and PXIe-6353 device I/O connector.

The voltage input and output levels and the current drive levels of the DIO lines are listed in the PCIe-6353 and PXIe-6353 device specifications.

Digital Input Triggering

Digital Input supports start, reference, and pause trigger actions. A digital or analog trigger can initiate these actions, depending on the capabilities of your device. To verify your triggering options, refer to the device specifications.

I/O Protection

Each DIO and PFI signal is protected against overvoltage, undervoltage, and overcurrent conditions as well as ESD events. However, you should avoid these fault conditions by following these guidelines:

- If you configure a PFI or DIO line as an output, do not connect it to any external signal source, ground, or power supply.
- If you configure a PFI or DIO line as an output, understand the current requirements of the load connected to these signals. Do not exceed the specified current output limits of the DAQ device. NI has several signal conditioning solutions for digital applications requiring high current drive.
- If you configure a PFI or DIO line as an input, do not drive the line with voltages outside of its normal operating range. The PFI or DIO lines have a smaller operating range than the AI signals.
- Treat the DAQ device as you would treat any static sensitive device. Always properly ground yourself and the equipment when handling the DAQ device or connecting to it.

Programmable Power-Up States

At system startup and reset, the hardware sets all PFI and DIO lines to high-impedance inputs by default. The DAQ device does not drive the signal high or low. Each line has a weak pull-down resistor connected to it, as described in the device specifications.

NI-DAQmx supports programmable power-up states for PFI and DIO lines. Software can program any value at power up to the P0, P1, or P2 lines. The PFI and DIO lines can be set as:

- A high-impedance input with a weak pull-down resistor (default)

- An output driving a 0
- An output driving a 1

Refer to the NI-DAQmx Help or the LabVIEW Help for more information about setting power-up states in NI-DAQmx or MAX.



Note When using your X Series device to control an SCXI chassis, DIO lines 0, 1, 2, and 4 are used as communication lines and must be left to power-up in the default high-impedance state to avoid potential damage to these signals.

Related information:

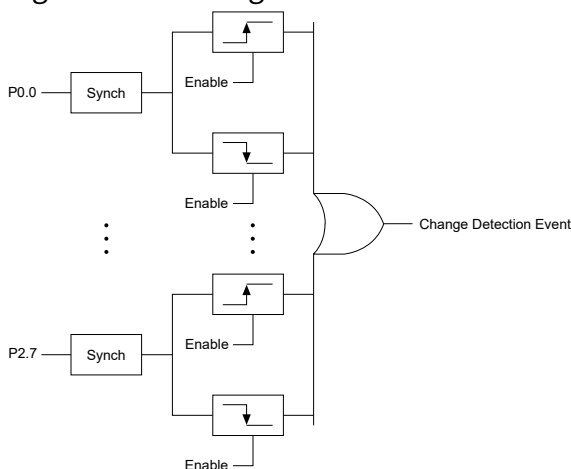
- [DAQmx Set Power Up States \(VI\)](#)
- [DAQmx Get Power Up States \(VI\)](#)
- [NI-DAQmx User Manual](#)

DI Change Detection

You can configure the DAQ device to detect changes on all 32 digital input lines (P0, P1, and P2) and all 16 PFI lines.

The following figure shows a block diagram of the DIO change detection circuitry.

Figure 26. DI Change Detection



You can enable the DIO change detection circuitry to detect rising edges, falling edges, or either edge individually on each DIO line. The DAQ devices synchronize each DI

signal to the 100 MHz Timebase, and then sends the signal to the change detectors. The circuitry ORs the output of all enabled change detectors from every DI signal. The result of this OR is the Change Detection Event signal.

Change detection performs bus correlation by considering all changes within a 50 ns window one change detection event, which keeps signals on the same bus synchronized in samples and prevents overruns.

The Change Detection Event signal can do the following:

- Drive any RTSI <0..7>, PFI <0..15>, or PXI_STAR signal
- Drive the DO Sample Clock or DI Sample Clock
- Generate an interrupt

The Change Detection Event signal can also be used to detect changes on digital output events.

DI Change Detection Applications

The DIO change detection circuitry can interrupt a user program when one of several DIO signals changes state.

You can also use the output of the DIO change detection circuitry to trigger a DI or counter acquisition on the logical OR of several digital signals. To trigger on a single digital signal, refer to the *Triggering with a Digital Source* topic. By routing the Change Detection Event signal to a counter, you can also capture the relative time between bus changes.

You can also use the Change Detection Event signal to trigger DO or counter generations.

Related concepts:

- [Triggering with a Digital Source](#)

Digital Filtering

You can enable a programmable debouncing filter on each digital line on Port 0. When

the filters are enabled, your device samples the input on each rising edge of a filter clock. X Series devices divide down the onboard 100 MHz or 100 kHz clocks to generate the filter clock. The following is an example of low-to-high transitions of the input signal. High-to-low transitions work similarly.

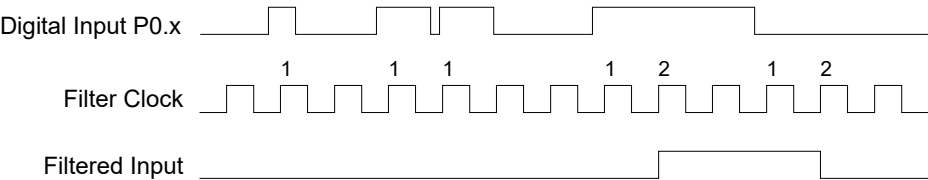
Assume that an input terminal has been low for a long time. The input terminal then changes from low-to-high, but glitches several times. When the filter clock has sampled the signal high on two consecutive edges and the signal remained stable in between, the low-to-high transition is propagated to the rest of the circuit.

Table 5. Filters

Filter Setting	Filter Clock	Pulse Width Guaranteed to Pass Filter	Pulse Width Guaranteed to Not Pass Filter
Short	12.5 MHz	160 ns	80 ns
Medium	195.3125 kHz	10.24 μ s	5.12 μ s
High	390.625 Hz	5.12 ms	2.56 ms
None	—	—	—

The filter setting for each input can be configured independently. On power up, the filters are disabled. The following figure shows an example of a low-to-high transition on an input.

Figure 27. Input Low-to-High Transition



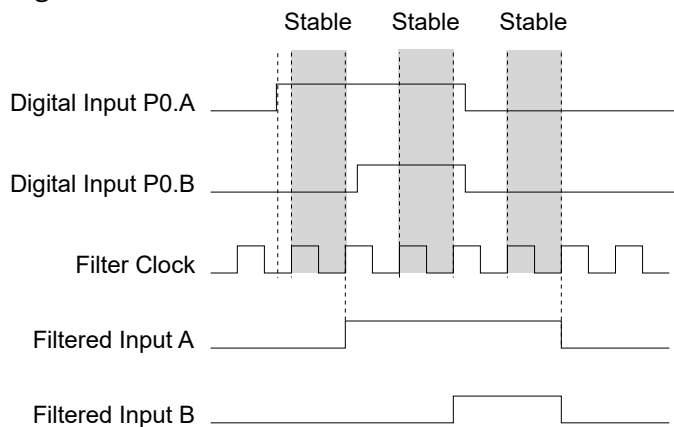
When multiple lines are configured with the same filter settings they are considered a bus. There are two filtering modes for use with multiple lines: line filtering and bus filtering. With line filtering, each line transitions independently of the other lines in the bus and acts like the behavior described above. With bus filtering, if any one line in the bus has jitter then all lines in the bus hold the state until the bus becomes stable. However, each individual line only waits one extra filter tick before changing, which prevents a noisy line from holding a valid transition indefinitely. With bus mode if all the bus line transitions become stable in less than one filter clock period and the bus

period is more than two filter clock periods, then all the bus lines are guaranteed to be correlated at the output of the filter.

The behavior for each transition can be thought of as a state machine. If a line transitions and stays high for two consecutive filter clock edges, then one of two options occurs:

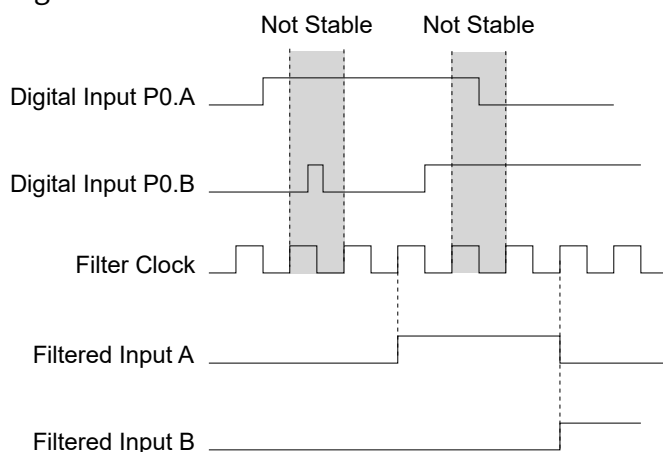
- Case 1—If no transitions have occurred on the other lines, the transition propagates on the second filtered clock edge, as shown in the following figure.

Figure 28. Case 1



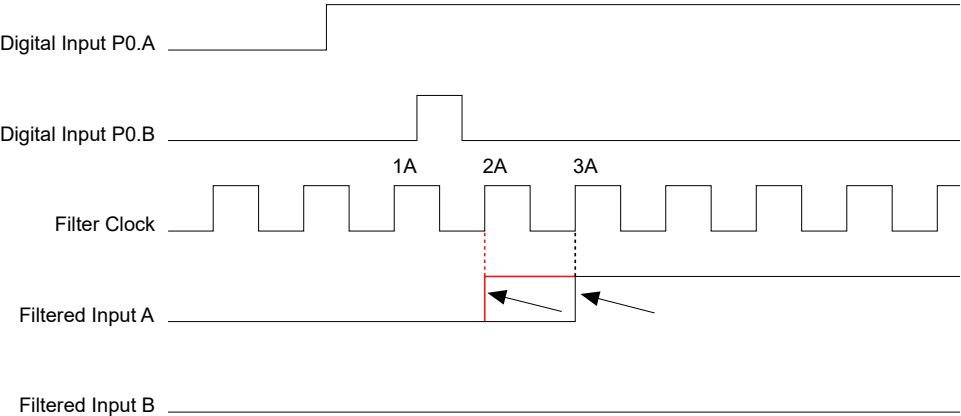
- Case 2—If an additional line on the bus also has a transition during the filter clock period, the change is not propagated until the next filter clock edge, as shown in the following figure.

Figure 29. Case 2



The following figure illustrates the difference between line and bus filtering.

Figure 30. Line and Bus Filtering



2A	With line filtering, filtered input A would ignore the glitch on digital input P0.B and transition after two filter clocks.
3A	Filtered input A goes high when sampled high for two consecutive filter clocks and transitions on the next filter edge because digital input P0.B glitches.

Watchdog Timer

The watchdog timer is a software-configurable feature used to set critical outputs to safe states in the event of a software failure, a system crash, or any other loss of communication between the application and the X Series device.



Note The NI-DAQmx Watchdog feature is meant to protect a system from software errors and hangs. In the case of a PXI system with remote control through MXI, a lost MXI connection could result in unexpected Watchdog behavior and therefore improperly implemented Watchdog states.

When the watchdog timer is enabled, if the X Series device does not receive a watchdog reset software command within the time specified for the watchdog timer, the outputs go to a user-defined safe state and remain in that state until the watchdog timer is disarmed by the application and new values are written, the device is reset, or the computer is restarted. The expiration signal that indicates an expired watchdog will continue to assert until the watchdog is disarmed. After the watchdog timer expires, the device ignores any digital writes until the watchdog timer is disarmed.



Note When the watchdog timer is enabled and the computer enters a fault condition, ports that are set to tri-state remain tri-stated and do not go to user-defined safe states.

You can set the watchdog timer timeout period to specify the amount of time that must elapse before the watchdog timer expires. The counter on the watchdog timer is configurable up to $(2^{32} - 1) \times 8$ ns (approximately 34 seconds) before it expires. A watchdog timer can be set for all DIO and PFI lines.

Getting Started with DIO Applications in Software

You can use the X Series device in the following digital I/O applications:

- Static digital input
- Static digital output
- Digital waveform generation
- Digital waveform acquisition
- DI change detection



Note For more information about programming digital I/O applications and triggers in software, refer to the *NI-DAQmx Help*.

X Series devices use the NI-DAQmx driver. NI-DAQmx includes a collection of programming examples to help you get started developing an application. You can modify example code and save it in an application. You can use examples to develop a new application or add example code to an existing application.

Related reference:

- [Finding Examples](#)

Related information:

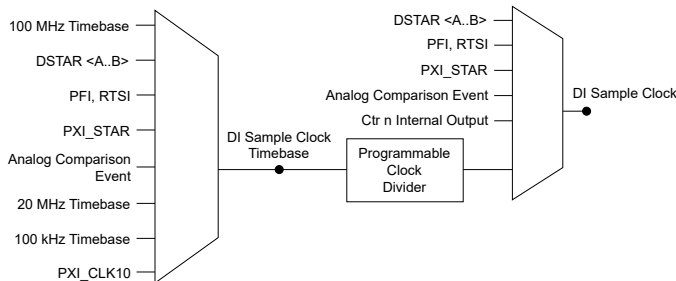
- [NI-DAQmx User Manual](#)

Digital Waveform Acquisition and Input Triggering

You can acquire digital waveforms on the Port 0 DIO lines. The DI waveform acquisition FIFO stores the digital samples. X Series devices have a DMA controller dedicated to moving data from the DI waveform acquisition FIFO to system memory. The DAQ device samples the DIO lines on each rising or falling edge of a clock signal, DI Sample Clock.

The following figure summarizes all of the timing options provided by the digital input timing engine.

Figure 31. Digital Input Timing Options



You can configure each DIO line to be an output, a static input, or a digital waveform acquisition input.

X Series devices feature the following digital input timing signals. Signals with an * support digital filtering.

- DI Sample Clock Signal*
- DI Sample Clock Timebase Signal
- DI Start Trigger Signal*
- DI Reference Trigger Signal*
- DI Pause Trigger Signal*

Related concepts:

- [PFI Filters](#)

Digital Input Data Acquisition Methods

When performing digital input measurements, you either can perform software-timed or hardware-timed acquisitions.

Software-Timed Acquisitions

With a software-timed acquisition, software controls the rate of the acquisition. Software sends a separate command to the hardware to initiate each acquisition. In NI-DAQmx, software-timed acquisitions are referred to as having on-demand timing. Software-timed acquisitions are also referred to as immediate or static acquisitions and are typically used for reading a single sample of data.

Each of the X Series DIO lines can be used as a static DI or DO line. You can use static DIO lines to monitor or control digital signals. Each DIO can be individually configured as a digital input (DI) or digital output (DO).

All samples of static DI lines and updates of static DO lines are software-timed.

Hardware-Timed Acquisitions

With hardware-timed acquisitions, a digital hardware signal (di/SampleClock) controls the rate of the acquisition. This signal can be generated internally on your device or provided externally.

Hardware-timed acquisitions have several advantages over software-timed acquisitions.

- The time between samples can be much shorter.
- The timing between samples is deterministic.
- Hardware-timed acquisitions can use hardware triggering.

The PCIe-6353 and PXIe-6353 supports buffered hardware-timed operations and hardware-timed single point operations.

A buffer is a temporary storage in computer memory for to-be-transferred samples. Data is moved from the DAQ device's onboard FIFO memory to a PC buffer using DMA before it is transferred to application memory. Buffered acquisitions typically allow for

much faster transfer rates than non-buffered acquisitions because data is moved in large blocks, rather than one point at a time.

One property of buffered I/O operations is the sample mode. The sample mode can be either finite or continuous:

- Finite sample mode acquisition refers to the acquisition of a specific, predetermined number of data samples. Once the specified number of samples has been read in, the acquisition stops. If you use a reference trigger, you must use finite sample mode.
- Continuous acquisition refers to the acquisition of an unspecified number of samples. Instead of acquiring a set number of data samples and stopping, a continuous acquisition continues until you stop the operation. Continuous acquisition is also referred to as double-buffered or circular-buffered acquisition.

If data cannot be transferred across the bus fast enough, the FIFO becomes full. New acquisitions overwrite data in the FIFO before it can be transferred to host memory, which causes the device to generate an error. With continuous operations, if the user program does not read data out of the PC buffer fast enough to keep up with the data transfer, the buffer could reach an overflow condition, causing an error to be generated.

Hardware-timed single point (HWTSP) operations are typically used to read single samples at known time intervals. While buffered operations are optimized for high throughput, HWTSP operations are optimized for low latency and low jitter. In addition, HWTSP can notify software if it falls behind hardware. These features make HWTSP ideal for real time control applications. HWTSP operations, in conjunction with the wait for next sample clock function, provide tight synchronization between the software layer and the hardware layer. Refer to the NI-DAQmx Hardware-Timed Single Point Lateness Checking document for more information. To access this document, go to ni.com/info and enter the Info Code daqhwtsp.

DI Sample Clock Signal

The device uses the DI Sample Clock (di/SampleClock) signal to sample the Port 0 terminals and store the result in the DI waveform acquisition FIFO.

You can specify an internal or external source for DI Sample Clock. You can also specify

whether the measurement sample begins on the rising edge or falling edge of DI Sample Clock.

If the DAQ device receives a DI Sample Clock when the FIFO is full, it reports an overflow error to the host software.

Using an Internal Source

To use DI Sample Clock with an internal source, specify the signal source and the polarity of the signal. The source can be any of the following signals:

- DI Sample Clock (di/SampleClock)
- DO Sample Clock (do/SampleClock)
- AI Sample Clock (ai/SampleClock)
- AI Convert Clock (ai/ConvertClock)
- AO Sample Clock (ao/SampleClock)
- Counter n Sample Clock
- Counter n Internal Output
- Frequency Output
- DI Change Detection output

Several other internal signals can be routed to DI Sample Clock through internal routes. Refer to *Device Routing in MAX* in the *NI-DAQmx Help* for more information.



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Using an External Source

Any of the following signals can be routed as a DI Sample Clock, if supported by your device:

- PFI <0..15>
- PXI_STAR
- PXIe_DSTAR<A,B>
- RTSI <0..7>

You can sample data on the rising or falling edge of DI Sample Clock.

Routing DI Sample Clock to an Output Terminal

You can route DI Sample Clock out to any PFI <0..15> terminal. The PFI circuitry inverts the polarity of DI Sample Clock before driving the PFI terminal.

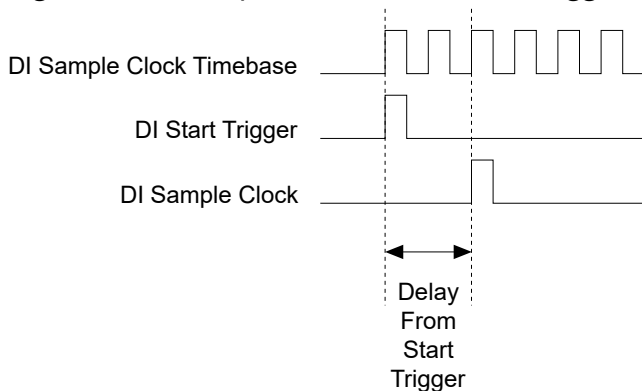
Other Timing Requirements

Your DAQ device only acquires data during an acquisition. The device ignores DI Sample Clock when a measurement acquisition is not in progress. During a measurement acquisition, you can cause your DAQ device to ignore DI Sample Clock using the DI Pause Trigger signal.

The DI timing engine on your device internally generates DI Sample Clock unless you select an external source. DI Start Trigger starts the timing engine and either software or hardware can stop it once a finite acquisition completes. When using the DI timing engine, you can also specify a configurable delay from DI Start Trigger to the first DI Sample Clock pulse.

By default, this delay is set to two ticks of the DI Sample Clock Timebase signal.

Figure 32. DI Sample Clock and DI Start Trigger



Related concepts:

- [Device Routing in MAX](#)

DI Sample Clock Timebase Signal

You can route any of the following signals to be the DI Sample Clock Timebase (di/

SampleClockTimebase) signal:



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

- 100 MHz Timebase (default)
- 20 MHz Timebase
- 100 kHz Timebase
- PXI_CLK10
- Analog Comparison Event (an analog trigger)
- PFI <0..15>
- PXI_STAR
- PXIe_DSTAR<A,B>
- RTSI <0..7>

Refer to the device routing table in MAX for all additional routable signals. To find the device routing table for your device, launch MAX and select **Devices and Interfaces** » **NI-DAQmx Devices**. Click a device to open a tabbed window in the middle pane. Click the **Device Routes** tab at the bottom of the pane to display the device routing table.

DI Sample Clock Timebase is not available as an output on the I/O connector. DI Sample Clock Timebase is divided down to provide one of the possible sources for DI Sample Clock. You can configure the polarity selection for DI Sample Clock Timebase as either rising or falling edge except for the 100 MHz Timebase or 20 MHz Timebase.

You might use DI Sample Clock Timebase if you want to use an external sample clock signal, but need to divide the signal down. If you want to use an external sample clock signal, but do not need to divide the signal, then you should use DI Sample Clock rather than DI Sample Clock Timebase.

DI Start Trigger Signal

Use the DI Start Trigger (di/StartTrigger) signal to begin a measurement acquisition. A measurement acquisition consists of one or more samples. If you do not use triggers, begin a measurement with a software command. Once the acquisition begins, configure the acquisition to stop:

- When a certain number of points are sampled (in finite mode)
- After a hardware reference trigger (in finite mode)
- With a software command (in continuous mode)

An acquisition that uses a start trigger (but not a reference trigger) is sometimes referred to as a posttriggered acquisition.



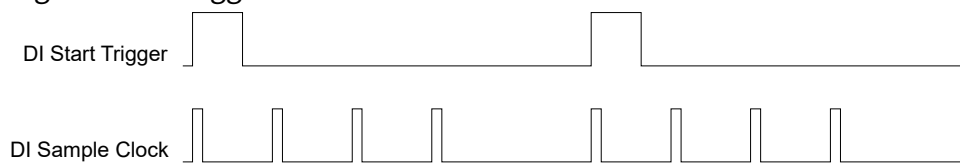
Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Retriggerable DI

The DI Start Trigger is configurable as retriggerable. When the DI Start Trigger is configured as retriggerable, the timing engine generates the sample and convert clocks for the configured acquisition in response to each pulse on a DI Start Trigger signal.

The timing engine ignores the DI Start Trigger signal while the clock generation is in progress. After the clock generation is finished, the timing engine waits for another Start Trigger to begin another clock generation. Figure 6-4 shows a retriggerable DI of four samples.

Figure 33. Retriggerable DI



Note Waveform information from LabVIEW does not reflect the delay between triggers. They are treated as a continuous acquisition with constant t_0 and dt information.

Reference triggers are not retriggerable.

Using a Digital Source

To use DI Start Trigger with a digital source, specify a source and an edge. The source

can be any of the following signals:

- PFI <0..15>
- PXI_STAR
- PXIe_DSTAR<A,B>
- RTSI <0..7>
- Counter n Internal Output
- Change Detection Event
- AI Start Trigger (ai/StartTrigger)
- AO Start Trigger (ao/StartTrigger)
- DO Start Trigger (do/StartTrigger)

The source can also be one of several other internal signals on your DAQ device. Refer to *Device Routing in MAX* in the *NI-DAQmx Help* for more information.

You can also specify whether the measurement acquisition begins on the rising edge or falling edge of DI Start Trigger.

Using an Analog Source

When you use an analog trigger source, the acquisition begins on the first rising or falling edge of the Analog Comparison Event signal.

Routing DI Start Trigger to an Output Terminal

You can route DI Start Trigger out to any PFI <0..15>, RTSI <0..7>, or PXIe_DSTARC terminal. The output is an active high pulse. All PFI terminals are configured as inputs by default.

The device also uses DI Start Trigger to initiate pretriggered DAQ operations. In most pretriggered applications, a software trigger generates DI Start Trigger. Refer to the *DI Reference Trigger Signal* topic for a complete description of the use of DI Start Trigger and DI Reference Trigger in a pretriggered DAQ operation.

Related concepts:

- [Device Routing in MAX](#)

DI Reference Trigger Signal

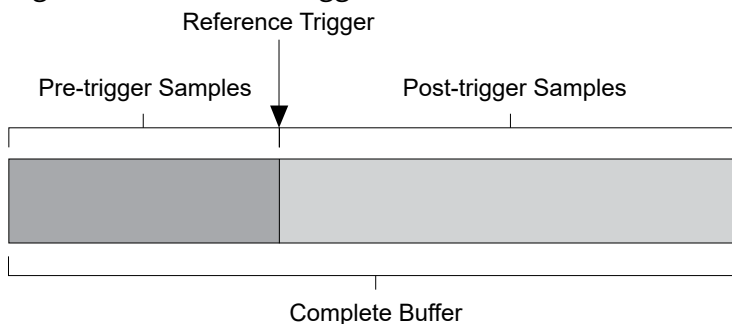
Use the DI Reference Trigger (di/ReferenceTrigger) signal to stop a measurement acquisition. To use a reference trigger, specify a buffer of finite size and a number of pretrigger samples (samples that occur before the reference trigger). The number of posttrigger samples (samples that occur after the reference trigger) desired is the buffer size minus the number of pretrigger samples.

Once the acquisition begins, the DAQ device writes samples to the buffer. After the DAQ device captures the specified number of pretrigger samples, the DAQ device begins to look for the reference trigger condition. If the reference trigger condition occurs before the DAQ device captures the specified number of pretrigger samples, the DAQ device ignores the condition.

If the buffer becomes full, the DAQ device continuously discards the oldest samples in the buffer to make space for the next sample. This data can be accessed (with some limitations) before the DAQ device discards it. Refer to the document *Can a Pretriggered Analog Acquisition be Continuous?* for more information. To access this document, go to ni.com/info and enter the Info Code `rdcanq`.

When the reference trigger occurs, the DAQ device continues to write samples to the buffer until the buffer contains the number of posttrigger samples desired. The following figure shows the final buffer.

Figure 34. Reference Trigger Final Buffer



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Using a Digital Source

To use DI Reference Trigger with a digital source, specify a source and an edge. The source can be any of the following signals:

- PFI <0..15>
- PXI_STAR
- PXIe_DSTAR<A,B>
- RTSI <0..7>
- Change Detection Event
- Counter *n* Internal Output
- AI Reference Trigger (ai/ReferenceTrigger)
- AO Start Trigger (ao/StartTrigger)
- DO Start Trigger (do/StartTrigger)

The source can also be one of several internal signals on your DAQ device. Refer to *Device Routing in MAX* in the *NI-DAQmx Help* for more information.

You can also specify whether the measurement acquisition stops on the rising or falling edge or falling edge of DI Reference Trigger.

Using an Analog Source

When you use an analog trigger source, the acquisition stops on the first rising edge of the Analog Comparison Event signal.

Routing DI Reference Trigger Signal to an Output Terminal

You can route DI Reference Trigger out to any PFI <0..15>, RTSI <0..7>, PXI_TRIG<0..7>, or PXIe_DSTARC terminal. All PFI terminals are configured as inputs by default.

Related concepts:

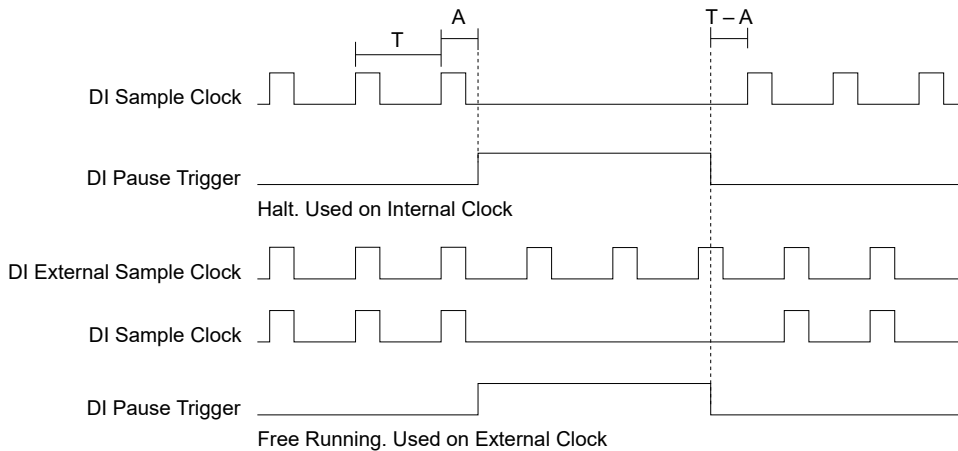
- [Device Routing in MAX](#)

DI Pause Trigger Signal

You can use the DI Pause Trigger (di/PauseTrigger) signal to pause and resume a

measurement acquisition. The internal sample clock pauses while the external trigger signal is active and resumes when the signal is inactive. You can program the active level of the pause trigger to be high or low, as shown in the following figure. In the figure, T represents the period, and A represents the unknown time between the clock pulse and the posttrigger.

Figure 35. Halt (Internal Clock) and Free Running (External Clock)



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Using a Digital Source

To use DI Pause Trigger, specify a source and a polarity. The source can be any of the following signals:

- PFI <0..15>
- PXI_STAR
- PXIe_DSTAR<A,B>
- RTSI <0..7>
- Counter n Internal Output
- Counter n Gate
- AI Pause Trigger (ai/PauseTrigger)
- AO Pause Trigger (ao/PauseTrigger)
- DO Pause Trigger (do/PauseTrigger)

The source can also be one of several other internal signals on your DAQ device. Refer

to *Device Routing in MAX* in the *NI-DAQmx Help* for more information.

Using an Analog Source

When you use an analog trigger source, the internal sample clock pauses when the Analog Comparison Event signal is low and resumes when the signal goes high (or vice versa).

Routing DI Pause Trigger Signal to an Output Terminal

You can route DI Pause Trigger out to any PFI <0..15>, RTSI <0..7>, PXI_STAR, or PXIe_DSTARC terminal.



Note Pause triggers are only sensitive to the level of the source, not the edge.

Related concepts:

- [Device Routing in MAX](#)

Digital Waveform Generation and Output Triggering

You can generate digital waveforms on the Port 0 DIO lines. The DO waveform generation FIFO stores the digital samples. X Series devices have a DMA controller dedicated to moving data from the system memory to the DO waveform generation FIFO. The DAQ device moves samples from the FIFO to the DIO terminals on each rising or falling edge of a clock signal, DO Sample Clock. You can configure each DIO signal to be an input, a static output, or a digital waveform generation output.

The FIFO supports a retransmit mode. In the retransmit mode, after all the samples in the FIFO have been clocked out, the FIFO begins outputting all of the samples again in the same order. For example, if the FIFO contains five samples, the pattern generated consists of sample #1, #2, #3, #4, #5, #1, #2, #3, #4, #5, #1, and so on.

X Series devices feature the following DO (waveform generation) timing signals:

- DO Sample Clock Signal*

- DO Sample Clock Timebase Signal
- DO Start Trigger Signal*
- DO Pause Trigger Signal*

Signals with an * support digital filtering. Refer to the *PFI Filters* section for more information.

Related concepts:

- [PFI Filters](#)

Digital Output Triggering

Digital output supports two different triggering actions:

- Start trigger
- Pause trigger

An analog or digital trigger can initiate these actions. Refer to the device specifications to verify the triggering options for your device.

Analog and Digital Output Data Generation Methods

You can use software-timed or hardware-timed data generations methods when performing an analog output or digital waveform operation.

Software-Timed Generations

With a software-timed generation, software controls the rate at which data is generated. Software sends a separate command to the hardware to initiate each update. In NI-DAQmx, software-timed generations are referred to as on-demand timing. Software-timed generations are also referred to as immediate or static operations. They are typically used for writing a single value out, such as a constant digital value.

Hardware-Timed Generations

With a hardware-timed generation, a digital hardware signal controls the rate of the

generation. This signal can be generated internally on your device or provided externally.

Hardware-timed generations have several advantages over software-timed generations:

- The time between samples can be much shorter.
- The timing between samples can be deterministic.
- Hardware-timed acquisitions can use hardware triggering.

PCIe-6353 and PXIe-6353 devices support buffered hardware-timed operations and hardware-timed single point operations.

A buffer is a temporary storage in computer memory for to-be-transferred samples. Data is moved from a PC buffer to the DAQ device's onboard FIFO using DMA before it is written to the output lines one sample at a time. Buffered generation typically allow for much faster transfer rates than non-buffered acquisitions because data is moved in large blocks, rather than one point at a time.

One property of buffered I/O operations is the sample mode. The sample mode can be either finite or continuous:

- Finite sample mode acquisition refers to the acquisition of a specific, predetermined number of data samples. Once the specified number of samples has been written out, the generation stops.
- Continuous generation refers to the generation of an unspecified number of samples. Instead of generating a set number of data samples and stopping, a continuous generation continues until you stop the operation. There are several different methods of continuous generation that control what data is written. These methods are regeneration, FIFO regeneration and non-regeneration modes:
 - Regeneration is the repetition of the data that is already in the buffer. Standard regeneration is when data from the PC buffer is continually downloaded to the FIFO to be written out. New data can be written to the PC buffer at any time without disrupting the output. Use the NI-DAQmx write property `regenMode` to allow (or not allow) regeneration. The NI-DAQmx default is to allow regeneration.
 - With FIFO regeneration, the entire buffer is downloaded to the FIFO and regenerated from there. Once the data is downloaded, new data cannot be

written to the FIFO. To use FIFO regeneration, the entire buffer must fit within the FIFO size. The advantage of using FIFO regeneration is that it does not require communication with the main host memory once the operation is started, thereby preventing any problems that may occur due to excessive bus traffic. Use the NI-DAQmx UseOnlyOnBoardMemory DO channel property to enable or disable FIFO regeneration.

- With non-regeneration, old data is not repeated. New data must be continually written to the buffer. If the program does not write new data to the buffer at a fast enough rate to keep up with the generation, the buffer underflows and causes an error.

Hardware-timed single point (HWTSP) operations are typically used to write single samples at known time intervals. While buffered operations are optimized for high throughput, HWTSP operations are optimized for low latency and low jitter. In addition, HWTSP can notify software if it falls behind hardware. These features make HWTSP ideal for real time control applications. HWTSP operations, in conjunction with the wait for next sample clock function, provide tight synchronization between the software layer and the hardware layer. Refer to the NI-DAQmx Hardware-Timed Single Point Lateness Checking document for more information. To access this document, go to ni.com/info and enter the Info Code daqhwtsp.

DO Sample Clock Signal

The device uses the DO Sample Clock (do/SampleClock) signal to update the DO terminals with the next sample from the DO waveform generation FIFO.

You can specify an internal or external source for DO Sample Clock. You can also specify whether the DAC update begins on the rising edge or falling edge of DO Sample Clock.

If the DAQ device receives a DO Sample Clock when the FIFO is empty, it reports an underflow error to the host software.



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Using an Internal Source

One of the following internal signals can drive the DO Sample Clock.

- DI Sample Clock (di/SampleClock)
- DO Sample Clock (do/SampleClock)
- AI Sample Clock (ai/SampleClock)
- AI Convert Clock (ai/ConvertClock)
- AO Sample Clock (ao/SampleClock)
- Counter n Sample Clock
- Counter n Internal Output
- Frequency Output
- DI Change Detection output

Several other internal signals can be routed to DO Sample Clock through internal routes. Refer to *Device Routing in MAX* in the *NI-DAQmx Help* for more information.

Using an External Source

Any of the following signals can be routed as a DO Sample Clock, if supported by your device:

- PFI <0..15>
- PXI_STAR
- PXIe_DSTAR<A,B>
- RTSI <0..7>
- Analog Comparison Event (an analog trigger)

Routing DO Sample Clock to an Output Terminal

You can route DO Sample Clock out to any PFI <0..15>, RTSI <0..7>, or PXIe_DSTARC terminal.

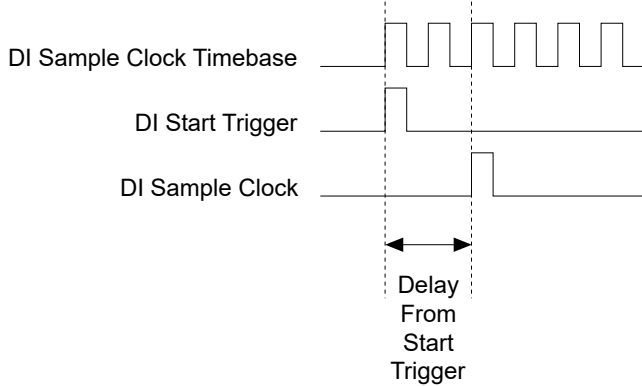
Other Timing Requirements

The DO timing engine on your device internally generates DO Sample Clock unless you select an external source. DO Start Trigger starts the timing engine and either software or hardware can stop it once a finite acquisition completes. When using the DO timing

engine, you can also specify a configurable delay from DO Start Trigger to the first DO Sample Clock pulse.

By default, this delay is set to two ticks of the DO Sample Clock Timebase signal.

Figure 36. DO Sample Clock and DO Start Trigger



DO Sample Clock Timebase Signal

The DO Sample Clock Timebase (do/SampleClockTimebase) signal is divided down to provide a source for DO Sample Clock. You can route any of the following signals to be the DO Sample Clock Timebase signal:



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

- 100 MHz Timebase (default)
- 20 MHz Timebase
- 100 kHz Timebase
- Analog Comparison Event (an analog trigger)
- PFI <0..15>
- PXI_CLK10
- PXI_STAR
- PXIe_DSTAR<A,B>
- RTSI <0..7>

DO Sample Clock Timebase is not available as an output on the I/O connector.

You might use DO Sample Clock Timebase if you want to use an external sample clock signal, but need to divide the signal down. If you want to use an external sample clock signal, but do not need to divide the signal, then you should use DO Sample Clock rather than DO Sample Clock Timebase.

DO Start Trigger Signal

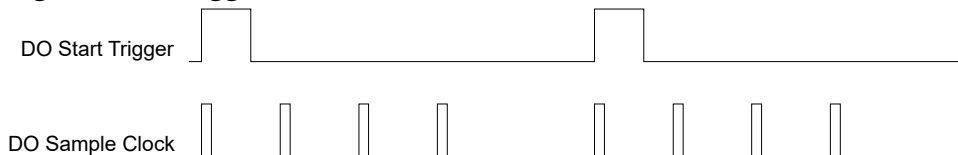
Use the DO Start Trigger (do/StartTrigger) signal to initiate a waveform generation. If you do not use triggers, you can begin a generation with a software command.

Retriggerable DI

The DO Start Trigger is configurable as retriggerable. When DO Start Trigger is configured as retriggerable, the timing engine generates the sample clocks for the configured generation in response to each pulse on a DO Start Trigger signal.

The timing engine ignores the DO Start Trigger signal while the clock generation is in progress. After the clock generation is finished, the timing engine waits for another start trigger to begin another clock generation. Figure 6-8 shows a retriggerable DO of four samples.

Figure 37. Retriggerable DO



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Using a Digital Source

To use DO Start Trigger with a digital source, specify a source and an edge. The source can be any of the following signals:

- A pulse initiated by host software
- AI Reference Trigger (ai/ReferenceTrigger)

- AI Start Trigger (ai/StartTrigger)
- AO Start Trigger (ao/StartTrigger)
- Counter n Internal Output
- DI Start Trigger (do/StartTrigger)
- DI Reference Trigger (do/ReferenceTrigger)
- Change Detection Event
- PFI <0..15>
- PXI_STAR
- PXIe_DSTAR<A,B>
- RTSI <0..7>

The source can also be one of several other internal signals on your DAQ device. Refer to *Device Routing in MAX* in the *NI-DAQmx Help* for more information.

You can also specify whether the measurement acquisition begins on the rising edge or falling edge of DI Start Trigger.

Using an Analog Source

When you use an analog trigger source, the waveform generation begins on the first rising or falling edge of the Analog Comparison Event signal.

Routing DO Start Trigger to an Output Terminal

You can route DO Start Trigger out to any PFI <0..15>, RTSI <0..7>, or PXIe_DSTARC terminal. The output is an active high pulse. All PFI terminals are configured as inputs by default.

Related concepts:

- [Device Routing in MAX](#)

DO Pause Trigger Signal

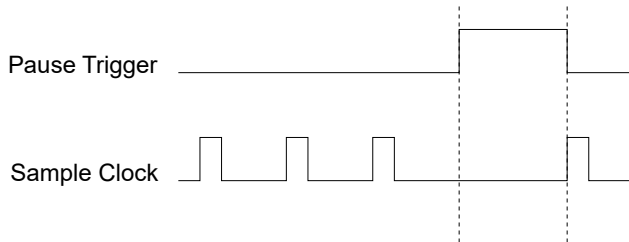
Use the DO Pause Trigger (do/PauseTrigger) signal to mask off samples in a DAQ sequence. That is, when DO Pause Trigger is active, no samples occur.

DO Pause Trigger does not stop a sample that is in progress. The pause does not take

effect until the beginning of the next sample.

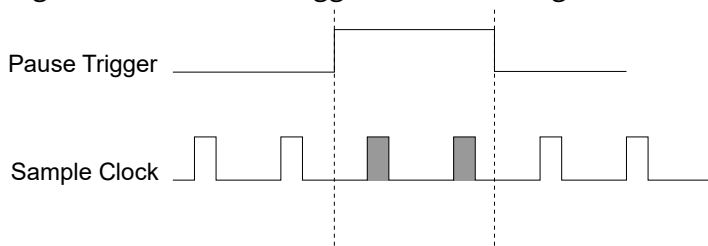
When you generate digital output signals, the generation pauses as soon as the pause trigger is asserted. If the source of your sample clock is the onboard clock, the generation resumes as soon as the pause trigger is deasserted, as shown in the following figure.

Figure 38. DO Pause Trigger with the Onboard Clock Source



If you are using any signal other than the onboard clock as the source of your sample clock, the generation resumes as soon as the pause trigger is deasserted and another edge of the sample clock is received, as shown in the following figure.

Figure 39. DO Pause Trigger with Other Signal Source



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Using a Digital Source

To use DO Pause Trigger, specify a source and a polarity. The source can be one of the following signals:

- RTSI <0..7>
- PFI <0..15>
- PXI_STAR
- PXIe_DSTAR<A,B>

- Counter n Internal Output
- Counter n Gate
- AI Pause Trigger (ai/PauseTrigger)
- AO Pause Trigger (ao/PauseTrigger)
- DI Pause Trigger (di/PauseTrigger)

The source can also be one of several other internal signals on your DAQ device. Refer to *Device Routing in MAX* in the *NI-DAQmx Help* for more information.

You can also specify whether the samples are paused when DO Pause Trigger is at a logic high or low level.

Using an Analog Source

When you use an analog trigger source, the samples are paused when the Analog Comparison Event signal is at a high level.

Routing DO Pause Trigger Signal to an Output Terminal

You can route DI Pause Trigger out to any RTSI <0..7>, PFI <0..15>, or PXIe_DSTARC terminal.

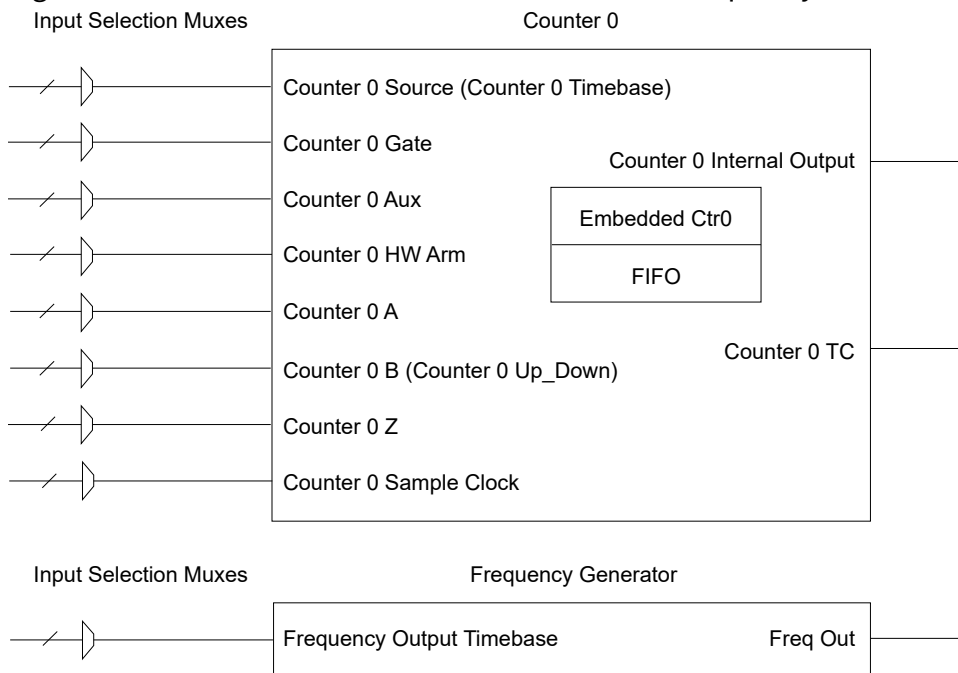
Related concepts:

- [Device Routing in MAX](#)

PCIe-6353 and PXIe-6353 Counters

X Series devices have four general-purpose 32-bit counter/timers and one frequency generator. The general-purpose counter/timers can be used for many measurement and pulse generation applications. The following figure shows the PCIe-6353 and PXIe-6353 Counter 0 and the frequency generator. All four counters on X Series devices are identical.

Figure 40. PCIe-6353 and PXIe-6353 Counter 0 and Frequency Generator



Counters have eight input signals, although in most applications only a few inputs are used.

Each counter has a FIFO that can be used for buffered acquisition and generation. Each counter also contains an embedded counter (Embedded Ctr n) for use in what are traditionally two-counter measurements and generations. The embedded counters cannot be programmed independent of the main counter; signals from the embedded counters are not routable.

Related reference:

- [Connecting to Counter Signals](#)

Counter Timing Engine

PCle-6353 and PXIe-6353 counters do not have the ability to divide down a timebase to produce an internal counter sample clock. For sample clocked operations, an external signal must be provided to supply a clock source. The source can be any of the following signals:

- AI Sample Clock
- AI Start Trigger
- AI Reference Trigger
- AO Sample Clock
- DI Sample Clock
- DI Start Trigger
- DO Sample Clock
- CTR n Internal Output
- Freq Out
- PFI <0..15>
- PXI_Trig <0..7>
- PXIe_DSTAR<A,B>
- Change Detection Event
- Analog Comparison Event

Not all timed counter operations require a sample clock. For example, a simple buffered pulse width measurement latches in data on each edge of a pulse. For this measurement, the measured signal determines when data is latched in. These operations are referred to as implicit timed operations. However, many of the same measurements can be clocked at an interval with a sample clock. These are referred to as sample clocked operations. The following table shows the different options for the different measurements.



Note All hardware-timed single point (HWTSP) operations are sample clocked.

Table 6. Counter Timing Measurements

Measurement	Implicit Timing Support	Sample Clocked Timing Support
Buffered Edge Count	No	Yes
Buffered Pulse Width	Yes	Yes
Buffered Pulse	Yes	Yes
Buffered Semi-Period	Yes	No
Buffered Frequency	Yes	Yes
Buffered Period	Yes	Yes
Buffered Position	No	Yes
Buffered Two-Signal Edge Separation	Yes	Yes

Counter Input Applications

Refer to the following sections for information about the various counter input applications that are supported by the PCIe-6353 and PXIe-6353.

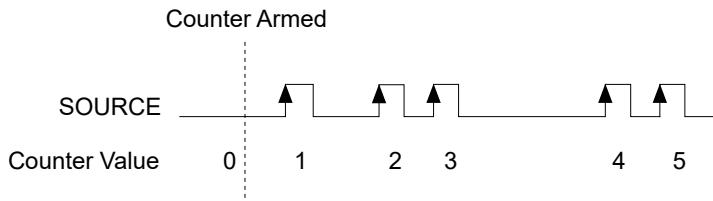
Counting Edges

In edge counting applications, the counter counts edges on its Source after the counter is armed. You can configure the counter to count rising or falling edges on its Source input. You can also control the direction of counting (up or down), as described in the Controlling the Direction of Counting section. The counter values can be read on demand or with a sample clock.

Single Point (On-Demand) Edge Counting

With single point (on-demand) edge counting, the counter counts the number of edges on the Source input after the counter is armed. On-demand refers to the fact that software can read the counter contents at any time without disturbing the counting process. The following figure shows an example of single point edge counting.

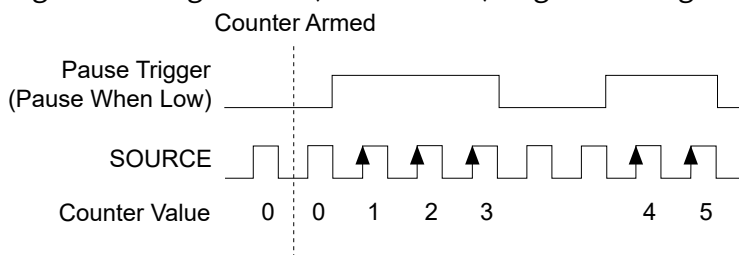
Figure 41. Single Point (On-Demand) Edge Counting



You can also use a pause trigger to pause (or gate) the counter. When the pause trigger is active, the counter ignores edges on its Source input. When the pause trigger is inactive, the counter counts edges normally.

You can route the pause trigger to the Gate input of the counter. You can configure the counter to pause counting when the pause trigger is high or when it is low. The following figure shows an example of on-demand edge counting with a pause trigger.

Figure 42. Single Point (On-Demand) Edge Counting with Pause Trigger



Buffered (Sample Clock) Edge Counting

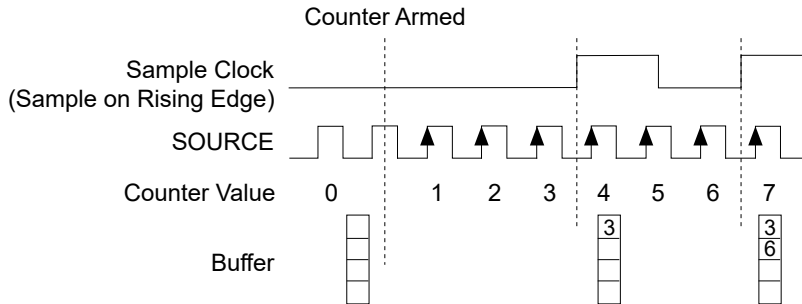
With buffered edge counting (edge counting using a sample clock), the counter counts the number of edges on the Source input after the counter is armed. The value of the counter is sampled on each active edge of a sample clock and stored in the FIFO. A DMA controller transfers the sampled values to host memory.

The count values returned are the cumulative counts since the counter armed event. That is, the sample clock does not reset the counter.

You can configure the counter to sample on the rising or falling edge of the sample clock.

The following figure shows an example of buffered edge counting. Notice that counting begins when the counter is armed, which occurs before the first active edge on Sample Clock.

Figure 43. Buffered (Sample Clock) Edge Counting



Controlling the Direction of Counting

In edge counting applications, the counter can count up or down. You can configure the counter to do the following:

- Always count up
- Always count down
- Count up when the Counter 0 B input is high; count down when it is low

Related reference:

- [Connecting to Counter Signals](#)

Pulse-Width Measurement

In pulse-width measurements, the counter measures the width of a pulse on its Gate input signal. You can configure the counter to measure the width of high pulses or low pulses on the Gate signal.

You can route an internal or external periodic clock signal (with a known period) to the Source input of the counter. The counter counts the number of rising (or falling) edges on the Source signal while the pulse on the Gate signal is active.

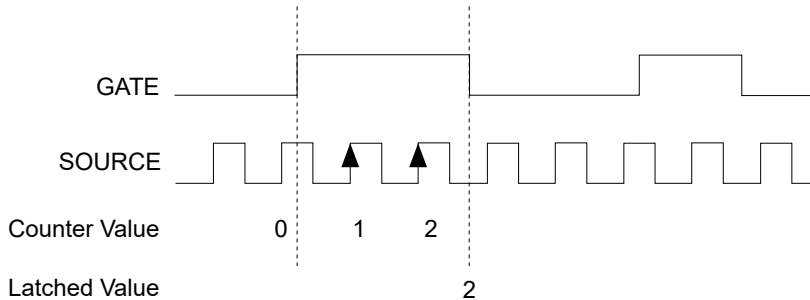
You can calculate the pulse width by multiplying the period of the Source signal by the number of edges returned by the counter.

A pulse-width measurement is accurate even if the counter is armed while a pulse train is in progress. If a counter is armed while the pulse is in the active state, it waits for the next transition to the active state to begin the measurement.

Single Pulse-Width Measurement

With single pulse-width measurement, the counter counts the number of edges on the Source input while the Gate input remains active. When the Gate input goes inactive, the counter stores the count in the FIFO and ignores other edges on the Gate and Source inputs. Software then reads the stored count. The following figure shows an example of a single pulse-width measurement.

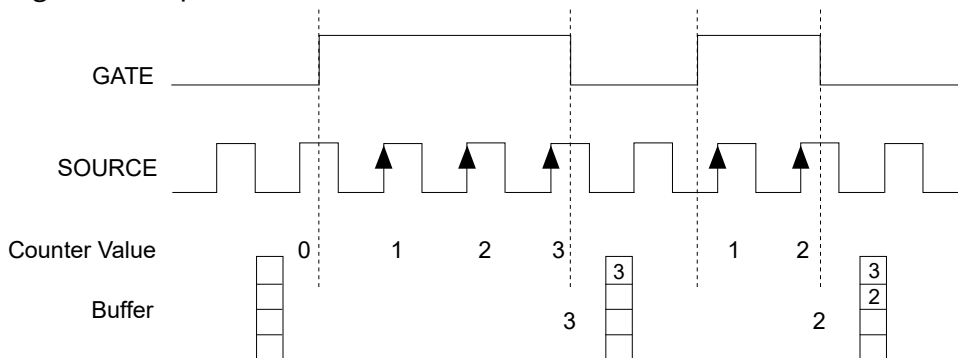
Figure 44. Single Pulse-Width Measurement



Implicit Buffered Pulse-Width Measurement

An implicit buffered pulse-width measurement is similar to single pulse-width measurement, but buffered pulse-width measurement takes measurements over multiple pulses. On each trailing edge of the Gate signal, the counter stores the count in the counter FIFO. A DMA controller transfers the stored values to host memory. The following figure shows an example of an implicit buffered pulse-width measurement.

Figure 45. Implicit Buffered Pulse-Width Measurement

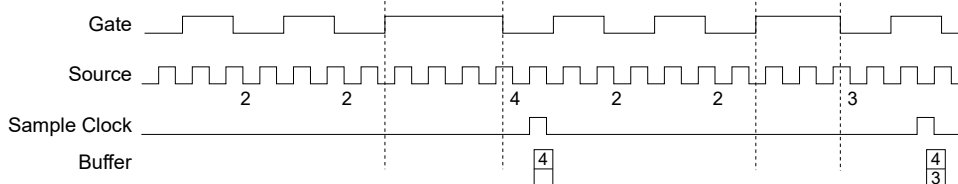


Sample Clocked Buffered Pulse-Width Measurement

A Sample Clocked Buffered pulse-width measurement is similar to single pulse-width measurement, but buffered pulse-width measurement takes measurements over multiple pulses correlated to a sample clock.

The counter counts the number of edges on the Source input while the Gate input remains active. On each sample clock edge, the counter stores the count in the FIFO of the last pulse width to complete. A DMA controller transfers the stored values to host memory. The following figure shows an example of a sample clocked buffered pulse-width measurement.

Figure 46. Single Pulse-Width Measurement



Related reference:

- [Connecting to Counter Signals](#)

Hardware-Timed Single Point Pulse-Width Measurement

A hardware-timed single point (HWTSP) pulse-width measurement has the same behavior as a sample clocked buffered pulse-width measurement.



Note If a pulse does not occur between sample clocks, an overrun error occurs.

Pulse Measurement

In pulse measurements, the counter measures the high and low time of a pulse on its Gate input signal after the counter is armed. A pulse is defined in terms of its high and low time, high and low ticks or frequency and duty cycle, which is similar to the pulse-width measurement, except that the inactive pulse is measured as well.

You can route an internal or external periodic clock signal (with a known period) to the

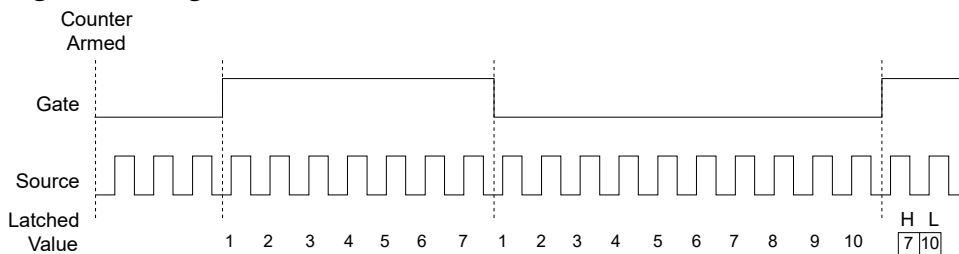
Source input of the counter. The counter counts the number of rising (or falling) edges occurring on the Source input between two edges of the Gate signal.

You can calculate the high and low time of the Gate input by multiplying the period of the Source signal by the number of edges returned by the counter.

Single-Pulse Measurement

Single (on-demand) pulse measurement is equivalent to two single pulse-width measurements on the high (H) and low (L) ticks of a pulse, as shown in the following figure:

Figure 47. Single (On-Demand) Pulse Measurement



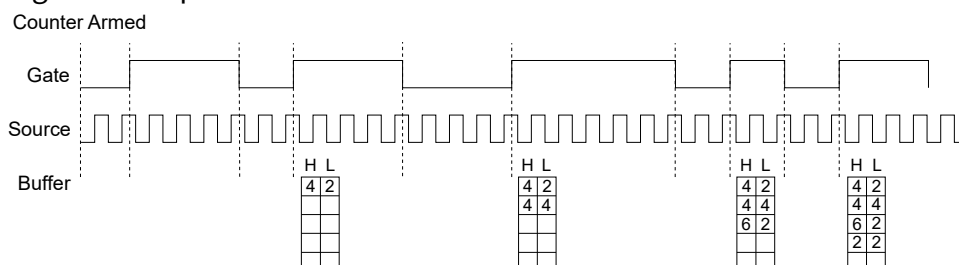
Implicit Buffered Pulse Measurement

In an implicit buffered pulse measurement, on each edge of the Gate signal, the counter stores the count in the FIFO. A DMA controller transfers the stored values to host memory.

The counter begins counting when it is armed. The arm usually occurs between edges on the Gate input, but the counting does not start until the desired edge. You can select whether to read the high pulse or low pulse first using the StartingEdge property in NI-DAQmx.

The following figure shows an example of an implicit buffered pulse measurement.

Figure 48. Implicit Buffered Pulse Measurement



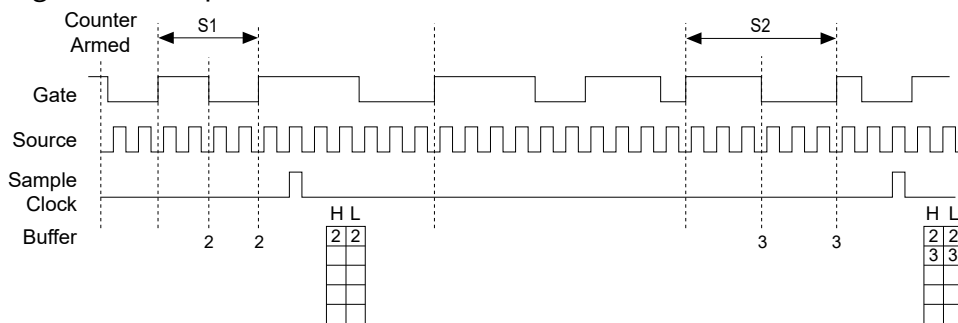
Sample Clocked Buffered Pulse Measurement

A sample clocked buffered pulse measurement is similar to single pulse measurement, but a buffered pulse measurement takes measurements over multiple pulses correlated to a sample clock.

The counter performs a pulse measurement on the Gate. On each sample clock edge, the counter stores the high and low ticks in the FIFO of the last pulse to complete. A DMA controller transfers the stored values to host memory.

The following figure shows an example of a sample clocked buffered pulse measurement.

Figure 49. Sample Clocked Buffered Pulse Measurement



Related reference:

- [Connecting to Counter Signals](#)

Hardware-Timed Single Point Pulse Measurement

A hardware-timed single point (HWTSP) pulse-width measurement has the same behavior as a sample clocked buffered pulse measurement.



Note If a pulse does not occur between sample clocks, an overrun error occurs.

Comparing Pulse and Semi-Period Measurements

In hardware, pulse measurement and semi-period are the same measurement. Both measure the high and low times of a pulse. The functional difference between the two

measurements is how the data is returned. In a semi-period measurement, each high or low time is considered one point of data and returned in units of seconds or ticks. In a pulse measurement, each pair of high and low times is considered one point of data and returned as a paired sample in units of frequency and duty cycle, high and low time or high and low ticks. When reading data, 10 points in a semi-period measurement gets an array of five high times and five low times. When you read 10 points in a pulse measurement, you get an array of 10 pairs of high and low times.

Pulse measurements support sample clock timing while semi-period measurements do not.

Semi-Period Measurement

In semi-period measurements, the counter measures a semi-period on its Gate input signal after the counter is armed. A semi-period is the time between any two consecutive edges on the Gate input.

You can route an internal or external periodic clock signal (with a known period) to the Source input of the counter. The counter counts the number of rising (or falling) edges occurring on the Source input between two edges of the Gate signal.

You can calculate the semi-period of the Gate input by multiplying the period of the Source signal by the number of edges returned by the counter.

Single Semi-Period Measurement

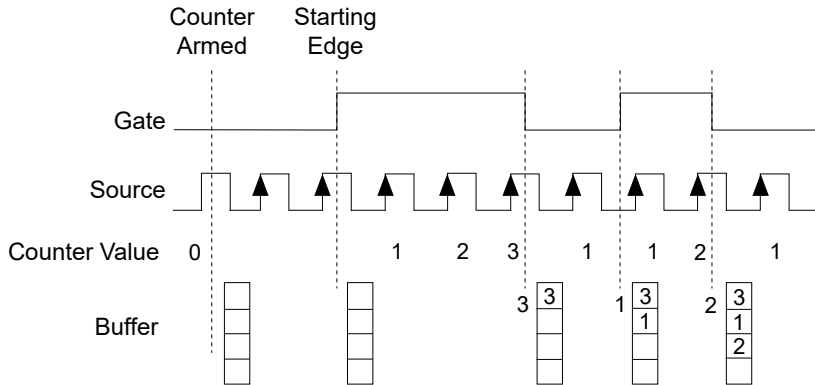
Single semi-period measurement is equivalent to single pulse-width measurement.

Implicit Buffered Semi-Period Measurement

In implicit buffered semi-period measurement, on each edge of the Gate signal, the counter stores the count in the FIFO. A DMA controller transfers the stored values to host memory.

The counter begins counting when it is armed. The arm usually occurs between edges on the Gate input. You can select whether to read the first active low or active high semi period using the `Cl.SemiPeriod.StartingEdge` property in NI-DAQmx. The following figure shows an example of an implicit buffered semi-period measurement.

Figure 50. Implicit Buffered Semi-Period Measurement



Frequency Measurement

You can use the counters to measure frequency in several different ways.

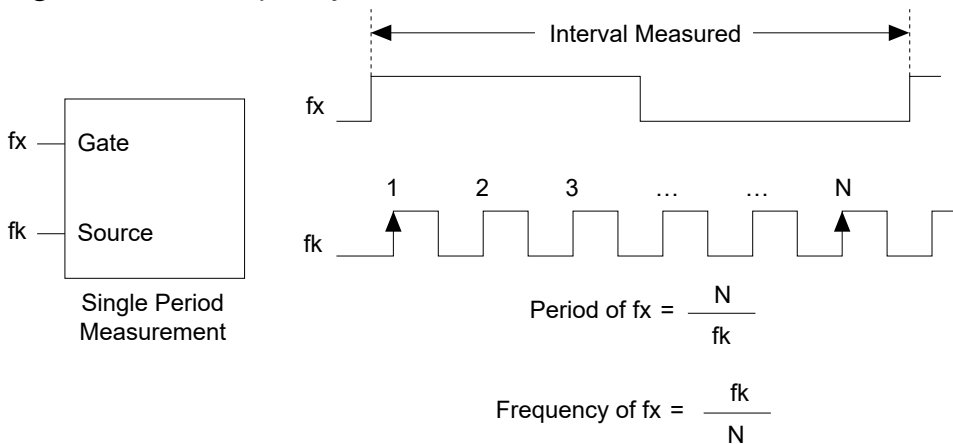
Low Frequency with One Counter

For low frequency measurements with one counter, you measure one period of your signal using a known timebase. You can route the signal to measure (f_x) to the Gate of a counter.

You can route a known timebase (f_k) to the Source of the counter. The known timebase can be an onboard timebase, such as 100 MHz Timebase, 20 MHz Timebase, or 100 kHz Timebase, or any other signal with a known rate.

You can configure the counter to measure one period of the gate signal. The frequency of f_x is the inverse of the period. The following figure illustrates this method.

Figure 51. Low Frequency with One Counter



High Frequency with Two Counters

For high frequency measurements with two counters, you measure one pulse of a known width using your signal and derive the frequency of your signal from the result.

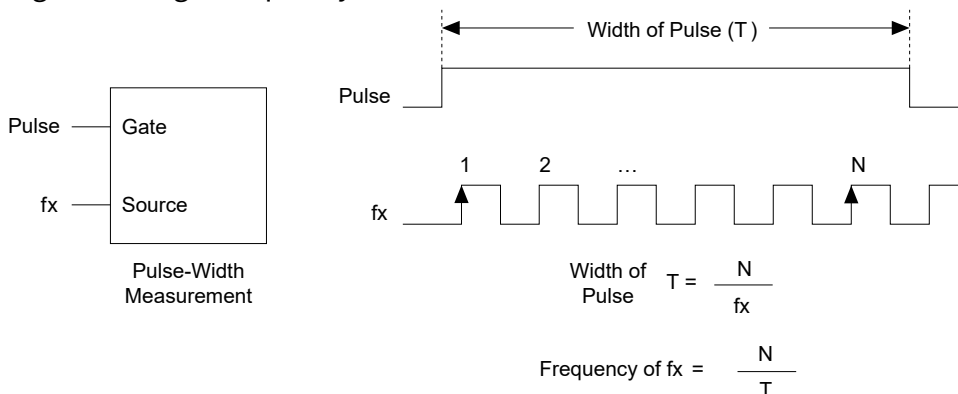


Note Counter 0 is always paired with Counter 1. Counter 2 is always paired with Counter 3.

In this method, you route a pulse of known duration (T) to the Gate of a counter. You can generate the pulse using a second counter. You can also generate the pulse externally and connect it to a PFI or RTSI terminal. You only need to use one counter if you generate the pulse externally.

Route the signal to measure (fx) to the Source of the counter. Configure the counter for a single pulse-width measurement. If you measure the width of pulse T to be N periods of fx, the frequency of fx is N/T. The following figure illustrates this method.

Figure 52. High Frequency with Two Counters



Another option is to measure the width of a known period instead of a known pulse.

Large Range of Frequencies with Two Counters

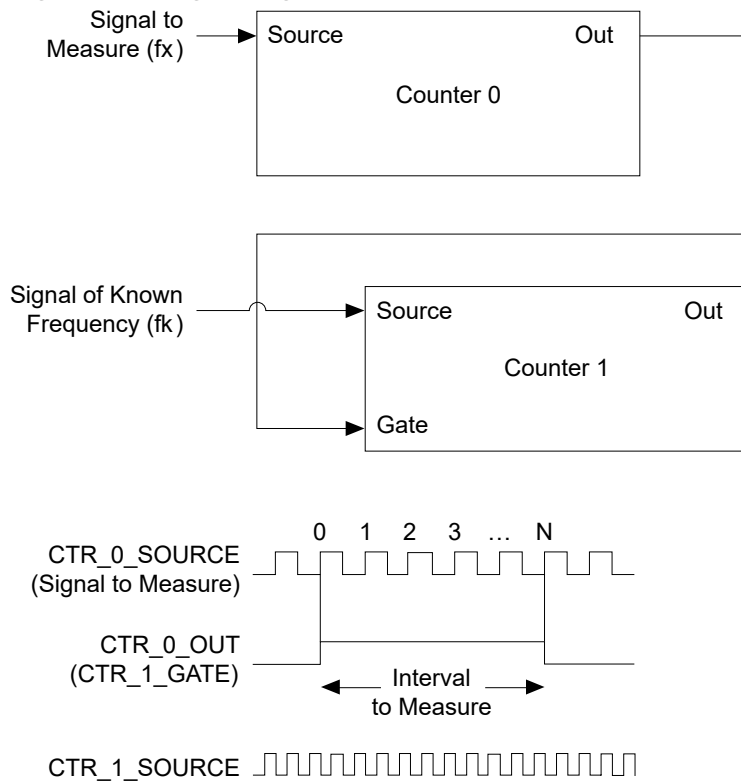
By using two counters, you can accurately measure a signal that might be high or low frequency. This technique is called reciprocal frequency measurement. When measuring a large range of frequencies with two counters, you generate a long pulse using the signal to measure. You then measure the long pulse with a known timebase. The X Series device can measure this long pulse more accurately than the faster input signal.



Note Counter 0 is always paired with Counter 1. Counter 2 is always paired with Counter 3.

You can route the signal to measure to the Source input of Counter 0, as shown in the following figure. Assume this signal to measure has frequency f_x . NI-DAQmx automatically configures Counter 0 to generate a single pulse that is the width of N periods of the source input signal.

Figure 53. Large Range of Frequencies with Two Counters



NI-DAQmx then routes the Counter 0 Internal Output signal to the gate of Counter 1. You can then route a signal of known frequency (f_k) as a counter timebase to the Counter 1 Source input. NI-DAQmx configures Counter 1 to perform a single pulse-width measurement. Suppose the result is that the pulse width is J periods of the f_k clock.

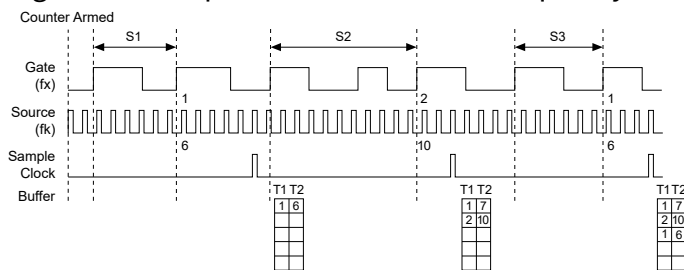
From Counter 0, the length of the pulse is N/f_x . From Counter 1, the length of the same pulse is J/f_k . Therefore, the frequency of f_x is given by $f_k = f_k * (N/J)$.

Sample Clocked Buffered Frequency Measurement

Sample clocked buffered point frequency measurements can either be a single frequency measurement or an average between sample clocks. Use `Cl.Freq.EnableAveraging` to set the behavior. For buffered frequency, the default is True. For hardware-timed single point (HWTSP), the default is False.

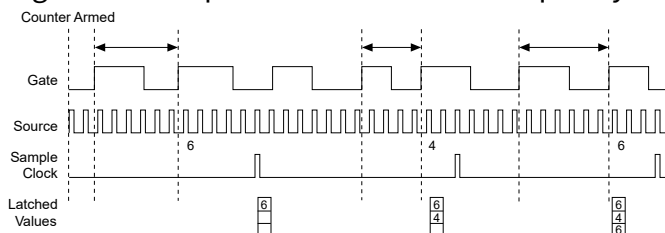
A sample clocked buffered frequency measurement with `Cl.Freq.EnableAveraging` set to True uses the embedded counter and a sample clock to perform a frequency measurement. For each sample clock period, the embedded counter counts the signal to measure (f_x) and the primary counter counts the internal time-base of a known frequency (f_k). Suppose T_1 is the number of ticks of the unknown signal counted between sample clocks and T_2 is the number of ticks counted of the known time-base. The frequency measured will be $f_x = f_k * (T_1/T_2)$.

Figure 54. Sample Clocked Buffered Frequency Measurement (Averaging)



When `Cl.Freq.EnableAveraging` is set to false, the frequency measurement returns the frequency of the pulse just before the sample clock. This single measurement is a single frequency measurement and is not an average between clocks.

Figure 55. Sample Clocked Buffered Frequency Measurement (Non-Averaging)



With sample clocked frequency measurements, ensure that the frequency to measure is twice as fast as the sample clock to prevent a measurement overflow.

Related concepts:

- [Cascading Counters](#)

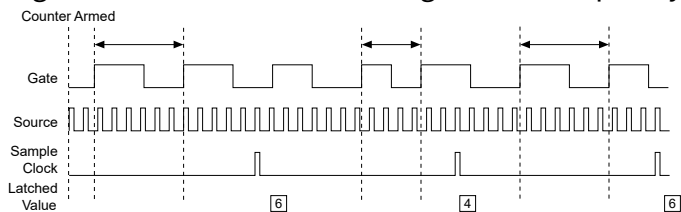
Related reference:

- [Connecting to Counter Signals](#)

Hardware-Timed Single Point Frequency Measurement

Hardware-timed single point (HWTSP) frequency measurements can either be a single frequency measurement or an average between sample clocks. Use `Cl.Freq.EnableAveraging` to set the behavior. For hardware-timed single point, the default is `False`.

Figure 56. Hardware-Timed Single Point Frequency Measurement



Choosing a Method for Measuring Frequency

The best method to measure frequency depends on several factors including the expected frequency of the signal to measure, the desired accuracy, how many counters are available, and how long the measurement can take.

Which Method is Best?

This depends on the frequency to be measured, the rate at which you want to monitor the frequency and the accuracy you desire.

- Low frequency measurements with one counter is a good method for many applications. However, the accuracy of the measurement decreases as the frequency increases.
- High frequency measurements with two counters is accurate for high frequency signals. However, the accuracy decreases as the frequency of the signal to measure decreases. At very low frequencies, this method may be too inaccurate for your application. Another disadvantage of this method is that it requires two counters (if you cannot provide an external signal of known width). An advantage of high

frequency measurements with two counters is that the measurement completes in a known amount of time.

- Measuring a large range of frequencies with two counters measures high and low frequency signals accurately. However, it requires two counters, and it has a variable sample time and variable error % dependent on the input signal.

The following table summarizes some of the differences in methods of measuring frequency.

Table 7. Frequency Measurement Method Comparison

Method	Number of Counters Used	Number of Measurements Returned	Measures High Frequency Signals Accurately	Measures Low Frequency Signals Accurately
Low frequency with one counter	1	1	Poor	Good
High frequency with two counters	1 or 2	1	Good	Poor
Large range of frequencies with two counters	2	1	Good	Good
Sample clocked (averaged)	1	1	Good	Good

The remainder of this topic provides more detail.

Considerations for Choosing Frequency Measurement Method

For all frequency measurement methods, assume the following:

f_x	is the frequency to be measured if no error
f_k	is the known source or gate frequency
measurement time (T)	is the time it takes to measure a single sample
Divide down (N)	is the integer to divide down measured frequency, only used in large range two counters
f_s	is the sample clock rate, only used in sample

	clocked frequency measurements
--	--------------------------------

Here is how these variables apply to each method, with a summary in the following table:

- **One counter**—With one counter measurements, a known timebase is used for the source frequency (f_k). The measurement time is the period of the frequency to be measured, or $1/f_k$.
- **Two counter high frequency**—With the two counter high frequency method, the second counter provides a known measurement time. The gate frequency equals $1/\text{measurement time}$.
- **Two counter large range**—The two counter larger range measurement is the same as a one counter measurement, but now the user has an integer divide down of the signal. An internal timebase is still used for the source frequency (f_k), but the divide down means that the measurement time is the period of the divided down signal, or N/f_x where N is the divide down.
- **Sample clocked**—For sample clocked frequency measurements, a known timebase is counted for the source frequency (f_k). The measurement time is the period of the sample clock (f_s).

Table 8. Frequency measurement methods

Variable	Sample Clocked	One Counter	Two Counter	
			High Frequency	Large Range
f_k	Known timebase	Known timebase	$\frac{1}{\text{gating period}}$	Known timebase
Measurement time	$\frac{1}{f_s}$	$\frac{1}{f_x}$	gating period	$\frac{N}{f_x}$
Max. frequency error	$f_x \times \frac{f_x}{f_k \times \left[\frac{f_x}{f_s} - 1 \right]}$	$f_x \times \frac{f_x}{f_k - f_x}$	f_k	$f_x \times \frac{f_x}{N \times f_k - f_x}$

Variable	Sample Clocked	One Counter	Two Counter	
			High Frequency	Large Range
Max. error %	$\frac{f_x}{f_k \times \left[\frac{f_x}{f_s} - 1 \right]}$	$\frac{f_x}{f_k - f_x}$	$\frac{f_k}{f_x}$	$\frac{f_x}{N \times f_k - f_x}$



Note Accuracy equations do not take clock stability into account. Refer to your device specifications for clock stability.

For a practical example, consider measuring a 50 kHz signal. Assuming that the measurement times for the sample clocked (with averaging) and two counter frequency measurements are configured the same, the following table summarizes the results.

Table 9. 50 kHz Frequency Measurement Methods

Variable	Sample Clocked	One Counter	Two Counter	
			High Frequency	Large Range
f_x	50,000	50,000	50,000	50,000
f_k	100 M	100 M	1,000	100 M
Measurement time (ms)	1	.02	1	1
N	—	—	—	50
Max. frequency error (Hz)	.512	25	1,000	.5
Max. error %	.00102	.05	2	.001

From these results, you can see that while the measurement time for one counter is shorter, the accuracy is best in the sample clocked and two counter large range measurements. For another example, the following table shows the results for 5 MHz.

Table 10. 5 MHz Frequency Measurement Methods

Variable	Sample Clocked	One Counter	Two Counter	
			High Frequency	Large Range
f_x	5 M	5 M	5 M	5 M
f_k	100 M	100 M	1,000	100 M
Measurement time (ms)	1	.0002	1	1
N	—	—	—	5,000
Max. Frequency error (Hz)	50.01	263 k	1,000	50
Max. Error %	.001	5.26	.02	.001

Again the measurement time for the one counter measurement is lowest, but the accuracy is lower. Note that the accuracy and measurement time of the sample clocked and two counter large range are almost the same. The advantage of the sample clocked method is that even when the frequency to measure changes, the measurement time does not and error percentage varies little. For example, if you configured a large range two counter measurement to use a divide down of 50 for a 50 k signal, then you would get the accuracy measurement time and accuracy listed in the preceding *50 kHz Frequency Measurement Methods* table. But if your signal ramped up to 5 M, then with a divide down of 50, your measurement time is 0.01 ms, but your error is now 0.1%. The error with a sample clocked frequency measurement is not as dependent on the measured frequency so at 50 k and 5 M with a measurement time of 1 ms the error percentage is still close to 0.001%. One of the disadvantages of a sample clocked frequency measurement is that the frequency to be measured must be at least twice the sample clock rate to ensure that a full period of the frequency to be measured occurs between sample clocks.

Period Measurement

In period measurements, the counter measures a period on its Gate input signal after the counter is armed. You can configure the counter to measure the period between two rising edges or two falling edges of the Gate input signal.

You can route an internal or external periodic clock signal (with a known period) to the Source input of the counter. The counter counts the number of rising (or falling) edges

occurring on the Source input between the two active edges of the Gate signal.

You can calculate the period of the Gate input by multiplying the period of the Source signal by the number of edges returned by the counter.

Period measurements return the inverse results of frequency measurements.

Position Measurement

You can use the counters to perform position measurements with quadrature encoders or two-pulse encoders. You can measure angular position with X1, X2, and X4 angular encoders. Linear position can be measured with two-pulse encoders.

You can choose to do either a single point (on-demand) position measurement or a buffered (sample clock) position measurement. You must arm a counter to begin position measurements.

Measurements Using Quadrature Encoders with X1, X2, or X4 encoding

When channel A leads channel B in a quadrature cycle, the counter increments on the rising edge of channel A. When channel B leads channel A in a quadrature cycle, the counter decrements on the falling edge of channel A.

The amount of increments and decrements per cycle depends on the type of encoding. X1 encoded cycles contain a single increment or decrement, X2 encoded cycles contain two increments or decrements, and X4 encoded cycles contain four increments or decrements. The counters can perform measurements of quadrature encoders that use X1, X2, or X4 encoding. The following figures illustrate X1, X2, and X4 encoding.

Figure 57. X1 Encoding

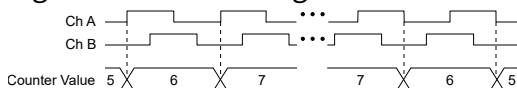


Figure 58. X2 Encoding

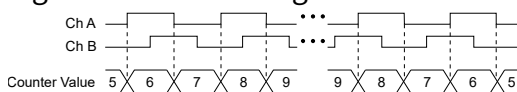
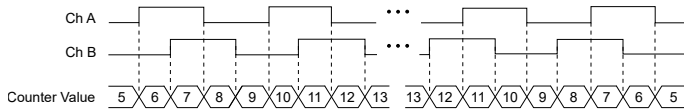


Figure 59. X4 Encoding



Measurements Using Quadrature Encoders with Channel Z

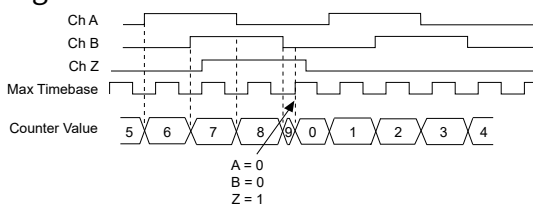
A quadrature encoder can have up to three channels—channels A, B, and Z.

Some quadrature encoders have a third channel, channel Z, which is also referred to as the index channel. A high level on channel Z causes the counter to be reloaded with a specified value in a specified phase of the quadrature cycle. You can program the counter reload to occur in any one of the four phases in a quadrature cycle.

Channel Z behavior—when it goes high and how long it stays high—differs with quadrature encoder designs. You must refer to the documentation for your quadrature encoder to obtain timing of channel Z with respect to channels A and B. You must then ensure that channel Z is high during at least a portion of the phase you specify for reload. For instance, in the following figure, channel Z is never high when channel A is high and channel B is low. Thus, the reload must occur in some other phase.

In the following figure, the reload phase is when both channel A and channel B are low. The reload occurs when the phase is true and channel Z is high. Incrementing and decrementing takes priority over reloading. Thus, when the channel B goes low to enter the reload phase, the increment occurs first. The reload occurs within one maximum timebase period after the reload phase becomes true. After the reload occurs, the counter continues to count as before. The following figure illustrates channel Z reload with X4 decoding.

Figure 60. Channel Z Reload with X4 Decoding

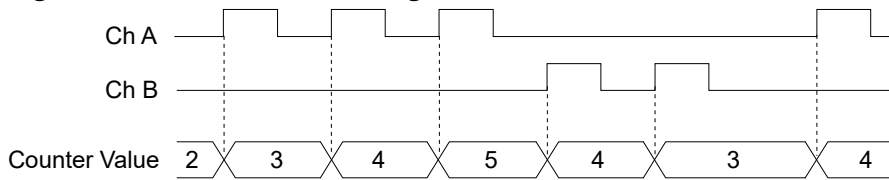


Measurements Using Two Pulse Encoders

The counter supports two pulse encoders that have two channels—channels A and B. The counter increments on each rising edge of channel A. The counter decrements on

each rising edge of channel B, as shown in the following figure.

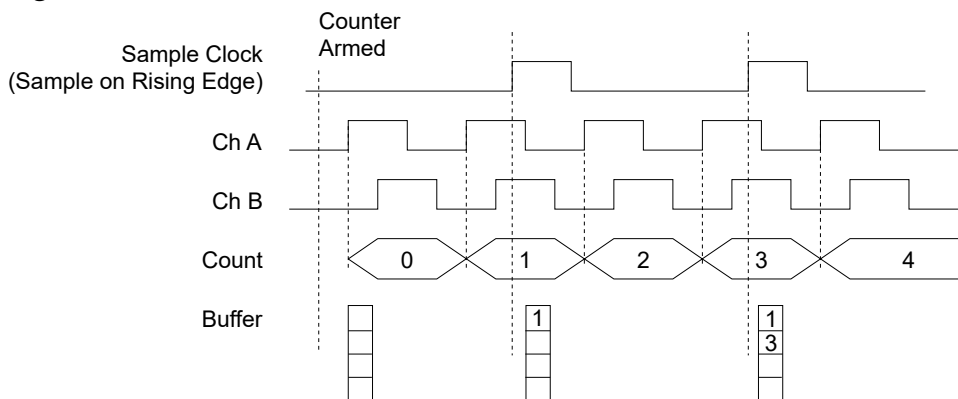
Figure 61. Measurements Using Two Pulse Encoders



Buffered (Sample Clock) Position Measurement

With buffered position measurement (position measurement using a sample clock), the counter increments based on the encoding used after the counter is armed. The value of the counter is sampled on each active edge of a sample clock. A DMA controller transfers the sampled values to host memory. The count values returned are the cumulative counts since the counter armed event; that is, the sample clock does not reset the counter. You can route the counter sample clock to the Gate input of the counter. You can configure the counter to sample on the rising or falling edge of the sample clock. The following figure shows an example of a buffered X1 position measurement.

Figure 62. Buffered Position Measurement



Related reference:

- [Connecting to Counter Signals](#)

Hardware-Timed Single Point Position Measurement

A hardware-timed single point (HWTSP) position measurement has the same behavior as a buffered (sample clock) position measurement.

Two-Signal Edge-Separation Measurement

Two-signal edge-separation measurement is similar to pulse-width measurement, except that there are two measurement signals—Aux and Gate. An active edge on the Aux input starts the counting and an active edge on the Gate input stops the counting. You must arm a counter to begin a two edge separation measurement.

After the counter has been armed and an active edge occurs on the Aux input, the counter counts the number of rising (or falling) edges on the Source. The counter ignores additional edges on the Aux input.

The counter stops counting upon receiving an active edge on the Gate input. The counter stores the count in the FIFO.

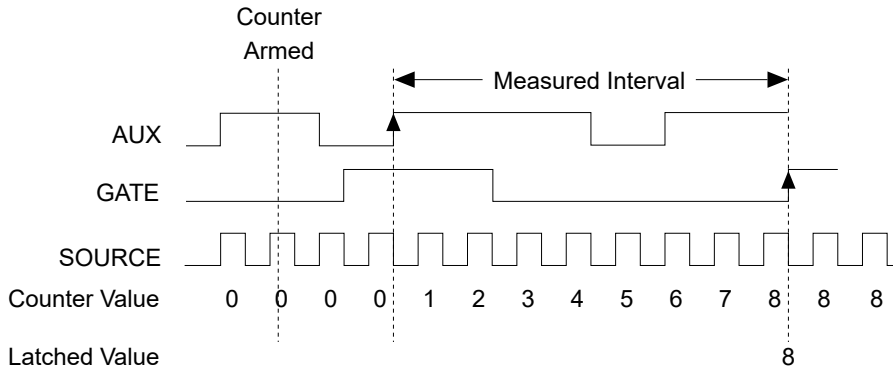
You can configure the rising or falling edge of the Aux input to be the active edge. You can configure the rising or falling edge of the Gate input to be the active edge.

Use this measurement type to count events or measure the time that occurs between edges on two signals. This type of measurement is sometimes referred to as start/stop trigger measurement, second gate measurement, or A-to-B measurement.

Single Two-Signal Edge-Separation Measurement

With single two-signal edge-separation measurement, the counter counts the number of rising (or falling) edges on the Source input occurring between an active edge of the Gate signal and an active edge of the Aux signal. The counter then stores the count in the FIFO and ignores other edges on its inputs. Software then reads the stored count. The following figure shows an example of a single two-signal edge-separation measurement.

Figure 63. Single Two-Signal Edge-Separation Measurement

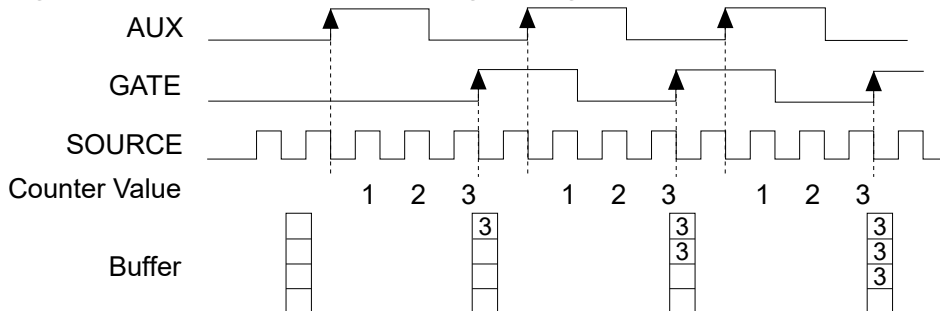


Implicit Buffered Two-Signal Edge-Separation Measurement

Implicit buffered and single two-signal edge-separation measurements are similar, but implicit buffered measurement measures multiple intervals.

The counter counts the number of rising (or falling) edges on the Source input occurring between an active edge of the Gate signal and an active edge of the Aux signal. The counter then stores the count in the FIFO. On the next active edge of the Gate signal, the counter begins another measurement. A DMA controller transfers the stored values to host memory. The following figure shows an example of an implicit buffered two-signal edge-separation measurement.

Figure 64. Implicit Buffered Two-Signal Edge-Separation Measurement



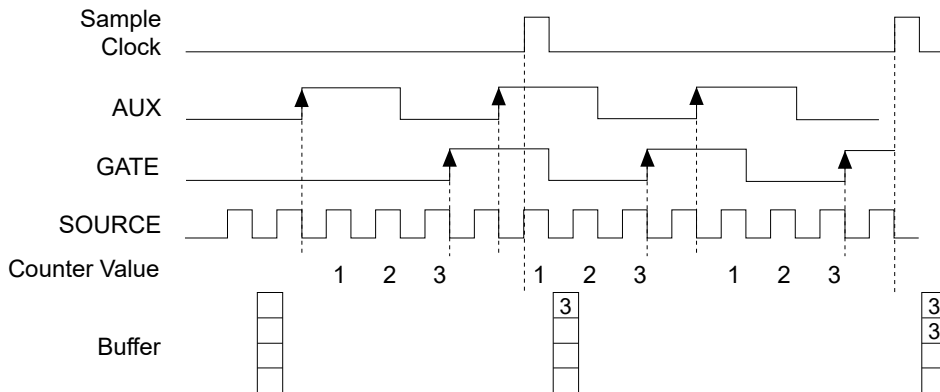
Sample Clocked Buffered Two-Signal Separation Measurement

A sample clocked buffered two-signal separation measurement is similar to single two-signal separation measurement, but buffered two-signal separation measurement takes measurements over multiple intervals correlated to a sample clock. The counter counts the number of rising (or falling) edges on the Source input occurring between an active edge of the Gate signal and an active edge of the Aux signal. The counter then stores the count in the FIFO on a sample clock edge. On the next active edge of the

Gate signal, the counter begins another measurement. A DMA controller transfers the stored values to host memory.

The following figure shows an example of a sample clocked buffered two-signal separation measurement.

Figure 65. Sample Clocked Buffered Two-Signal Separation Measurement



Related reference:

- [Connecting to Counter Signals](#)

Hardware-Timed Single Point Two-Signal Separation Measurement

A hardware-timed single point (HWTSP) two-signal separation measurement has the same behavior as a sample clocked buffered two-signal separation measurement.



Note If an active edge on the Gate and an active edge on the AUX does not occur between sample clocks, an overrun error occurs.

Counter Output Applications

Refer to the following sections for information about the various counter output applications that are supported by the PCIe-6353 and PXIe-6353.

Simple Pulse Generation

Single Pulse Generation

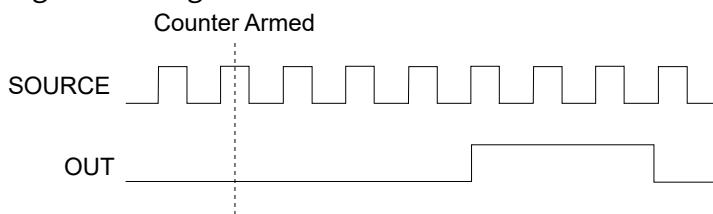
The counter can output a single pulse. The pulse appears on the Counter n Internal Output signal of the counter.

You can specify a delay from when the counter is armed to the beginning of the pulse. The delay is measured in terms of a number of active edges of the Source input.

You can specify a pulse width. The pulse width is also measured in terms of a number of active edges of the Source input. You can also specify the active edge of the Source input (rising or falling).

The following figure shows a generation of a pulse with a pulse delay of four and a pulse width of three (using the rising edge of Source).

Figure 66. Single Pulse Generation



Single Pulse Generation with Start Trigger

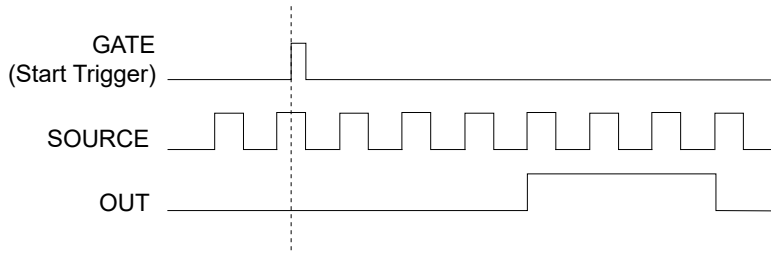
The counter can output a single pulse in response to one pulse on a hardware Start Trigger signal. The pulse appears on the Counter n Internal Output signal of the counter.

You can route the Start Trigger signal to the Gate input of the counter. You can specify a delay from the Start Trigger to the beginning of the pulse. You can also specify the pulse width. The delay and pulse width are measured in terms of a number of active edges of the Source input.

After the Start Trigger signal pulses once, the counter ignores the Gate input.

The following figure shows a generation of a pulse with a pulse delay of four and a pulse width of three (using the rising edge of Source).

Figure 67. Single Pulse Generation with Start Trigger



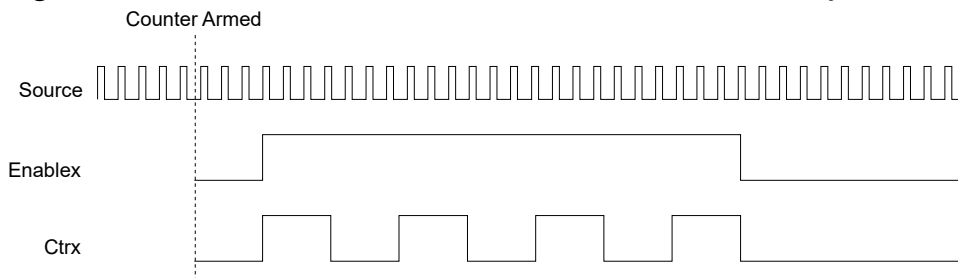
Pulse Train Generation

Refer to the following sections for information about pulse train generation options supported by the PCIe-6353 and PXIe-6353.

Finite Pulse Train Generation

Finite pulse train generation creates a train of pulses with programmable frequency and duty cycle for a predetermined number of pulses, as shown in the following figure. With X Series counters, the primary counter generates the specified pulse train and the embedded counter counts the pulses generated by the primary counter. When the embedded counter reaches the specified tick count, it generates a trigger that stops the primary counter generation.

Figure 68. Finite Pulse Train Generation: Four Ticks Initial Delay, Four Pulses



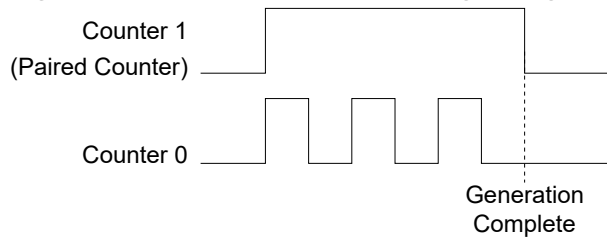
In Legacy Mode, the counter operation requires two counters and does not use the embedded counter. For example, to generate four pulses on Counter 0, Counter 0 generates the pulse train, which is gated by the paired second counter. The paired counter, Counter 1, generates a pulse of desired width.



Note Counter 0 is always paired with Counter 1. Counter 2 is always paired with Counter 3.

The routing is done internally. The following figure shows an example finite pulse train timing diagram.

Figure 69. Finite Pulse Train Timing in Legacy Mode



Retriggerable Pulse or Pulse Train Generation

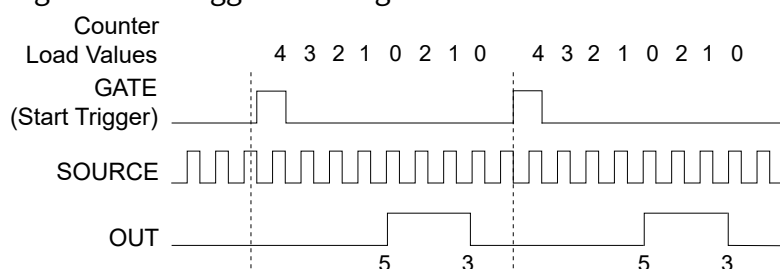
The counter can output a single pulse or multiple pulses in response to each pulse on a hardware Start Trigger signal. The generated pulses appear on the Counter n Internal Output signal of the counter.

You can route the Start Trigger signal to the Gate input of the counter. You can specify a delay from the Start Trigger to the beginning of each pulse. You can also specify the pulse width. The delay and pulse width are measured in terms of a number of active edges of the Source input. The initial delay can be applied to only the first trigger or to all triggers using the `CO.EnableInitialDelayOnRetrigger` property. The default for a single pulse is `True`, while the default for finite pulse trains is `False`.

The counter ignores the Gate input while a pulse generation is in progress. After the pulse generation is finished, the counter waits for another Start Trigger signal to begin another pulse generation. For retriggered pulse generation, pause triggers are not allowed since the pause trigger also uses the gate input.

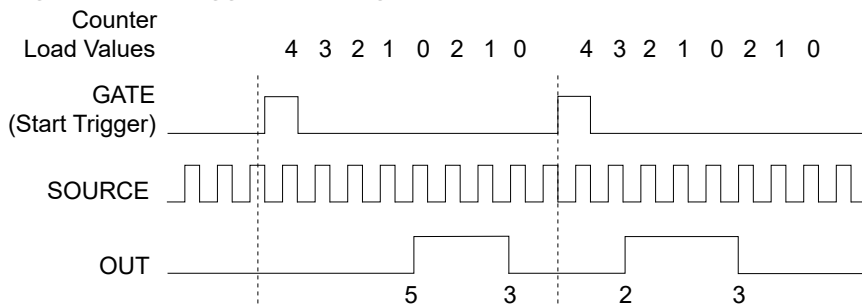
The following figure shows a generation of two pulses with a pulse delay of five and a pulse width of three (using the rising edge of Source) with `CO.EnableInitialDelayOnRetrigger` set to the default `True`.

Figure 70. Retriggerable Single Pulse Generation with Initial Delay on Retrigger



The following figure shows the same pulse train with CO.EnableInitialDelayOnRetrigger set to the default False.

Figure 71. Retriggerable Single Pulse Generation with Initial Delay on Retrigger Set to False



The minimum time between the trigger and the first active edge is two ticks of the source.

Continuous Pulse Train Generation

Continuous pulse train generation creates a train of pulses with programmable frequency and duty cycle. The pulses appear on the Counter n Internal Output signal of the counter.

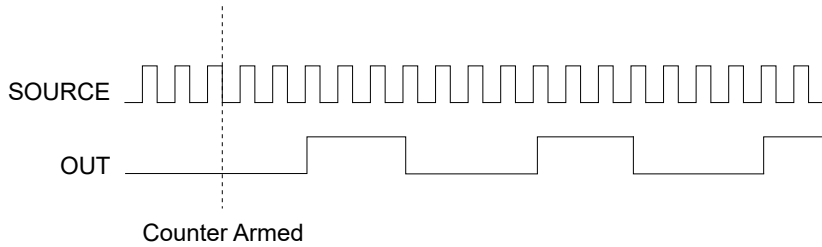
You can specify a delay from when the counter is armed to the beginning of the pulse train. The delay is measured in terms of a number of active edges of the Source input.

You specify the high and low pulse widths of the output signal. The pulse widths are also measured in terms of a number of active edges of the Source input. You can also specify the active edge of the Source input (rising or falling).

The counter can begin the pulse train generation as soon as the counter is armed, or in response to a hardware Start Trigger. You can route the Start Trigger to the Gate input of the counter.

You can also use the Gate input of the counter as a Pause Trigger (if it is not used as a Start Trigger). The counter pauses pulse generation when the Pause Trigger is active. The following figure shows a continuous pulse train generation (using the rising edge of Source).

Figure 72. Continuous Pulse Train Generation



Continuous pulse train generation is sometimes called frequency division. If the high and low pulse widths of the output signal are M and N periods, then the frequency of the Counter n Internal Output signal is equal to the frequency of the Source input divided by $M + N$.

Buffered Pulse Train Generation

X Series counters can use the FIFO to perform a buffered pulse train generation. Buffered pulse train generation can use implicit timing or sample clock timing. When using implicit timing, the pulse idle time and active time changes with each sample you write. With sample clocked timing, each sample you write updates the idle time and active time of your generation on each sample clock edge. Idle time and active time can also be defined in terms of frequency and duty cycle or idle ticks and active ticks.



Note On buffered implicit pulse trains, the pulse specifications in the DAQmx Create Counter Output Channel are ignored so that you generate the number of pulses defined in the multipoint write. On buffered sample clock pulse trains, the pulse specifications in the DAQmx Create Counter Output Channel are generated after the counters start, and before the first sample clock, so that you generate the number of updates defined in the multipoint write.

Finite Implicit Buffered Pulse Train Generation

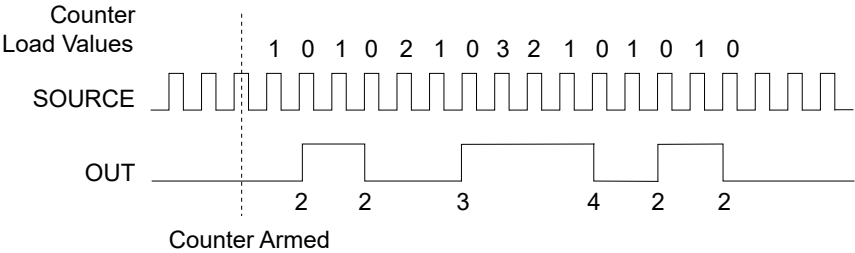
Finite implicit buffered pulse train generation creates a predetermined number of pulses with variable idle and active times. Each point you write generates a single pulse. The number of pairs of idle and active times (pulse specifications) you write determines the number of pulses generated. All points are generated back to back to create a user defined pulse train.

The following table and figure detail a finite implicit generation of three samples.

Table 11. Finite implicit buffered pulse train generation

Sample	Idle Ticks	Active Ticks
1	2	2
2	3	4
3	2	2

Figure 73. Finite Implicit Buffered Pulse Train Generation



Continuous Buffered Implicit Pulse Train Generation

Continuous buffered implicit pulse train generation creates a continuous train of pulses with variable idle and active times. Instead of generating a set number of data samples and stopping, a continuous generation continues until you stop the operation. Each point you write generates a single pulse. All points are generated back to back to create a user defined pulse train.

Finite Buffered Sample Clocked Pulse Train Generation

Finite buffered sample clocked pulse train generation creates a predetermined number of pulse train updates. Each point you write defines pulse specifications that are updated with each sample clock. When a sample clock occurs, the current pulse (idle followed by active) finishes generation and the next pulse updates with the next sample specifications.



Note When the last sample is generated, the pulse train continues to generate with these specifications until the task is stopped.

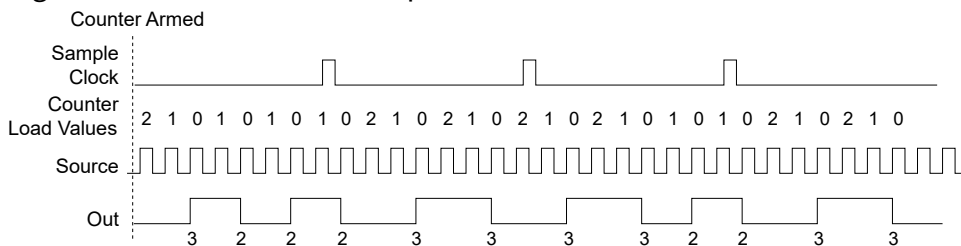
The following table and figure detail a finite sample clocked generation of three

samples where the pulse specifications from the create channel are two ticks idle, two ticks active, and three ticks initial delay.

Table 12. Finite Buffered Sample Clocked Pulse Train Generation

Sample	Idle Ticks	Active Ticks
1	3	3
2	2	2
3	3	3

Figure 74. Finite Buffered Sample Clocked Pulse Train Generation



There are several different methods of continuous generation that control what data is written. These methods are regeneration, FIFO regeneration, and non-regeneration modes.

Regeneration is the repetition of the data that is already in the buffer.

Standard regeneration is when data from the PC buffer is continually downloaded to the FIFO to be written out. New data can be written to the PC buffer at any time without disrupting the output. With FIFO regeneration, the entire buffer is downloaded to the FIFO and regenerated from there. Once the data is downloaded, new data cannot be written to the FIFO. To use FIFO regeneration, the entire buffer must fit within the FIFO size. The advantage of using FIFO regeneration is that it does not require communication with the main host memory once the operation is started, thereby preventing any problems that may occur due to excessive bus traffic.

With non-regeneration, old data is not repeated. New data must be continually written to the buffer. If the program does not write new data to the buffer at a fast enough rate to keep up with the generation, the buffer underflows and causes an error.

Continuous Buffered Sample Clocked Pulse Train Generation

Continuous buffered sample clocked pulse train generation creates a continuous train of pulses with variable idle and active times. Instead of generating a set number of data samples and stopping, a continuous generation continues until you stop the operation. Each point you write specifies pulse specifications that are updated with each sample clock. When a sample clock occurs, the current pulse finishes generation and the next pulse uses the next sample specifications.

Related reference:

- [Connecting to Counter Signals](#)

Frequency Generation

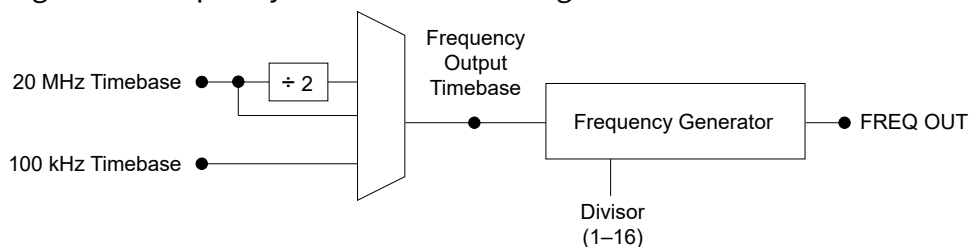
You can generate a frequency by using a counter in pulse train generation mode or by using the frequency generator circuit.

Using the Frequency Generator

The frequency generator can output a square wave at many different frequencies. The frequency generator is independent of the four general-purpose 32-bit counter/timer modules on X Series devices.

The following figure shows a block diagram of the frequency generator.

Figure 75. Frequency Generator Block Diagram

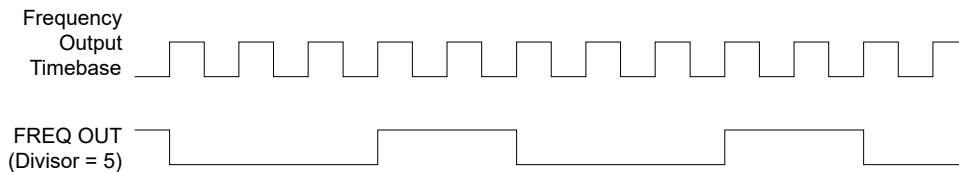


The frequency generator generates the Frequency Output signal. The Frequency Output signal is the Frequency Output Timebase divided by a number you select from 1 to 16. The Frequency Output Timebase can be either the 20 MHz Timebase, the 20 MHz Timebase divided by 2, or the 100 kHz Timebase.

The duty cycle of Frequency Output is 50% if the divider is either 1 or an even number. For an odd divider, suppose the divider is set to D . In this case, Frequency Output is low for $(D + 1)/2$ cycles and high for $(D - 1)/2$ cycles of the Frequency Output Timebase.

The following figure shows the output waveform of the frequency generator when the divider is set to 5.

Figure 76. Frequency Generator Output Waveform



Frequency Output can be routed out to any PFI <0..15> or RTSI <0..7> terminal. All PFI terminals are set to high-impedance at startup. The FREQ OUT signal can also be routed to many internal timing signals.

In software, program the frequency generator as you would program one of the counters for pulse train generation.

Frequency Division

The counters can generate a signal with a frequency that is a fraction of an input signal. This function is equivalent to continuous pulse train generation.

Related reference:

- [Connecting to Counter Signals](#)

Pulse Generation for ETS

In the equivalent time sampling (ETS) application, the counter produces a pulse on the output a specified delay after an active edge on Gate. After each active edge on Gate, the counter cumulatively increments the delay between the Gate and the pulse on the output by a specified amount. Thus, the delay between the Gate and the pulse produced successively increases.

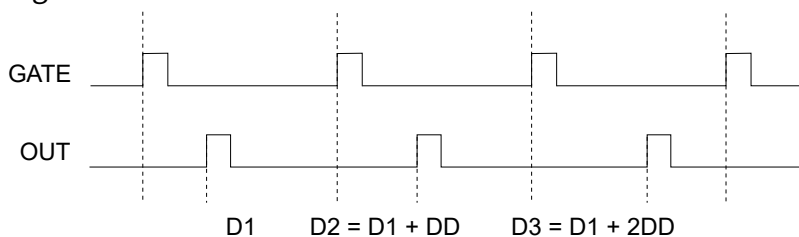
The increase in the delay value can be between 0 and 255. For instance, if you specify

the increment to be 10, the delay between the active Gate edge and the pulse on the output increases by 10 every time a new pulse is generated.

Suppose you program your counter to generate pulses with a delay of 100 and pulse width of 200 each time it receives a trigger. Furthermore, suppose you specify the delay increment to be 10. On the first trigger, your pulse delay is 100, on the second it is 110, on the third it is 120; the process repeats until the counter is disarmed. The counter ignores any Gate edge that is received while the pulse triggered by the previous Gate edge is in progress.

The waveform thus produced at the counter's output can be used to provide timing for undersampling applications where a digitizing system can sample repetitive waveforms that are higher in frequency than the Nyquist frequency of the system. The following figure shows an example of pulse generation for ETS; the delay from the trigger to the pulse increases after each subsequent Gate active edge.

Figure 77. Pulse Generation for ETS



Related reference:

- [Connecting to Counter Signals](#)

Counter Timing Signals



Note In this section, n refers to the source input to X Series Counter 0, 1, 2, or 3, inclusively. For example, Counter n Source refers to four signals: Counter 0 Source, Counter 1 Source, Counter 2 Source, or Counter 3 Source.



Note All counter timing signals can be filtered. Refer to the *PFI Filters* topic for more information.

Counter n Source Signal

The selected edge of the Counter n Source signal increments and decrements the counter value depending on the application the counter is performing.

The following table lists how the terminal is used in various applications.

Table 13. Counter Applications and Counter n Source

Application	Purpose of Source Terminal
Pulse Generation	Counter Timebase
One Counter Time Measurements	Counter Timebase
Two Counter Time Measurements	Input Terminal
Non-Buffered Edge Counting	Input Terminal
Buffered Edge Counting	Input Terminal
Two-Edge Separation	Counter Timebase

Routing a Signal to Counter n Source

Each counter has independent input selectors for the Counter n Source signal. Any of the following signals can be routed to the Counter n Source input:

- PFI <0..15>
- PXI_STAR
- PXIe_DSTAR<A,B>
- RTSI <0..7>
- 100 MHz Timebase
- 20 MHz Timebase
- 100 kHz Timebase
- PXI_CLK10
- Analog Comparison Event
- Change Detection Event

In addition, TC or Gate from a counter can be routed to a different counter source.



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Some of these options may not be available in some driver software.

Routing Counter n Source to an Output Terminal

You can route Counter n Source out to any PFI <0..15>, RTSI <0..7>, or PXIe_DSTARC terminal. All PFIs are set to high-impedance at startup.



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Counter n Gate Signal

The Counter n Gate signal can perform many different operations depending on the application including starting and stopping the counter, and saving the counter contents.

Routing a Signal to Counter n Gate

Each counter has independent input selectors for the Counter n Gate signal. Any of the following signals can be routed to the Counter n Gate input:

- PFI <0..15>
- PXI_STAR
- PXIe_DSTAR<A,B>
- RTSI <0..7>
- AI Reference Trigger (ai/ReferenceTrigger)
- AI Start Trigger (ai/StartTrigger)
- AO Sample Clock (ao/SampleClock)
- DI Sample Clock (di/SampleClock)
- DI Reference Trigger (di/ReferenceTrigger)
- DO Sample Clock (do/SampleClock)

- Change Detection Event
- Analog Comparison Event

In addition, a counter's Internal Output or Source can be routed to a different counter's gate.



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Some of these options may not be available in some driver software.

Routing Counter *n* Gate to an Output Terminal

You can route Counter *n* Gate out to any PFI <0..15>, RTSI <0..7>, or PXIe_DSTARC terminal. All PFIs are set to high-impedance at startup.



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Counter *n* Aux Signal

The Counter *n* Aux signal indicates the first edge in a two-signal edge-separation measurement.

Routing a Signal to Counter *n* Aux

Each counter has independent input selectors for the Counter *n* Aux signal. Any of the following signals can be routed to the Counter *n* Aux input:

- PFI <0..15>
- RTSI <0..7>
- PXI_STAR
- PXIe_DSTAR<A,B>
- AI Reference Trigger (ai/ReferenceTrigger)

- AI Start Trigger (ai/StartTrigger)
- Analog Comparison Event
- Change Detection Event

In addition, a counter's Internal Output, Gate or Source can be routed to a different counter's Aux. A counter's own gate can also be routed to its Aux input.



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Some of these options may not be available in some driver software.

Counter *n* A, Counter *n* B, and Counter *n* Z Signals

Counter *n* B can control the direction of counting in edge counting applications. Use the A, B, and Z inputs to each counter when measuring quadrature encoders or measuring two pulse encoders.

Routing Signals to A, B, and Z Counter Inputs

Each counter has independent input selectors for each of the A, B, and Z inputs. Any of the following signals can be routed to each input:

- PFI <0..15>
- RTSI <0..7>
- PXI_STAR
- PXIe_DSTAR<A,B>
- Analog Comparison Event



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Routing Counter n Z Signal to an Output Terminal

You can route Counter n Z out to any RTSI <0..7> terminal.

Counter n Up_Down Signal

Counter n Up_Down is another name for the Counter n B signal.

Counter n HW Arm Signal

The Counter n HW Arm signal enables a counter to begin an input or output function.

To begin any counter input or output function, you must first enable, or arm, the counter. In some applications, such as a buffered edge count, the counter begins counting when it is armed. In other applications, such as single pulse-width measurement, the counter begins waiting for the Gate signal when it is armed. Counter output operations can use the arm signal in addition to a start trigger.

Software can arm a counter or configure counters to be armed on a hardware signal. Software calls this hardware signal the Arm Start Trigger. Internally, software routes the Arm Start Trigger to the Counter n HW Arm input of the counter.

Routing Signals to Counter n HW Arm Input

Any of the following signals can be routed to the Counter n HW Arm input:

- PFI <0..15>
- RTSI <0..7>
- PXI_STAR
- PXIe_DSTAR<A,B>
- AI Reference Trigger (ai/ReferenceTrigger)
- AI Start Trigger (ai/StartTrigger)
- Analog Comparison Event
- Change Detection Event

The Internal Output of a counter can be routed to the HW Arm of another counter.



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Some of these options may not be available in some driver software.

Counter n Sample Clock Signal

Use the Counter n Sample Clock (Ctr n SampleClock) signal to perform sample clocked acquisitions and generations.

You can specify an internal or external source for Counter n Sample Clock. You can also specify whether the measurement sample begins on the rising edge or falling edge of Counter n Sample Clock.

If the DAQ device receives a Counter n Sample Clock when the FIFO is full, it reports an overflow error to the host software.

Related concepts:

- [Device Routing in MAX](#)

Using an Internal Source

To use Counter n Sample Clock with an internal source, specify the signal source and the polarity of the signal. The source can be any of the following signals:

- DI Sample Clock (di/SampleClock)
- DO Sample Clock (do/SampleClock)
- AI Sample Clock (ai/SampleClock)
- AI Convert Clock (ai/ConvertClock)
- AO Sample Clock (ao/SampleClock)
- DI Change Detection output

Several other internal signals can be routed to Counter n Sample Clock through internal routes. Refer to *Device Routing in MAX* in the *NI-DAQmx Help* for more information.

Using an External Source

You can route any of the following signals as Counter n Sample Clock:

- PFI <0..15>
- PXI_STAR
- PXIe_DSTAR<A,B>
- RTSI <0..7>
- Analog Comparison Event



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

You can sample data on the rising or falling edge of Counter n Sample Clock.

Routing Counter n Sample Clock to an Output Terminal

You can route Counter n Sample Clock out to any PFI <0..15> terminal. The PFI circuitry inverts the polarity of Counter n Sample Clock before driving the PFI terminal.

Counter n Internal Output and Counter n TC Signals

The Counter n Internal Output signal changes in response to Counter n TC.

The two software-selectable output options are pulse output on TC and toggle output on TC. The output polarity is software-selectable for both options.

With pulse or pulse train generation tasks, the counter drives the pulse(s) on the Counter n Internal Output signal. The Counter n Internal Output signal can be internally routed to be a counter/timer input or an “external” source for AI, AO, DI, or DO timing signals.

Routing Counter n Internal Output to an Output Terminal

You can route Counter n Internal Output to any PFI <0..15>, RTSI <0..7>, or PXIe_DSTARC terminal. All PFIs are set to high-impedance at startup.



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Frequency Output Signal

The Frequency Output (FREQ OUT) signal is the output of the frequency output generator.

Routing Frequency Output to a Terminal

You can route Frequency Output to any PFI <0..15> or PXIe_DSTARC terminal. All PFIs are set to high-impedance at startup. The FREQ OUT signal can also be routed to DO Sample Clock and DI Sample Clock.

Counter Triggering

Counters support three different triggering actions.

Arm Start Trigger

To begin any counter input or output function, you must first enable, or arm, the counter. Software can arm a counter or configure counters to be armed on a hardware signal. Software calls this hardware signal the Arm Start Trigger. Internally, software routes the Arm Start Trigger to the Counter n HW Arm input of the counter.

For counter output operations, you can use it in addition to the start and pause triggers. For counter input operations, you can use the arm start trigger to have start trigger-like behavior. The arm start trigger can be used for synchronizing multiple counter input and output tasks.

When using an arm start trigger, the arm start trigger source is routed to the Counter n HW Arm signal.

Start Trigger

For counter output operations, a start trigger can be configured to begin a finite or

continuous pulse generation. Once a continuous generation has triggered, the pulses continue to generate until you stop the operation in software. For finite generations, the specified number of pulses is generated and the generation stops unless you use the retriggerable attribute. When you use this attribute, subsequent start triggers cause the generation to restart.

When using a start trigger, the start trigger source is routed to the Counter n Gate signal input of the counter.

Counter input operations can use the arm start trigger to have start trigger-like behavior.

Pause Trigger

You can use pause triggers in edge counting and continuous pulse generation applications. For edge counting acquisitions, the counter stops counting edges while the external trigger signal is low and resumes when the signal goes high or vice versa. For continuous pulse generations, the counter stops generating pulses while the external trigger signal is low and resumes when the signal goes high or vice versa.

When using a pause trigger, the pause trigger source is routed to the Counter n Gate signal input of the counter.

Cascading Counters

You can internally route the Counter n Internal Output and Counter n TC signals of each counter to the Gate inputs of the other counter. By cascading two counters together, you can effectively create a 64-bit counter.

By cascading counters, you can also enable other applications. For example, to improve the accuracy of frequency measurements, use reciprocal frequency measurement, as described in the frequency generation *Large Range of Frequencies with Two Counters* section.

Related concepts:

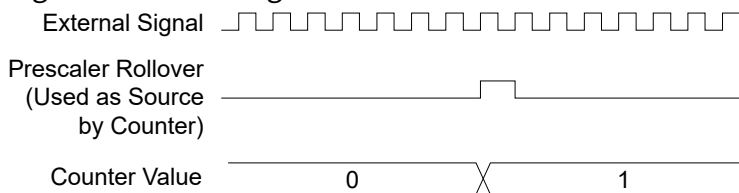
- [Frequency Measurement](#)

Counter Signal Prescaling

Prescaling allows the counter to count a signal that is faster than the maximum timebase of the counter. X Series devices offer 8X and 2X prescaling on each counter (prescaling can be disabled). Each prescaler consists of a small, simple counter that counts to eight (or two) and rolls over. This counter can run faster than the larger counters, which simply count the rollovers of this smaller counter. Thus, the prescaler acts as a frequency divider on the Source and puts out a frequency that is one-eighth (or one-half) of what it is accepting.

The following figure illustrates prescaling.

Figure 78. Prescaling



Prescaling is intended to be used for frequency measurement where the measurement is made on a continuous, repetitive signal. The prescaling counter cannot be read; therefore, you cannot determine how many edges have occurred since the previous rollover. Prescaling can be used for event counting provided it is acceptable to have an error of up to seven (or one) ticks. Prescaling can be used when the counter Source is an external signal. Prescaling is not available if the counter Source is one of the internal timebases (100MHzTimebase, 20MHzTimebase, or 100kHzTimebase).

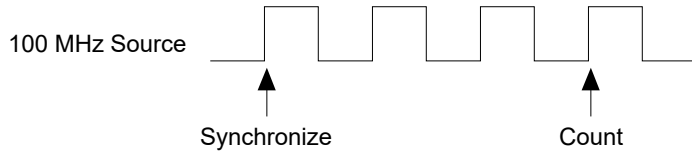
Counter Signal Synchronization Modes

The 32-bit counter counts up or down synchronously with the Source signal. The Gate signal and other counter inputs are asynchronous to the Source signal, so X Series devices synchronize these signals before presenting them to the internal counter.

100 MHz Source Mode

In 100 MHz source mode, the device synchronizes signals on the rising edge of the source, and counts on the third rising edge of the source. Edges are pipelined so no counts are lost, as shown in the following figure.

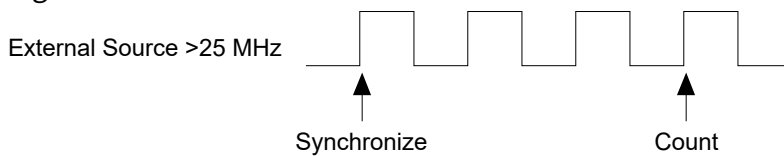
Figure 79.



External Source Greater than 25 MHz

With an external source greater than 25 MHz, the device synchronizes signals on the rising edge of the source, and counts on the third rising edge of the source. Edges are pipelined so no counts are lost, as shown in the following figure.

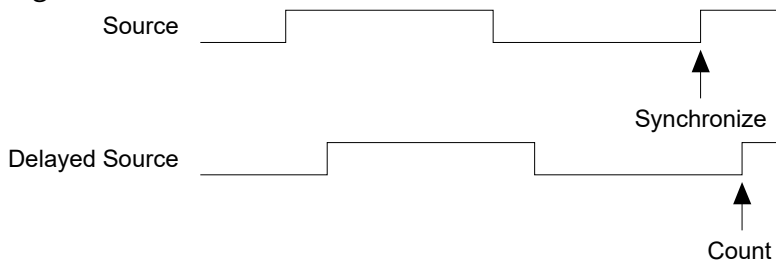
Figure 80.



External or Internal Source Less than 25 MHz

With an external or internal source less than 25 MHz, the device generates a delayed Source signal by delaying the Source signal by several nanoseconds. The device synchronizes signals on the rising edge of the delayed Source signal, and counts on the following rising edge of the source, as shown in the following figure.

Figure 81.



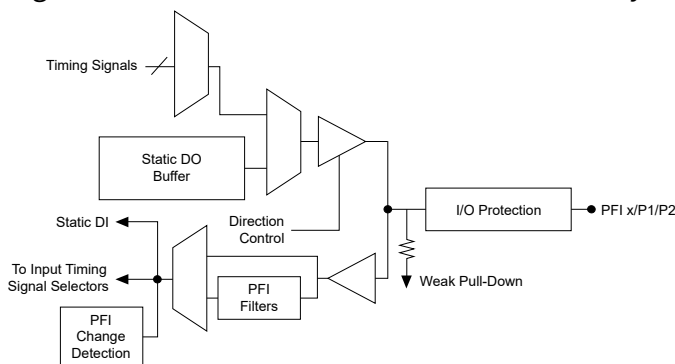
PCIe-6353 and PXIe-6353 PFI

Each Programmable Function Interface (PFI) can be individually configured as the following:

- A static digital input
- A static digital output
- A timing input signal for AI, AO, DI, DO, or counter/timer functions
- A timing output signal from AI, AO, DI, DO, or counter/timer functions

Each PFI input also has a programmable debouncing filter. The following figure shows the circuitry of one PFI line. Each PFI line is similar.

Figure 82. PCIe-6353 and PXIe-6353 PFI Circuitry



When a terminal is used as a timing input or output signal, it is called PFI x (where x is an integer from 0 to 15).

When a terminal is used as a static digital input or output, it is called P1.x or P2.x. On the I/O connector, each terminal is labeled PFI x/P1.x or PFI x/P2.x.

The voltage input and output levels and the current drive levels of the PFI signals are listed in the device specifications.



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

Using PFI Terminals as Timing Input Signals

Use PFI terminals to route external timing signals to many different X Series functions. Each PFI terminal can be routed to any of the following signals:

- AI Convert Clock (ai/ConvertClock)
- AI Sample Clock (ai/SampleClock)
- AI Sample Clock Timebase (ai/SampleClockTimebase)
- AI Start Trigger (ai/StartTrigger)
- AI Reference Trigger (ai/ReferenceTrigger)
- AI Pause Trigger (ai/PauseTrigger)
- AO Start Trigger (ao/StartTrigger)
- AO Sample Clock (ao/SampleClock)
- AO Sample Clock Timebase (ao/SampleClockTimebase)
- AO Pause Trigger (ao/PauseTrigger)
- Counter input signals for all counters—Source, Gate, Aux, HW_Arm, A, B, Z
- Counter n Sample Clock
- DI Sample Clock (di/SampleClock)
- DI Sample Clock Timebase (di/SampleClockTimebase)
- DI Reference Trigger (di/ReferenceTrigger)
- DO Sample Clock (do/SampleClock)

Most functions allow you to configure the polarity of PFI inputs and whether the input is edge or level sensitive.

Exporting Timing Output Signals Using PFI Terminals

You can route any of the following timing signals to any PFI terminal configured as an output:

- AI Convert Clock* (ai/ConvertClock)
- AI Hold Complete Event (ai/HoldCompleteEvent)
- AI Reference Trigger (ai/ReferenceTrigger)
- AI Sample Clock (ai/SampleClock)
- AI Start Trigger (ai/StartTrigger)
- AI Pause Trigger (ai/PauseTrigger)
- AO Sample Clock* (ao/SampleClock)

- AO Start Trigger (ao/StartTrigger)
- AO Pause Trigger (ao/PauseTrigger)
- DI Sample Clock (di/SampleClock)
- DI Start Trigger (di/StartTrigger)
- DI Reference Trigger (di/ReferenceTrigger)
- DI Pause Trigger (di/PauseTrigger)
- DO Sample Clock* (do/SampleClock)
- DO Start Trigger (do/StartTrigger)
- DO Pause Trigger (do/PauseTrigger)
- Counter n Source
- Counter n Gate
- Counter n Internal Output
- Counter n Sample Clock
- Counter n Counter n HW Arm
- Frequency Output
- PXI_STAR
- RTSI <0..7>
- Analog Comparison Event
- Change Detection Event
- Watchdog timer expired pulse



Note Signals with an * are inverted before being driven to a terminal; that is, these signals are active low.

Using PFI Terminals as Static Digital I/Os

Each PFI can be individually configured as a static digital input or a static digital output. When a terminal is used as a static digital input or output, it is called P1.x or P2.x. On the I/O connector, each terminal is labeled PFI x/P1.x or PFI x/P2.x.

In addition, PCIe-6353 and PXIe-6353 devices have up to 32 lines of bidirectional DIO signals.

Using PFI Terminals to Digital Detection Events

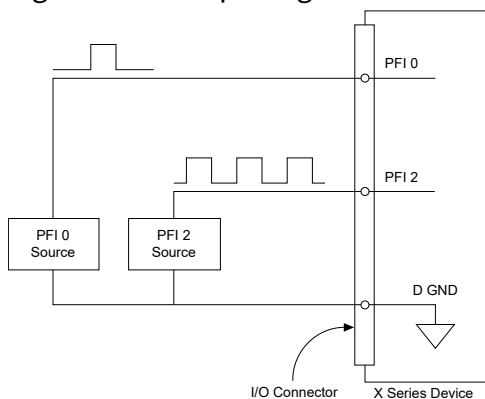
Each PFI can be configured to detect digital changes. The values on the PFI lines cannot be read in a hardware-timed task, but they can be used to fire the change

detection event. For example, if you wanted to do change detection on eight timed DIO lines but wanted to ensure that the value of the lines was updated every second independent of the eight lines changing you could set a PFI line up for change detection and connect a 1 Hz signal to it.

Connecting PFI Input Signals

All PFI input connections are referenced to D GND. The following figure shows this reference, and how to connect an external PFI 0 source and an external PFI 2 source to two PFI terminals.

Figure 83. PFI Input Signal Connections



I/O Protection

Each DIO and PFI signal is protected against overvoltage, undervoltage, and overcurrent conditions as well as ESD events. However, you should avoid these fault conditions by following these guidelines:

- If you configure a PFI or DIO line as an output, do not connect it to any external signal source, ground, or power supply.
- If you configure a PFI or DIO line as an output, understand the current requirements of the load connected to these signals. Do not exceed the specified current output limits of the DAQ device. NI has several signal conditioning solutions for digital applications requiring high current drive.
- If you configure a PFI or DIO line as an input, do not drive the line with voltages outside of its normal operating range. The PFI or DIO lines have a smaller operating range than the AI signals.
- Treat the DAQ device as you would treat any static sensitive device. Always

properly ground yourself and the equipment when handling the DAQ device or connecting to it.

Programmable Power-Up States

At system startup and reset, the hardware sets all PFI and DIO lines to high-impedance inputs by default. The DAQ device does not drive the signal high or low. Each line has a weak pull-down resistor connected to it, as described in the device specifications.

NI-DAQmx supports programmable power-up states for PFI and DIO lines. Software can program any value at power up to the P0, P1, or P2 lines. The PFI and DIO lines can be set as:

- A high-impedance input with a weak pull-down resistor (default)
- An output driving a 0
- An output driving a 1

Refer to the NI-DAQmx Help or the LabVIEW Help for more information about setting power-up states in NI-DAQmx or MAX.



Note When using an PCIe-6353 and PXIe-6353 device to control an SCXI chassis, DIO lines 0, 1, 2, and 4 are used as communication lines and must be left to power-up in the default high-impedance state to avoid potential damage to these signals.

Related information:

- [DAQmx Set Power Up States \(VI\)](#)
- [DAQmx Get Power Up States \(VI\)](#)

PFI Filters

You can enable a programmable debouncing filter on each PFI <0..15>, RTSI <0..7>, PXI_STAR, or PXIe_DSTARC signal. When the filters are enabled, your device samples the input on each rising edge of a filter clock. X Series devices use an onboard oscillator to generate the filter clock.



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

The following is an example of low to high transitions of the input signal. High-to-low transitions work similarly.

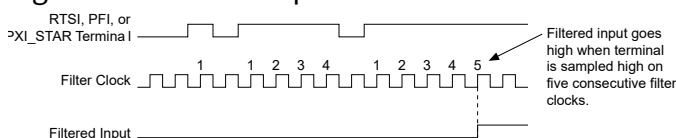
Assume that an input terminal has been low for a long time. The input terminal then changes from low to high, but glitches several times. When the filter clock has sampled the signal high on N consecutive edges, the low to high transition is propagated to the rest of the circuit. The value of N depends on the filter setting as illustrated in the following table.

Table 14. Filters

Filter Setting	Filter Clock	N (Filter Clocks Needed to Pass Signal)	Pulse Width Guaranteed to Pass Filter	Pulse Width Guaranteed to Not Pass Filter
None	—	—	—	—
90 ns(short)	100 MHz	9	90 ns	80 ns
5.12 μ s (medium)	100 MHz	512	5.12 μ s	5.11 μ s
2.56 ms (high)	100 kHz	256	2.56 ms	2.55 ms
Custom	User configurable	N	$N/\text{timebase}$	$(N - 1)/\text{timebase}$

The filter setting for each input can be configured independently. On power up, the filters are disabled. The following figure shows an example of a low to high transition on an input that has a custom filter set to $N = 5$.

Figure 84. Filter Example



Enabling filters introduces jitter on the input signal. The maximum jitter is one period of the timebase.

When a RTSI input is routed directly to PFI, the X Series device does not use the filtered version of the input signal.

Digital Routing and Clock Generation for PCIe-6353 and PXIe-6353 Devices

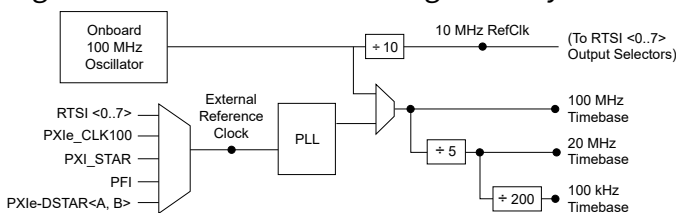
The digital routing circuitry has the following main functions:

- Manages the flow of data between the bus interface and the acquisition/generation sub-systems (analog input, analog output, digital I/O, and the counters). The digital routing circuitry uses FIFOs (if present) in each sub-system to ensure efficient data movement.
- Routes timing and control signals. The acquisition/generation sub-systems use these signals to manage acquisitions and generations. These signals can come from the following sources:
 - PCIe-6353 and PXIe-6353 devices
 - Other devices in your system through RTSI
 - User input through the PFI terminals
 - User input through the PXI_STAR terminal
- Routes and generates the main clock signals for the PCIe-6353 and PXIe-6353 device.

Clock Routing

The following figure shows the clock routing circuitry of an X Series device.

Figure 85. X Series Clock Routing Circuitry



Note RTSI signals apply only to PCI or PCIe devices connected with RTSI cables. PXI_ and PXIe_ signals apply only to devices connected to a PXIe backplane.

100 MHz Timebase

The 100 MHz Timebase can be used as the timebase for all internal subsystems. The 100 MHz Timebase is generated from the following sources:

- Onboard oscillator
- External signal (by using the external reference clock)

20 MHz Timebase

The 20 MHz Timebase can be used to generate many of the AI and AO timing signals. The 20 MHz Timebase can also be used as the Source input to the 32-bit general-purpose counter/timers.

The 20 MHz Timebase is generated by dividing down the 100 MHz Timebase.

100 kHz Timebase

The 100 kHz Timebase can be used to generate many of the AI and AO timing signals. The 100 kHz Timebase can also be used as the Source input to the 32-bit general-purpose counter/timers.

The 100 kHz Timebase is generated by dividing down the 20 MHz Timebase by 200.

External Reference Clock

The external reference clock can be used as a source for the internal timebases (100 MHz Timebase, 20 MHz Timebase, and 100 kHz Timebase) on an X Series device. By using the external reference clock, you can synchronize the internal timebases to an external clock.

The following signals can be routed to drive the external reference clock, if supported by your device:

- PXIe_CLK100
- PXI_STAR
- PXIe_DSTAR<A,B>
- PFI <0..15>

- RTSI <0..7>

The external reference clock is an input to a Phase-Lock Loop (PLL). The PLL generates the internal timebases.



Notice

Do not disconnect an external reference clock once the devices have been synchronized or are used by a task. Doing so may cause the device to go into an unknown state. Make sure that all tasks using a reference clock are stopped before disconnecting it.

Enabling or disabling the PLL through the use of a reference clock affects the clock distribution to all subsystems. For this reason, the PLL can only be enabled or disabled when no other tasks are running in any of the device subsystems.

10 MHz Reference Clock

The 10 MHz reference clock can be used to synchronize other devices to your X Series device. The 10 MHz reference clock can be routed to the RTSI <0..7> or PFI <0..15> terminals. Other devices connected to the RTSI bus can use this signal as a clock input.

The 10 MHz reference clock is generated by dividing down the onboard oscillator.

PXIe Device Synchronization

On PXI Express systems, you can synchronize devices to PXIe_CLK100. In this application, the PXI Express chassis acts as the initiator. Each PXI Express module routes PXIe_CLK100 to its external reference clock.

Another option in PXI Express systems is to use PXI_STAR. The Star Trigger controller device acts as the initiator and drives PXI_STAR with a clock signal. Each target device routes PXI_STAR to its external reference clock.

PCIe Device Synchronization

With the RTSI and PFI buses and the routing capabilities of devices, there are several ways to synchronize multiple devices depending on your application.

To synchronize multiple devices to a common timebase, choose one device—the initiator—to generate the timebase. The initiator device routes its 10 MHz reference clock to one of the RTSI <0..7> or PFI <0..15> signals. All devices (including the initiator device) receive the 10 MHz reference clock from RTSI or PFI. This signal becomes the external reference clock. A PLL on each device generates the internal timebases synchronous to the external reference clock.

Once all of the devices are using or referencing a common timebase, you can synchronize operations across them by sending a common start trigger out across the RTSI or PFI bus and setting their sample clock rates to the same value.

Real-Time System Integration (RTSI)

Real-Time System Integration (RTSI) is a signal bus among devices that allows you to share trigger signals between devices or use a common clock (or timebase) to drive the timing engine on multiple devices.

Many National Instruments DAQ, motion, vision, and CAN devices support RTSI.

In a PCI Express system, the RTSI bus consists of the RTSI bus interface and a ribbon cable. The bus can route timing and trigger signals between several functions on as many as five DAQ, vision, motion, or CAN devices in the computer.

In a PXI Express system, the RTSI bus is replaced by the PXI and PXI Express trigger signals on the PXI Express backplane. This bus can route timing and trigger signals between several functions on as many as seven DAQ devices in the system.

USB devices do not support the RTSI bus.

Using RTSI as Outputs

RTSI <0..7> are bidirectional terminals. As an output, you can drive any of the following

signals to any RTSI terminal:

- AI Start Trigger (ai/StartTrigger)
- AI Reference Trigger (ai/ReferenceTrigger)
- AI Convert Clock* (ai/ConvertClock)
- AI Sample Clock (ai/SampleClock)
- AI Pause Trigger (ai/PauseTrigger)
- AO Sample Clock* (ao/SampleClock)
- AO Start Trigger (ao/StartTrigger)
- AO Pause Trigger (ao/PauseTrigger)
- DI Start Trigger (di/StartTrigger)
- DI Sample Clock (di/SampleClock)
- DI Pause Trigger (di/PauseTrigger)
- DI Reference Trigger (di/ReferenceTrigger)
- DO Start Trigger (do/StartTrigger)
- DO Sample Clock* (do/SampleClock)
- DO Pause Trigger (do/PauseTrigger)
- 10 MHz Reference Clock
- Counter n Source, Gate, Z, Internal Output
- Change Detection Event
- Analog Comparison Event
- FREQ OUT
- PFI <0..5>



Note Signals with a * are inverted before being driven on the RTSI terminals.

Using RTSI Terminals as Timing Input Signals

You can use RTSI terminals to route external timing signals to many different X Series functions. Each RTSI terminal can be routed to any of the following signals:

- AI Convert Clock (ai/ConvertClock)
- AI Sample Clock (ai/SampleClock)
- AI Start Trigger (ai/StartTrigger)
- AI Reference Trigger (ai/ReferenceTrigger)
- AI Pause Trigger (ai/PauseTrigger)
- AI Sample Clock Timebase (ai/SampleClockTimebase)

- AO Start Trigger (ao/StartTrigger)
- AO Sample Clock (ao/SampleClock)
- AO Sample Clock Timebase (ao/SampleClockTimebase)
- AO Pause Trigger (ao/PauseTrigger)
- Counter input signals for all counters—Source, Gate, Aux, HW_Arm, A, B, or Z
- DI Sample Clock (di/SampleClock)
- DI Start Trigger (di/StartTrigger)
- DI Pause Trigger (di/PauseTrigger)
- DI Reference Trigger (di/ReferenceTrigger)
- DO Sample Clock (do/SampleClock)
- DO Sample Clock Timebase (do/SampleClockTimebase)

Most functions allow you to configure the polarity of RTSI inputs and whether the input is edge or level sensitive.

RTSI Filters

You can enable a programmable debouncing filter on each RTSI signal. Refer to the PFI Filters section for more information.

When a RTSI input is routed directly to PFI, the PCIe-6353 and PXIe-6353 device does not use the filtered version of the input signal.

Related concepts:

- [PFI Filters](#)

PXI and PXI Express Clock and Trigger Signals

PXI and PXI Express clock and trigger signals are only available on PXI Express devices.

PXIe_CLK100

PXIe_CLK100 is a common low-skew 100 MHz reference clock for synchronization of multiple modules in a PXI Express measurement or control system. The PXIe backplane is responsible for generating PXIe_CLK100 independently to each peripheral slot in a PXI Express chassis. For more information, refer to the PXI Express Specification at

www.pxisa.org.

PXIe_SYNC100

PXIe_SYNC100 is a common low-skew 10 MHz reference clock with a 10% duty cycle for synchronization of multiple modules in a PXI Express measurement or control system. This signal is used to accurately synchronize modules using PXIe_CLK100 along with those using PXI_CLK10. The PXI Express backplane is responsible for generating PXIe_SYNC100 independently to each peripheral slot in a PXI Express chassis. For more information, refer to the PXI Express Specification at www.pxisa.org.

PXI_CLK10

PXI_CLK10 is a common low-skew 10 MHz reference clock for synchronization of multiple modules in a PXI measurement or control system. The PXI backplane is responsible for generating PXI_CLK10 independently to each peripheral slot in a PXI chassis.



Note PXI_CLK10 cannot be used as a reference clock for X Series devices.

PXI Triggers

A PXI chassis provides eight bused trigger lines to each module in a system. Triggers may be passed from one module to another, allowing precisely timed responses to asynchronous external events that are being monitored or controlled. Triggers can be used to synchronize the operation of several different PXI peripheral modules.

On X Series devices, the eight PXI trigger signals are synonymous with RTSI <0..7>.

Note that in a PXI chassis with more than eight slots, the PXI trigger lines may be divided into multiple independent buses. Refer to the documentation for your chassis for details.

PXI_STAR Trigger

In a PXI Express system, the Star Trigger bus implements a dedicated trigger line between the system timing slot and the other peripheral slots. The Star Trigger can be

used to synchronize multiple devices or to share a common trigger signal among devices.

A Star Trigger controller can be installed in this system timing slot to provide trigger signals to other peripheral modules. Systems that do not require this functionality can install any standard peripheral module in this system timing slot.

An X Series device receives the Star Trigger signal (PXI_STAR) from a Star Trigger controller. PXI_STAR can be used as an external source for many AI, AO, and counter signals. An X Series device is not a Star Trigger controller.

An X Series device can be used in the system timing slot of a PXI system, but the system will not be able to use the Star Trigger feature.

PXIe_DSTAR<A..C>

PXI Express devices can provide high-quality and high-frequency point-to-point connections between each slot and a system timing slot. These connections come in the form of three low-voltage differential star triggers that create point-to-point, high-frequency connections between a PXI Express system timing module and a peripheral device. Using multiple connections enable you to create more applications because of the increased routing capabilities. The following table describes the three differential star (DSTAR) lines and how they are used.

Table 15. PXIe-DSTAR line descriptions

Trigger Line	Purpose
PXIe_DSTARA	Distributes high-speed, high-quality clock signals from the system timing slot to the peripherals (input).
PXIe_DSTARB	Distributes high-speed, high-quality trigger signals from the system timing slot to the peripherals (input).
PXIe_DSTARC	Sends high-speed, high-quality trigger or clock signals from the peripherals to the system timing slot (output).

The DSTAR lines are only available for PXI Express devices when used with a PXI

Express system timing module. For more information, refer to the PXI Express Specification at www.pxisa.org.

PXI_STAR and PXIe_DSTAR Filters

You can enable a programmable debouncing filter on each PXIe_DSTAR or PXI_STAR signal. Refer to the PFI Filters section for more information.

Related concepts:

- [PFI Filters](#)

Data Transfer Methods

Direct Memory Access

Direct Memory Access (DMA) is the default method of data transfer for PCI Express and PXI Express devices. DMA is a method to transfer data between the device and computer memory without the involvement of the CPU. This method makes DMA the fastest available data transfer method. NI uses DMA hardware and software technology to achieve high throughput rates and increase system utilization.

NI PCI Express and PXI Express X Series devices have eight fully-independent DMA controllers for high-performance transfers of data blocks. One DMA controller is available for each measurement and acquisition block:

- Analog input
- Analog output
- Counter 0
- Counter 1
- Counter 2
- Counter 3
- Digital waveform generation (digital output)
- Digital waveform acquisition (digital input)

Each DMA controller channel contains a FIFO and independent processes for filling and emptying the FIFO. This allows the buses involved in the transfer to operate independently for maximum performance. Data is transferred simultaneously between the ports. The DMA controller supports burst transfers to and from the FIFO.

Each DMA controller supports several features to optimize PCI Express/PXI Express bus utilization. The DMA controllers pack and unpack data through the FIFOs. This feature allows the DMA controllers to combine multiple 16-bit transfers to the DAQ circuitry into a single 32-bit burst transfer on PCI Express. The DMA controllers also automatically handle unaligned memory buffers on PCI Express/PXI Express.

Programmed I/O

Programmed I/O is a data transfer mechanism where the user's program is responsible for transferring data. Each read or write call in the program initiates the transfer of data. Programmed I/O is typically used in software-timed (on-demand) operations. Refer to the Analog Output Data Generation Methods section for more information.

Related concepts:

- [Real-Time System Integration \(RTSI\)](#)

Related reference:

- [Analog and Digital Output Data Generation Methods](#)

PXI Express Considerations

PXI Express X Series devices can be installed in any PXI Express slot in PXI Express chassis.

PXIe DAQ Bandwidth Considerations

PXI clock and trigger signals are only available on PXI Express devices.

PXI Express specifications are developed by the PXI System Alliance (www.pxisa.org).

In order to continuously transfer large amounts of data, the entire PXI Express system must be designed with sufficient data bandwidth.

Depending on the PXI Express connection to the PXI Express chassis backplane, the bandwidth bottleneck could be due to one of the following: the DAQ task, the backplane connection, a PCIe switch integrated into the PXI Express backplane, or between the connection of a PXI Express remote controller to a host machine (if using MXI).

Different PXI Express chassis have different architectures and per slot bandwidths.

If you are using a high channel count or high-speed MIO/SMIO DAQ device, pay special attention to these factors since buffer overflow or underflow errors can occur as you approach or pass the maximum theoretical system bandwidth.

Related concepts:

- [PXIe Device Synchronization](#)
- [PXI and PXI Express Clock and Trigger Signals](#)

Triggering into PCIe-6353 and PXIe-6353 Devices

A trigger is a signal that causes an action, such as starting or stopping the acquisition of data. When you configure a trigger, you must decide how you want to produce the trigger and the action you want the trigger to cause. The PCIe-6353 and PXIe-6353 device supports internal software triggering, external digital triggering, and analog triggering.



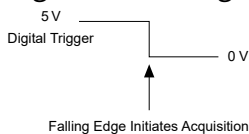
Note For more information about triggering compatibility, refer to the device specifications.

Triggering with a Digital Source

Your DAQ device can generate a trigger on a digital signal. You must specify a source and an edge. The digital source can be any of the PFI, RTSI, or PXI_STAR signals.

The edge can be either the rising edge or falling edge of the digital signal. A rising edge is a transition from a low logic level to a high logic level. A falling edge is a high-to-low transition. The following figure shows a falling-edge trigger.

Figure 86. Falling-Edge Trigger



You can also program your DAQ device to perform an action in response to a trigger from a digital source. The action can affect the following:

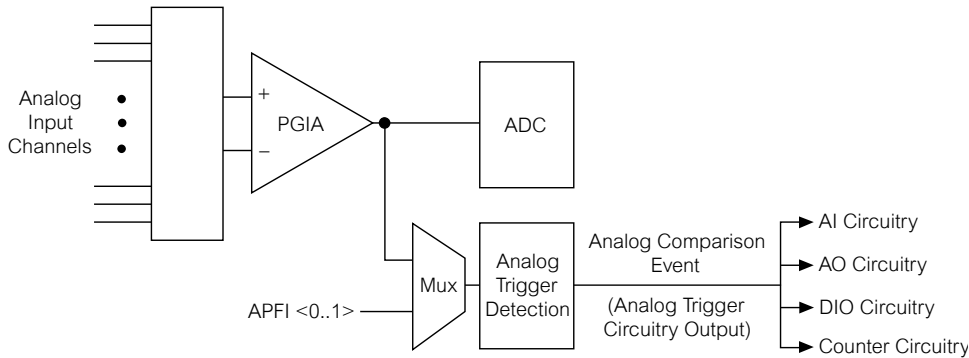
- Analog input acquisition
- Analog output generation
- Counter behavior
- Digital waveform acquisition and generation

Triggering with an Analog Source

The PCIe-6353 and PXle-6353 device can generate a trigger on an analog signal. To find your device triggering options, refer to the device specifications.

The following figure shows the analog trigger circuit on the PCIe-6353 and PXle-6353 devices.

Figure 87. PCIe-6353 and PXle-6353 Device Analog Trigger Circuitry



You must specify a source and an analog trigger type. The source can be either an APFI <0,1> terminal or an analog input channel.

APFI <0,1> Terminals

When you use either APFI <0,1> terminal as an analog trigger, you should drive the terminal with a low impedance signal source (less than 1 k Ω source impedance). If APFI <0,1> are left unconnected, they are susceptible to crosstalk from adjacent terminals, which can cause false triggering. Note that the APFI <0,1> terminals can also be used for other functions such as the AO External Reference input.

PCIe-6353 and PXle-6353 Analog Input Channels

Select any analog input channel to drive the NI-PGIA. The NI-PGIA amplifies the signal as determined by the input ground-reference setting and the input range. The output of the NI-PGIA then drives the analog trigger detection circuit. By using the NI-PGIA, you can trigger on very small voltage changes in the input signal.

When the PCIe-6353 and PXle-6353 device is waiting for an analog trigger with a AI channel as the source, the AI muxes should not route different AI channels to the NI-

PGIA. If a different channel is routed to the NI-PGIA, the trigger condition on the desired channel could be missed. The other channels could also generate false triggers.

This behavior places some restrictions on using AI channels as trigger sources. When you use an analog start trigger, the trigger channel must be the first channel in the channel list. When you use an analog reference or pause trigger, and the analog channel is the source of the trigger, there can be only one channel in the channel list.

Analog Trigger Actions

The output of the analog trigger detection circuit is the Analog Comparison Event signal. You can program your PCIe-6353 and PXIe-6353 device to perform an action in response to the Analog Comparison Event signal.

The action can affect the following:

- Analog input acquisition
- Analog output generation
- Digital input behavior
- Digital output behavior
- Counter behavior

Routing Analog Comparison Event to an Output Terminal

You can route Analog Comparison Event out to any PFI <0..15> or RTSI <0..7> terminal.

Analog Trigger Types

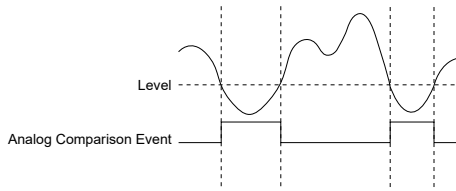
You can configure the analog trigger circuitry to different triggering modes.

Analog Edge Triggering

Configure the analog trigger circuitry to detect when the analog signal is below or above a level you specify.

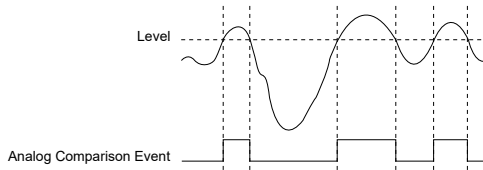
The following figure illustrates below-level analog triggering mode, where the trigger is generated when the signal value is less than Level.

Figure 88. Below-Level Analog Triggering Mode



The following figure illustrates above-level analog triggering mode, where the trigger is generated when the signal value is greater than Level.

Figure 89. Above-Level Analog Triggering Mode



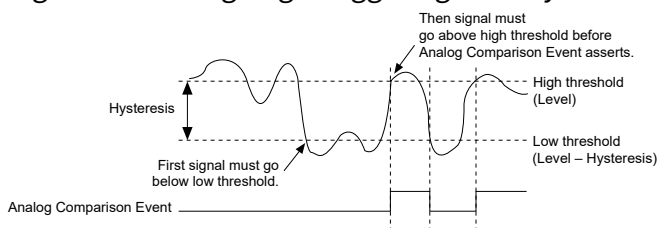
Analog Edge Triggering with Hysteresis

Hysteresis adds a programmable voltage region above or below the trigger level that an input signal must pass through before the DAQ device recognizes a trigger condition, and is often used to reduce false triggering due to noise or jitter in the signal.

When using hysteresis with a rising slope, you specify a trigger level and amount of hysteresis. The high threshold is the trigger level; the low threshold is the trigger level minus the hysteresis.

For the trigger to assert, the signal must first be below the low threshold, then go above the high threshold. The trigger stays asserted until the signal returns below the low threshold. The output of the trigger detection circuitry is the internal Analog Comparison Event signal, as shown in the following figure.

Figure 90. Analog Edge Triggering with Hysteresis Rising Slope Example

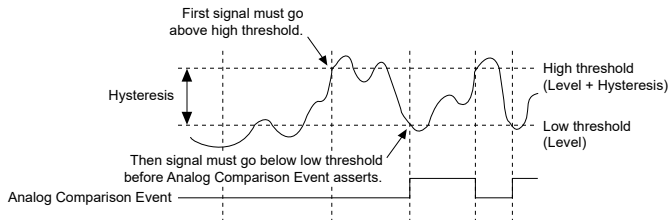


When using hysteresis with a falling slope, you specify a trigger level and amount of

hysteresis. The low threshold is the trigger level; the high threshold is the trigger level plus the hysteresis.

For the trigger to assert, the signal must first be above the high threshold, then go below the low threshold. The trigger stays asserted until the signal returns above the high threshold. The output of the trigger detection circuitry is the internal Analog Comparison Event signal, as shown in the following Figure.

Figure 91. Analog Edge Triggering with Hysteresis Falling Slope Example

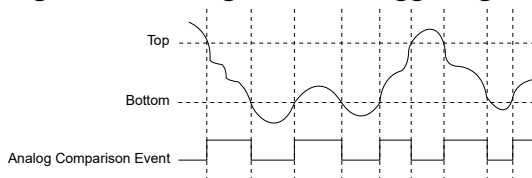


Analog Window Triggering

An analog window trigger occurs when an analog signal either passes into (enters) or passes out of (leaves) a window defined by two voltage levels. Specify the levels by setting the window Top value and the window Bottom value.

The following figure demonstrates a trigger that asserts when the signal enters the window.

Figure 92. Analog Window Triggering Mode (Entering Window)



Analog Trigger Accuracy

The analog trigger circuitry compares the voltage of the trigger source to the output of programmable trigger DACs. When you configure the level (or the high and low limits in window trigger mode), the device adjusts the output of the trigger DACs. Refer to the device specifications to find the accuracy or resolution of these DACs, which also shows the accuracy or resolution of analog triggers.

To improve accuracy, do the following:

- Use an AI channel (with a small input range) instead of APFI <0,1> as your trigger source. The DAQ device does not amplify the APFI <0, 1> signals. When using an AI channel, the NI-PGIA amplifies the AI channel signal before driving the analog trigger circuitry. If you configure the AI channel to have a small input range, you can trigger on very small voltage changes in the input signal.
- Software-calibrate the analog trigger circuitry. The propagation delay from when a valid trigger condition is met to when the analog trigger circuitry emits the Analog Comparison Event may have an impact on your measurements if the trigger signal has a high slew rate. If you find these conditions have a noticeable impact on your measurements, you can perform software calibration on the analog trigger circuitry by configuring your task as normal and applying a known signal for your analog trigger. Comparing the observed results against the expected results, you can calculate the necessary offsets to apply in software to fine-tune the desired triggering behavior.

Unpacking the Kit

Take precautions to prevent electrostatic discharge when unpacking and inspecting your hardware.



Notice To prevent electrostatic discharge (ESD) from damaging the device, ground yourself using a grounding strap or by holding a grounded object, such as your computer chassis.

1. Touch the antistatic package to a metal part of the computer chassis.
2. Remove the device from the package and inspect the device for loose components or any other sign of damage.



Notice Never touch the exposed pins of connectors.



Note Do not install a device if it appears damaged in any way.

3. Unpack any other items and documentation from the kit.



Note Store the device in the antistatic package when the device is not in use.

Safety Guidelines



Caution Observe all instructions and cautions in the user documentation. Using the product in a manner not specified can damage the product and compromise the built-in safety protection.



Attention Suivez toutes les instructions et respectez toutes les mises en garde de la documentation d'utilisation. L'utilisation du produit de toute autre façon que celle spécifiée risque de l'endommager et de compromettre la protection de sécurité intégrée.



Caution The product is designed for nonhazardous, live signals. You must ensure that all signals connected to the product are isolated from hazardous, live circuits and no unsafe voltages are present at the inputs. Voltages that exceed the specifications could result in damage to the product or electric shock.



Attention Le produit est conçu pour les signaux en direct non dangereux. Vous devez vous assurer que tous les signaux connectés au produit sont isolés des circuits dangereux sous tension et qu'aucune tension dangereuse n'est présente aux entrées. Des tensions supérieures à celles mentionnées dans les spécifications peuvent endommager le produit ou provoquer un choc électrique.

Related information:

- [Product Certifications](#)

PCIe-6353 and PXIe-6353 Pinout

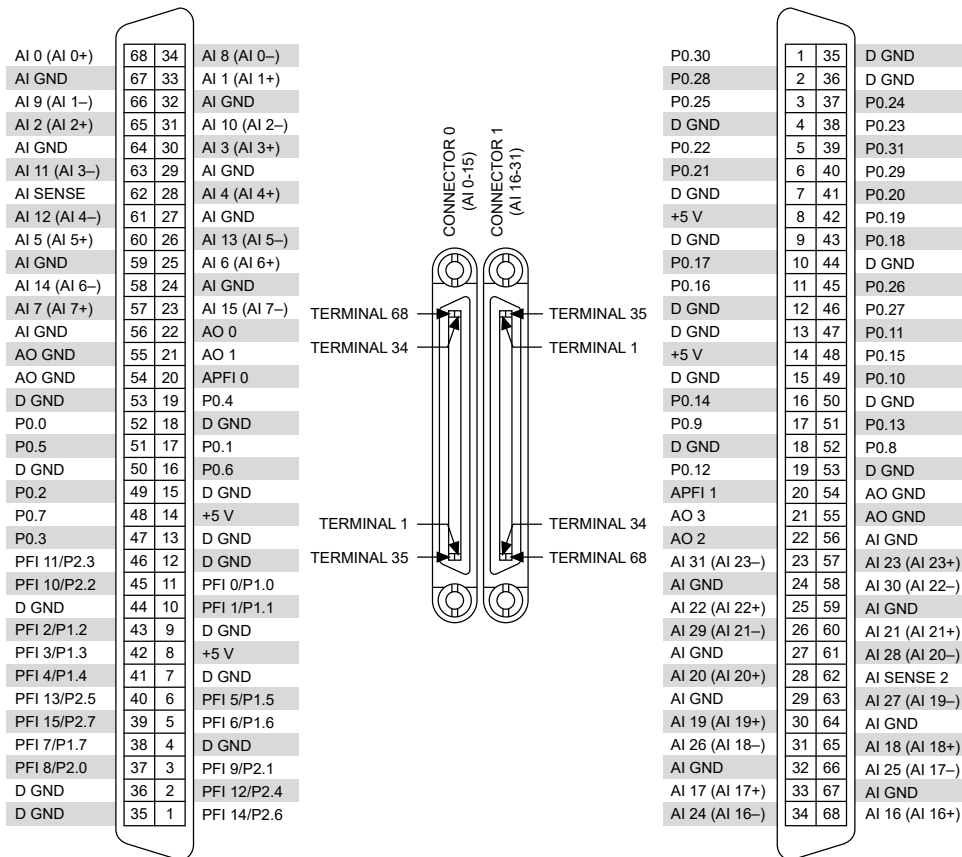


Table 16. Default Counter/Timer Terminals

Counter/Timer Signal	Default PFI Terminal
CTR 0 SRC	PFI 8
CTR 0 GATE	PFI 9
CTR 0 AUX	PFI 10
CTR 0 OUT	PFI 12
CTR 0 A	PFI 8
CTR 0 Z	PFI 9
CTR 0 B	PFI 10
CTR 1 SRC	PFI 3

Counter/Timer Signal	Default PFI Terminal
CTR 1 GATE	PFI 4
CTR 1 AUX	PFI 11
CTR 1 OUT	PFI 13
CTR 1 A	PFI 3
CTR 1 Z	PFI 4
CTR 1 B	PFI 11
CTR 2 SRC	PFI 0
CTR 2 GATE	PFI 1
CTR 2 AUX	PFI 2
CTR 2 OUT	PFI 14
CTR 2 A	PFI 0
CTR 2 Z	PFI 1
CTR 2 B	PFI 2
CTR 3 SRC	PFI 5
CTR 3 GATE	PFI 6
CTR 3 AUX	PFI 7
CTR 3 OUT	PFI 15
CTR 3 A	PFI 5
CTR 3 Z	PFI 6
CTR 3 B	PFI 7
FREQ OUT	PFI 14

Table 17. Signal Descriptions

Signal	Reference	Description
AI GND	—	Analog Input Ground—These terminals are the reference point for single-ended AI

Signal	Reference	Description
		measurements in RSE mode and the bias current return point for DIFF measurements. All ground references—AI GND, AO GND, and D GND—are connected on the device. Though AI GND, AO GND, and D GND are connected on the device, they are connected by small traces to reduce crosstalk between subsystems. Each ground has a slight difference in potential.
AI <0..31>	Varies	Analog Input Channels—For single-ended measurements, each signal is an analog input voltage channel. In RSE mode, AI GND is the reference for these signals. In NRSE mode, the reference for each AI signal is an AI SENSE. For differential measurements, AI 0 and AI 8 are the positive and negative inputs of differential analog input channel 0. Similarly, the following signal pairs also form differential input channels: AI <1,9>, AI <2,10>, and so on.
AI SENSE, AI SENSE 2	—	Analog Input Sense—In NRSE mode, the reference for each AI <0..15> signal is AI SENSE; the reference for each AI <16..31> signal is AI SENSE 2.

Signal	Reference	Description
AO <0..3>	AO GND	Analog Output Channels—These terminals supply voltage output.
AO GND	—	Analog Output Ground—AO GND is the reference for AO. All ground references—AI GND, AO GND, and D GND—are connected on the device. Though AI GND, AO GND, and D GND are connected on the device, they are connected by small traces to reduce crosstalk between subsystems. Each ground has a slight difference in potential.
D GND	—	Digital Ground—D GND supplies the reference for port 0, port 1, port 2 digital channels, PFI, and +5 V. All ground references—AI GND, AO GND, and D GND—are connected on the device. Though AI GND, AO GND, and D GND are connected on the device, they are connected by small traces to reduce crosstalk between subsystems. Each ground has a slight difference in potential.
P0.<0..31>	D GND	Port 0 Digital I/O Channels—You can configure each signal individually as an input or output.

Signal	Reference	Description
APFI <0,1>	AO GND or AI GND	Analog Programmable Function Interface Channels—Each APFI signal can be used as AO external reference inputs for AO, or as an analog trigger input. APFI are referenced to AI GND when they are used as analog trigger inputs. APFI are referenced to AO GND when they are used as AO external offset or reference inputs.
+5 V	D GND	+5 V Power Source—These terminals provide a fused +5 V power source.
PFI <0..7>/P1.<0..7>, PFI <8..15>/P2.<0..7>	D GND	Programmable Function Interface or Digital I/O Channels—Each of these terminals can be individually configured as a PFI terminal or a digital I/O terminal. As an input, each PFI terminal can be used to supply an external source for AI, AO, DI, and DO timing signals or counter/timer inputs. As a PFI output, you can route many different internal AI, AO, DI, or DO timing signals to each PFI terminal. You can also route the counter/timer outputs to each PFI terminal. As a port 1 or port 2 digital I/O signal, you can individually configure each signal as an input or output.

Related reference:

- [Connecting to Counter Signals](#)

Installing Software and Hardware

Before installing your DAQ device, you must install the software you plan to use with the device.

1. **Installing application software:** Refer to the installation instructions that accompany your software.
2. **Installing NI-DAQmx:** The getting started guides, packaged with NI-DAQmx and also available on ni.com/manuals, contain step-by-step instructions for installing software and hardware, configuring channels and tasks, and getting started developing an application.
3. **Installing the hardware:** Unpack your X Series device as described in the *Unpacking* section.

Related concepts:

- [Cables and Accessories](#)

Installing the Software

You must be an Administrator to install NI software on your computer.

1. Install an ADE, such as LabVIEW or LabWindows™/CVI™.
2. Download the driver software installer from ni.com/downloads.
Package Manager downloads with the driver software to handle the installation. Refer to the Package Manager Manual for more information about installing, removing, and upgrading NI software using Package Manager.
3. Follow the instructions in the installation prompts.



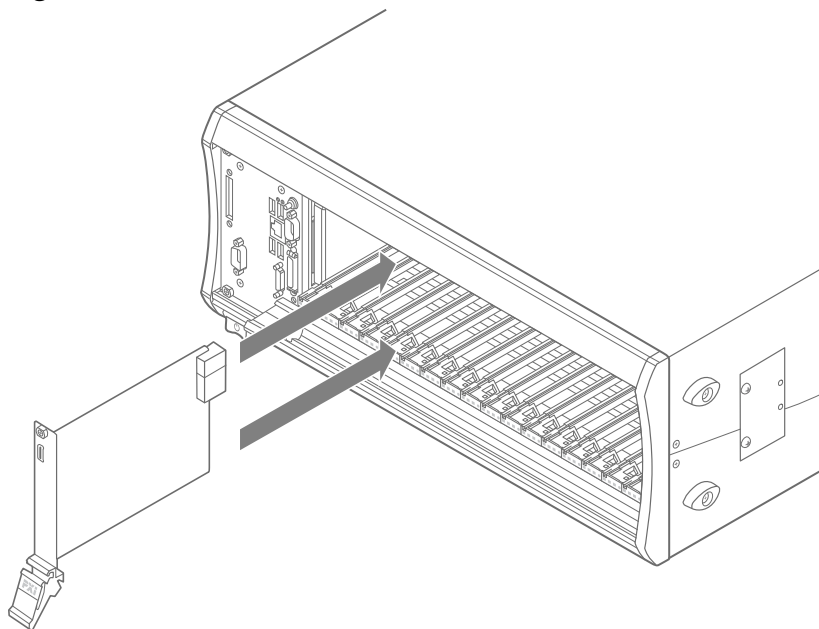
Note Windows users may see access and security messages during installation. Accept the prompts to complete the installation.

4. When the installer completes, select **Restart** in the dialog box that prompts you to restart, shut down, or restart later.

Installing the PXIe Device into a Chassis

1. If you are using an MXI-Express connected chassis, turn off your computer.
2. Power off the chassis.
3. Ensure the AC power source is connected to the chassis before installing the module.
The AC power cord grounds the chassis and protects it from electrical damage while you install the module.
4. Inspect the slot pins on the chassis backplane for any bends or damage prior to installation. Do not install a module if the backplane is damaged.
5. Position the chassis so that inlet and outlet vents are not obstructed.
For more information about optimal chassis positioning, refer to the chassis documentation.
6. Remove the black plastic covers from all the captive screws on the module front panel.
7. Identify a supported slot in the chassis. The PCIe-6353 and PXIe-6353 module can be placed in PXI Express hybrid peripheral slots (●^H), PXI Express system timing slots (●), or PXI Express peripheral slots (●).
8. Remove the filler panel of an unused PXI/PXI Express slot.
9. Touch any metal part of the chassis to discharge static electricity.
10. Ensure that the ejector handle is in the downward (unlatched) position.

Figure 93. Module Installation



11. Place the module edges into the module guides at the top and bottom of the

chassis. Slide the module into the slot until it is fully inserted.

12. Latch the module in place by pulling up on the ejector handle.
13. Secure the module front panel to the chassis using the front-panel mounting screws.



Note Tightening the top and bottom mounting screws increases mechanical stability and also electrically connects the front panel to the chassis, which can improve the signal quality and electromagnetic performance.

14. Power on the chassis.
15. If you are using an MXI-Express connected chassis, turn on your computer.

Related concepts:

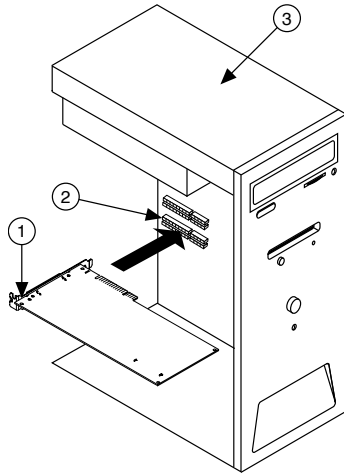
- [Cables and Accessories](#)

Installing the PCIe Device

1. Power off and unplug the computer.
2. Access the computer system expansion slots. This step might require you to remove one or more access panels on the computer case.
3. Locate a compatible slot and remove the corresponding slot cover on the computer back panel.
4. Touch any metal part of the computer to discharge any static electricity.
5. Insert the device into the applicable PCI/PCI Express system slot. Gently rock the device into place. Do not force the device into place.

Per the PCI standard, NI PCI DAQ devices with the Universal PCI connector are supported in PCI-compliant buses, including PCI-X. You cannot install PCI Express devices in PCI slots and vice versa. PCI Express devices support up-plugging into a PCI Express slot of higher lane width.

Figure 94. Installing a PCI/PCI Express Device



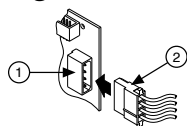
1. PCI/PCI Express DAQ Device
 2. PCI/PCI Express System Slot
 3. PC with PCI/PCI Express Slot
6. Secure the module mounting bracket to the computer back panel rail.



Note Tightening the top and bottom mounting screws increases mechanical stability and also electrically connects the front panel to the chassis, which can improve the signal quality and electromagnetic performance.

7. On PCI Express devices, such as the PCIe-625x/63xx, connect the PC and the device disk drive power connectors. Refer to the device user manual for information about when to use the disk drive power connector. Use a disk drive power connector that is not in the same power chain as the hard drive.

Figure 95. Attaching Disk Drive Power to the PCI Express Device



1. Device Disk Drive Power Connector
2. PC Disk Drive Power Connector



Note Connecting or disconnecting the disk drive power connector can affect the analog performance of your device. To compensate for this, NI recommends that you self-calibrate the PCI Express DAQ device in MAX after connecting or disconnecting the disk drive power connector; refer to the [Configuring the Device in NI MAX](#) section.

8. Replace any access panels on the computer case.
9. Plug in and power on your computer.
10. If applicable, install accessories and/or terminal blocks as described in the installation guides.
11. Attach sensors and signal lines to the device, terminal block, or accessory terminals. Refer to the documentation for your DAQ device or accessory for terminal/pinout information.

Related concepts:

- [Cables and Accessories](#)

PCI Express Device Disk Drive Power Connector

The disk drive power connector is a four-pin hard drive connector that, when connected, increases the current the device can supply on the +5 V terminal.



Note To ensure the best performance and accuracy, temperature regulation is required. For more information, refer to the document *Guidelines for Temperature Management of PCIe Devices*, by going to ni.com/info and entering info code `PCIeCooling`.

When to Use the Disk Drive Power Connector

For most applications, it is not necessary to install the disk drive power connector.

You should install the disk drive power connector in either of the following situations:

- You need more power than listed in the device specifications
- You are using an accessory without an external power supply, such as the SC-2345

Refer to the device specifications for more information about PCI Express power requirements and current limits.

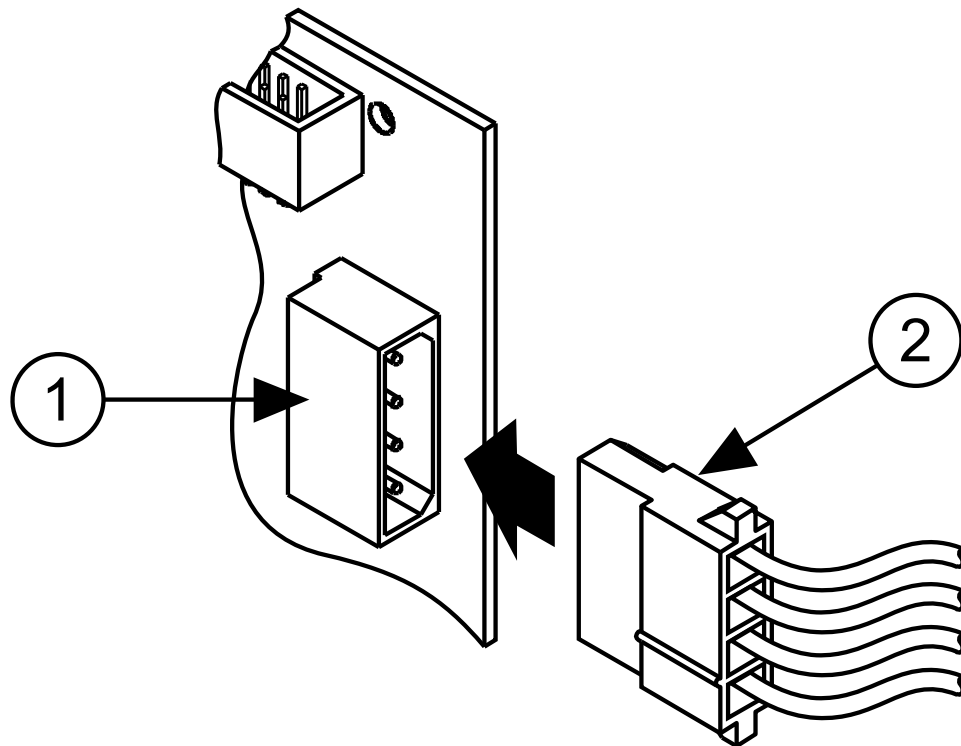
Disk Drive Power Connector Installation

Before installing the disk drive power connector, you must install and set up the PCI

Express X Series device as described in the DAQ Getting Started Guide for PCI/PCI Express. Complete the following steps to install the disk drive power connector.

1. Power off and unplug the computer.
2. Remove the computer cover.
3. Attach the PC disk drive power connector to the disk drive power connector on the device, as shown in the following figure.

Figure 96. Connecting to the Disk Drive Power Connector



- a. Device Disk Drive Power Connector
- b. PC Disk Drive Power Connector



Note The power available on the disk drive power connectors in a computer can vary. For example, consider using a disk drive power connector that is not in the same power chain as the hard drive.

4. Replace the computer cover, and plug in and power on the computer.

Related tasks:

- [Maintaining Forced-Air Cooling for PCIe Devices](#)

Real-Time System Integration (RTSI)

Real-Time System Integration (RTSI) is a signal bus among devices that allows you to share trigger signals between devices or use a common clock (or timebase) to drive the timing engine on multiple devices.

RTSI Connector Pinout

The following figure shows the RTSI connector pinout and signal descriptions.

Figure 97. PCI Express RTSI Pinout

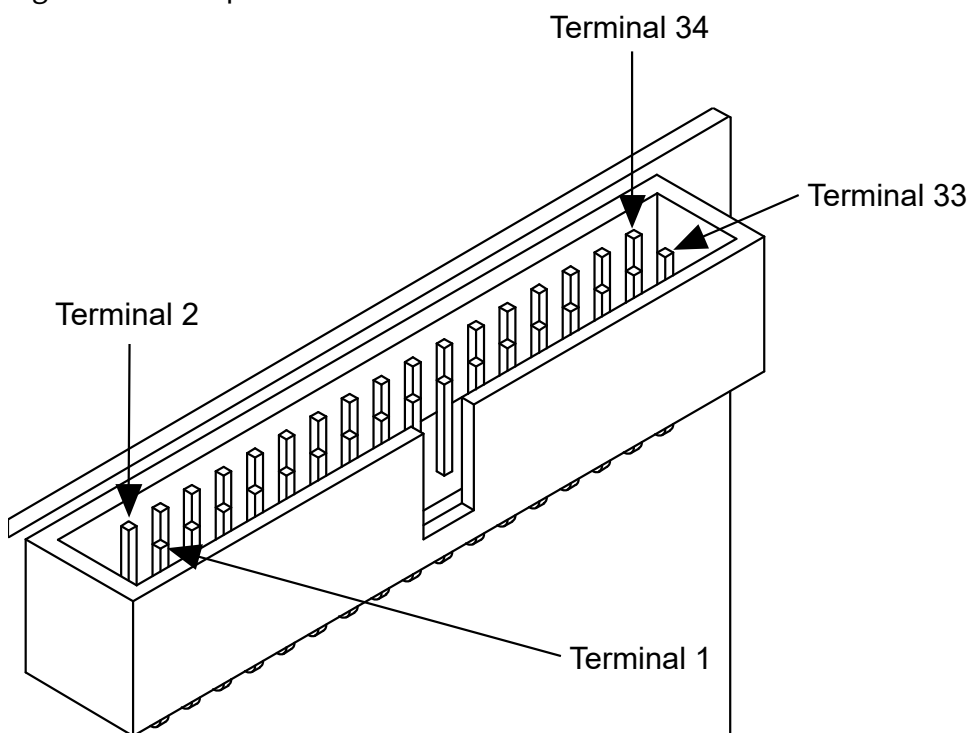


Table 18. RTSI Signals

RTSI Bus Signal	Terminal
RTSI 7	34
RTSI 6	32
RTSI 5	30
RTSI 4	28
RTSI 3	26
RTSI 2	24

RTSI Bus Signal	Terminal
RTSI 1	22
RTSI 0	20
Not Connected. Do not connect signals to these terminals.	1 through 18
D GND	19, 21, 23, 25, 27, 29, 31, 33

Using RTSI as Outputs

RTSI <0..7> are bidirectional terminals. As an output, you can drive any of the following signals to any RTSI terminal:

- AI Start Trigger (ai/StartTrigger)
- AI Reference Trigger (ai/ReferenceTrigger)
- AI Convert Clock* (ai/ConvertClock)
- AI Sample Clock (ai/SampleClock)
- AI Pause Trigger (ai/PauseTrigger)
- AO Sample Clock* (ao/SampleClock)
- AO Start Trigger (ao/StartTrigger)
- AO Pause Trigger (ao/PauseTrigger)
- DI Start Trigger (di/StartTrigger)
- DI Sample Clock (di/SampleClock)
- DI Pause Trigger (di/PauseTrigger)
- DI Reference Trigger (di/ReferenceTrigger)
- DO Start Trigger (do/StartTrigger)
- DO Sample Clock* (do/SampleClock)
- DO Pause Trigger (do/PauseTrigger)
- 10 MHz Reference Clock
- Counter n Source, Gate, Z, Internal Output
- Change Detection Event
- Analog Comparison Event
- FREQ OUT
- PFI <0..5>



Note Signals with a * are inverted before being driven on the RTSI terminals.

Using RTSI Terminals as Timing Input Signals

You can use RTSI terminals to route external timing signals to many different X Series functions. Each RTSI terminal can be routed to any of the following signals:

- AI Convert Clock (ai/ConvertClock)
- AI Sample Clock (ai/SampleClock)
- AI Start Trigger (ai/StartTrigger)
- AI Reference Trigger (ai/ReferenceTrigger)
- AI Pause Trigger (ai/PauseTrigger)
- AI Sample Clock Timebase (ai/SampleClockTimebase)
- AO Start Trigger (ao/StartTrigger)
- AO Sample Clock (ao/SampleClock)
- AO Sample Clock Timebase (ao/SampleClockTimebase)
- AO Pause Trigger (ao/PauseTrigger)
- Counter input signals for all counters—Source, Gate, Aux, HW_Arm, A, B, or Z
- DI Sample Clock (di/SampleClock)
- DI Start Trigger (di/StartTrigger)
- DI Pause Trigger (di/PauseTrigger)
- DI Reference Trigger (di/ReferenceTrigger)
- DO Sample Clock (do/SampleClock)
- DO Sample Clock Timebase (do/SampleClockTimebase)

Most functions allow you to configure the polarity of RTSI inputs and whether the input is edge or level sensitive.

RTSI Filters

You can enable a programmable debouncing filter on each RTSI signal. When a RTSI input is routed directly to PFI, the PCIe-6353 and PXIe-6353 device does not use the filtered version of the input signal.

Configuring the PCIe-6353 and PXIe-6353 in MAX

Use Measurement & Automation Explorer (MAX) to configure your NI hardware. MAX informs other programs about which NI hardware products are in the system and how they are configured. MAX is automatically installed with NI-DAQmx.

1. Launch MAX.
2. In the configuration tree, expand **Devices and Interfaces** to see the list of installed NI hardware.
Installed modules appear under the name of their associated chassis.
3. Expand your **Chassis** tree item.
MAX lists all modules installed in the chassis. Your default names may vary.



Note If you do not see your module listed, press <F5> to refresh the list of installed modules. If the module is still not listed, power off the system, ensure the module is correctly installed, and restart.

4. Record the identifier MAX assigns to the hardware. Use this identifier when programming the PCIe-6353 and PXIe-6353.
5. Self-test the hardware by selecting the item in the configuration tree and clicking **Self-Test** in the MAX toolbar.
The MAX self-test performs a basic verification of hardware resources.
6. (Optional) Right-click the PCIe-6353 and PXIe-6353 and select **Configure** to add accessory information and configure the module.
7. Right-click the PCIe-6353 and PXIe-6353 and select **Test Panels** to test device functionality. Click **Start** to test the device functions, and then **Stop** and **Close** to exit the test panel.
If the test panel displays an error message, contact NI Support.

What Should I Do if the PCIe-6353 and PXIe-6353 Does Not Appear in MAX?

1. In the MAX configuration tree, expand **Devices and Interfaces**.
2. Expand the **Chassis** tree to see the list of installed hardware, and press <F5> to refresh the list.
3. If the module is still not listed, power off the system, ensure that all hardware is correctly installed, and restart the system.
4. Navigate to the Device Manager by right-clicking the Start button, and selecting **Device Manager**.
5. Verify the PCIe-6353 and PXIe-6353 appears in the Device Manager.
 - a. Under an NI entry, confirm that a PCIe-6353 and PXIe-6353 entry appears.



Note If you are using a PC with a device for PXI remote control system, under **System Devices**, also confirm that no error conditions appear for the **PCI-to-PCI Bridge**.

- b. If error conditions appear, reinstall the NI-DAQmx driver.

What Should I Do if the PCIe-6353 and PXIe-6353 Fails the Self-Test?

1. Restart the system.
2. Launch MAX, and perform the self-test again.
3. Power off the chassis.
4. Reinstall the failed module in a different slot.
5. Power on the chassis.
6. Perform the self-test again.

Self-Calibrating the PCIe-6353 and PXIe-6353 in MAX

Self-calibration adjusts the PCIe-6353 and PXIe-6353 for variations in the module environment. The PCIe-6353 and PXIe-6353 is externally calibrated at the factory, however you should perform a complete self-calibration after you install the module and whenever the ambient temperature changes.

1. Install the PCIe-6353 and PXIe-6353 and let it warm up for the recommended warm-up time listed in the .



Note Warm up begins when the PXI chassis has been powered on and the operating system has completely loaded.



Note Remove all sensors and accessories from the PCIe-6353 and PXIe-6353 before self-calibrating.

2. Launch MAX.
3. Click the **Self-Calibrate** button in the upper right corner of MAX.
4. Right-click the name of the module in the MAX configuration tree and select **Self-Calibrate** from the drop-down menu.



Note You can programmatically self-calibrate your device with NI-DAQmx. Refer to *Device Calibration* for more information.

Related information:

- [Device Calibration in NI-DAQmx User Manual](#)

Configuring a Measurement

Complete the following steps to configure a measurement using the DAQ Assistant from MAX.



Note For Linux users, you must use Hardware Configuration Utility to configure your instrument as NI-DAQmx for Linux does not include MAX, which supports only Windows OS.

1. Launch MAX.
2. Right-click **Data Neighborhood** and select **Create New** to open the DAQ Assistant.
3. Select **NI-DAQmx Task** and click **Next**.
4. Select **Acquire Signals** or **Generate Signals**.
5. Select the I/O type, such as digital input, and the measurement type, such as line input or port input.
6. Select the physical channels to use and click **Next**.
7. Name the task and click **Finish**.
8. Configure individual channel settings. Each physical channel you assign to a task receives a virtual channel name.
 - a. Click **Details** for physical channel information.
 - b. Configure the timing and triggering for your task.
9. Click **Run**.

Connecting Signals to the PCIe-6353 and PXIe-6353

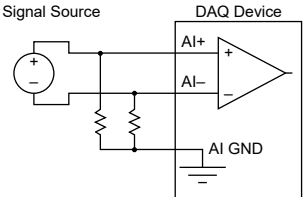
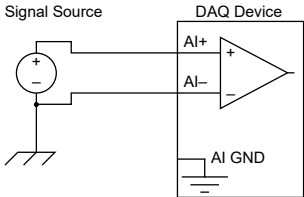
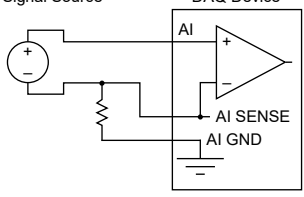
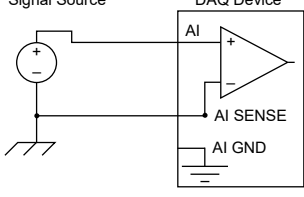
The PCIe-6353 and PXIe-6353 can connect to AI, AO, DIO, counter, timer, and PFI signals.

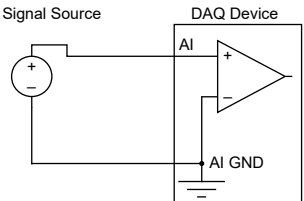
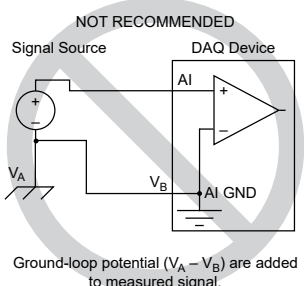
Refer to the following topics for guidance about PCIe-6353 and PXIe-6353 signal connections.

Connecting Analog Input Signals

The following table summarizes the recommended input configuration for floating and ground-referenced signal sources.

Table 19. PCIe-6353 and PXIe-6353 Analog Input Configuration

AI Ground-Reference Setting*	Floating Signal Sources (Not Connected to Building Ground)	Ground-Referenced Signal Sources†
Example scenarios:	Ungrounded thermocouples, signal conditioning with isolated outputs, battery devices	Plug-in instruments with non-isolated outputs
Differential		
Non-Referenced Single-Ended (NRSE)		

AI Ground-Reference Setting*	Floating Signal Sources (Not Connected to Building Ground)	Ground-Referenced Signal Sources†
Example scenarios:	Ungrounded thermocouples, signal conditioning with isolated outputs, battery devices	Plug-in instruments with non-isolated outputs
Referenced Single-Ended (RSE)		
* Refer to the <i>Analog Input Ground-Reference Settings</i> topic for descriptions of the RSE, NRSE, and DIFF modes and software considerations. † Refer to the <i>Connecting Ground-Referenced Signal Sources</i> topic for more information.		

Related reference:

- [Connecting Ground-Referenced Signal Sources](#)
- [Field Wiring Considerations](#)
- [Analog Input for PCIe-6353 and PXIe-6353 Devices](#)
- [Analog Input Common Questions](#)

Related information:

- [Analog Input Ground-Reference Settings \(16-channel\)](#)

Connecting Floating Signal Sources

A floating signal source is not connected to the building ground system, but has an isolated ground-reference point. Some examples of floating signal sources are outputs of transformers, thermocouples, battery-powered devices, optical isolators, and isolation amplifiers. An instrument or device that has an isolated output is a floating signal source.

When to Use Differential Connections with Floating Signal Sources

Use DIFF input connections for any channel that meets any of the following conditions:

- The input signal is low level (less than 1 V).
- The leads connecting the signal to the device are greater than 3 m (10 ft).
- The input signal requires a separate ground-reference point or return signal.
- The signal leads travel through noisy environments.
- Two analog input channels, AI+ and AI-, are available for the signal.

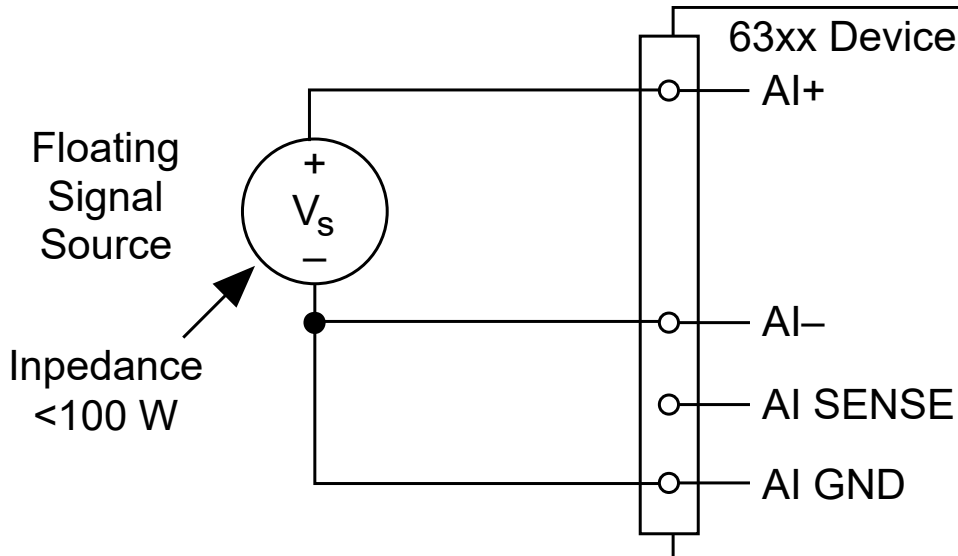
DIFF signal connections reduce noise pickup and increase common-mode noise rejection. DIFF signal connections also allow input signals to float within the common-mode limits of the NI-PGIA.

Using Differential Connections for Floating Signal Sources

It is important to connect the negative lead of a floating source to AI GND (either directly or through a bias resistor). Otherwise, the source may float out of the maximum working voltage range of the NI-PGIA and the DAQ device returns erroneous data.

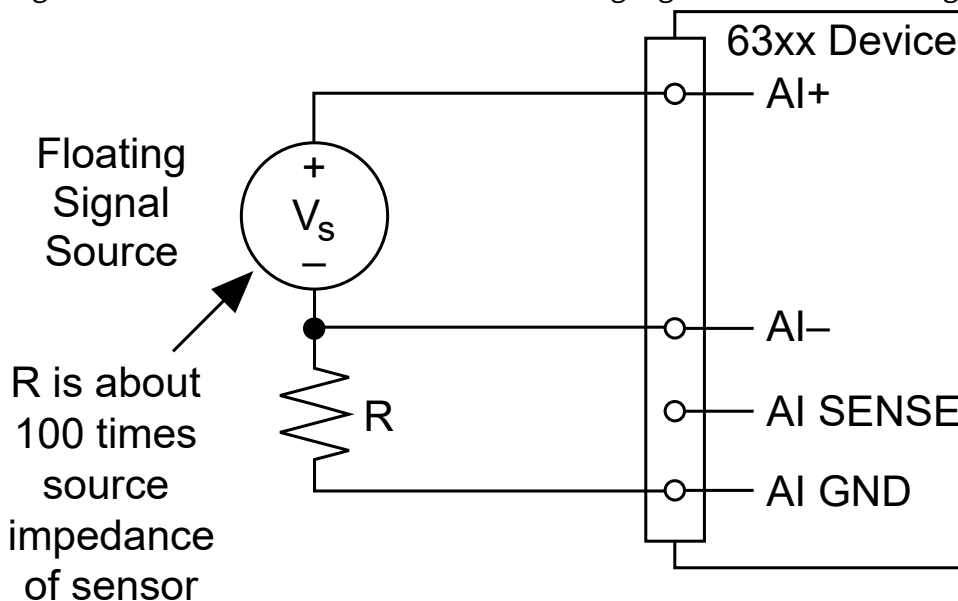
The easiest way to reference the source to AI GND is to connect the positive side of the signal to AI+ and connect the negative side of the signal to AI GND as well as to AI- without using resistors. This connection works well for DC-coupled sources with low source impedance (less than 100 Ω).

Figure 98. Differential Connections for Floating Signal Sources without Bias Resistors



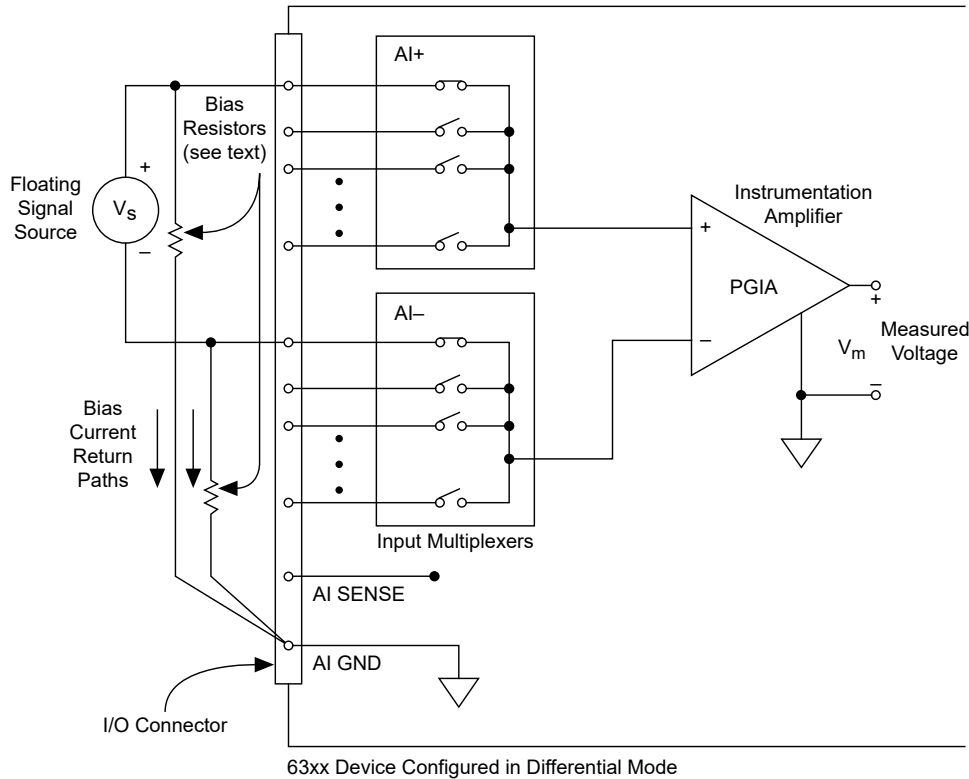
However, for larger source impedances, this connection leaves the DIFF signal path significantly off balance. Noise that couples electrostatically onto the positive line does not couple onto the negative line because it is connected to ground. This noise appears as a differential mode signal instead of a common-mode signal, and thus appears in your data. In this case, instead of directly connecting the negative line to AI GND, connect the negative line to AI GND through a resistor that is about 100 times the equivalent source impedance. The resistor puts the signal path nearly in balance, so that about the same amount of noise couples onto both connections, yielding better rejection of electrostatically coupled noise. This configuration does not load down the source (other than the very high input impedance of the NI-PGIA).

Figure 99. Differential Connections for Floating Signal Sources with Single Bias Resistor



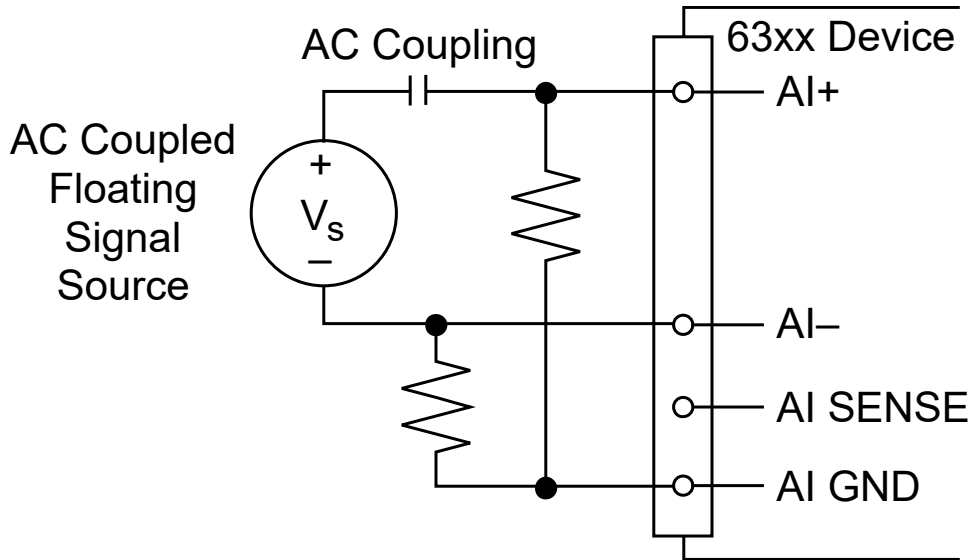
You can fully balance the signal path by connecting another resistor of the same value between the positive input and AI GND, as shown in the following figure. This fully balanced configuration offers slightly better noise rejection, but has the disadvantage of loading the source down with the series combination (sum) of the two resistors. If, for example, the source impedance is 2 k Ω and each of the two resistors is 100 k Ω , the resistors load down the source with 200 k Ω and produce a -1% gain error.

Figure 100. Differential Connections for Floating Signal Sources with Balanced Bias Resistors



Both inputs of the NI-PGIA require a DC path to ground in order for the NI-PGIA to work. If the source is AC coupled (capacitively coupled), the NI-PGIA needs a resistor between the positive input and AI GND. If the source has low-impedance, choose a resistor that is large enough not to significantly load the source, but small enough not to produce significant input offset voltage as a result of input bias current (typically 100 k Ω to 1 M Ω). In this case, connect the negative input directly to AI GND. If the source has high output impedance, balance the signal path as previously described using the same value resistor on both the positive and negative inputs; be aware that there is some gain error from loading down the source, as shown in the following figure.

Figure 101. Differential Connections for AC Coupled Floating Sources with Balanced Bias Resistors



When to Use Non-Referenced Single-Ended (NRSE) Connections with Floating Signal Sources

Only use NRSE input connections if the input signal meets the following conditions:

- The input signal is high-level (greater than 1 V).
- The leads connecting the signal to the device are less than 3 m (10 ft).

DIFF input connections are recommended for greater signal integrity for any input signal that does not meet the preceding conditions.

In the single-ended modes, more electrostatic and magnetic noise couples into the signal connections than in DIFF configurations. The coupling is the result of differences in the signal path. Magnetic coupling is proportional to the area between the two signal conductors. Electrical coupling is a function of how much the electric field differs between the two conductors.

With this type of connection, the NI-PGIA rejects both the common-mode noise in the signal and the ground potential difference between the signal source and the device ground.

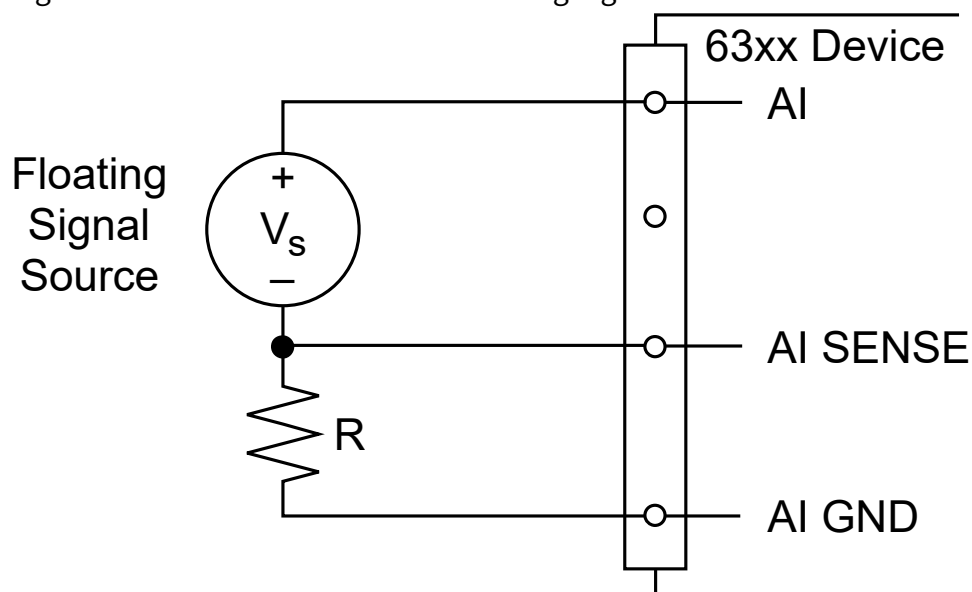
Using Non-Referenced Single-Ended (NRSE) Connections for Floating Signal Sources

It is important to connect the negative lead of a floating signals source to AI GND (either directly or through a resistor). Otherwise the source may float out of the valid

input range of the NI-PGIA and the DAQ device returns erroneous data.

The following figure shows a floating source connected to the DAQ device in NRSE mode.

Figure 102. NRSE Connections for Floating Signal Sources



All of the bias resistor configurations discussed in the *Using Differential Connections for Floating Signal Sources* section apply to the NRSE bias resistors as well. Replace AI- with AI SENSE in the figures included in that section for configurations with zero to two bias resistors. The noise rejection of NRSE mode is better than RSE mode because the AI SENSE connection is made remotely near the source. However, the noise rejection of NRSE mode is worse than DIFF mode because the AI SENSE connection is shared with all channels rather than being cabled in a twisted pair with the AI+ signal.

Using the DAQ Assistant, you can configure the channels for RSE or NRSE input modes. Refer to the *Configuring AI Ground-Reference Settings in Software* topic for more information about the DAQ Assistant.

When to Use Referenced Single-Ended (RSE) Connections with Floating Signal Sources

Only use RSE input connections if the input signal meets the following conditions:

- The input signal can share a common reference point, AI GND, with other signals that use RSE.

- The input signal is high-level (greater than 1 V).
- The leads connecting the signal to the device are less than 3 m (10 ft).

DIFF input connections are recommended for greater signal integrity for any input signal that does not meet the preceding conditions.

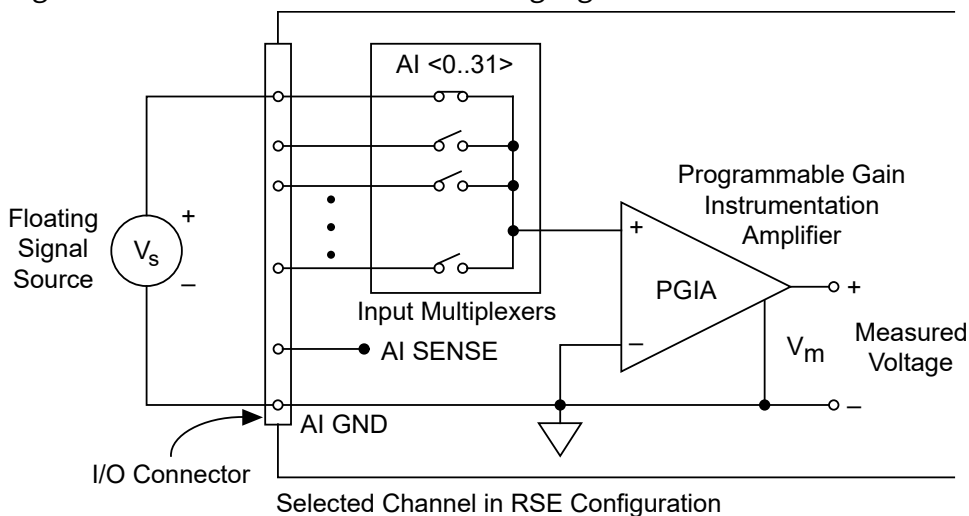
In the single-ended modes, more electrostatic and magnetic noise couples into the signal connections than in DIFF configurations. The coupling is the result of differences in the signal path. Magnetic coupling is proportional to the area between the two signal conductors. Electrical coupling is a function of how much the electric field differs between the two conductors.

With this type of connection, the NI-PGIA rejects both the common-mode noise in the signal and the ground potential difference between the signal source and the device ground.

Using Referenced Single-Ended (RSE) Connections for Floating Signal Sources

The following figure shows how to connect a floating signal source to the MIO X Series device configured for RSE mode.

Figure 103. RSE Connections for Floating Signal Sources



Using the DAQ Assistant, you can configure the channels for RSE or NRSE input modes. Refer to the *Configuring AI Ground-Reference Settings in Software* topic for more information about the DAQ Assistant.

Related reference:

- [Configuring AI Ground-Reference Settings in Software](#)

Connecting Ground-Referenced Signal Sources

A ground-referenced signal source is a signal source connected to the building system ground. It is already connected to a common ground point with respect to the device, assuming that the computer is plugged into the same power system as the source. Non-isolated outputs of instruments and devices that plug into the building power system fall into this category.

The difference in ground potential between two instruments connected to the same building power system is typically between 1 and 100 mV, but the difference can be much higher if power distribution circuits are improperly connected. If a grounded signal source is incorrectly measured, this difference can appear as measurement error. Follow the connection instructions for grounded signal sources to eliminate this ground potential difference from the measured signal.

When to Use Differential Connections with Ground-Referenced Signal Sources

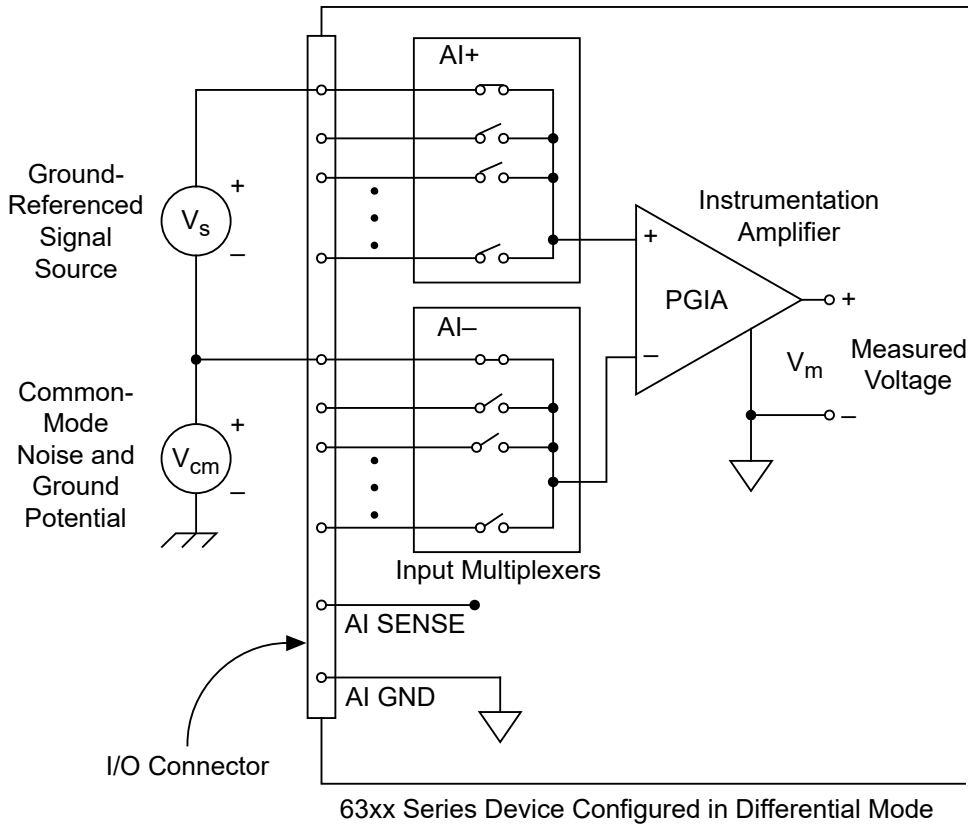
- The input signal is low level (less than 1 V).
- The leads connecting the signal to the device are greater than 3 m (10 ft).
- The input signal requires a separate ground-reference point or return signal.
- The signal leads travel through noisy environments.
- Two analog input channels, AI+ and AI-, are available.

DIFF signal connections reduce noise pickup and increase common-mode noise rejection. DIFF signal connections also allow input signals to float within the common-mode limits of the NI-PGIA.

Using Differential Connections for Ground-Referenced Signal Sources

The following figure shows how to connect a ground-referenced signal source to the PCIe-6353 and PXIe-6353 device configured in differential mode.

Figure 104. Differential Connections for Ground-Referenced Signal Sources



With this type of connection, the NI-PGIA rejects both the common-mode noise in the signal and the ground potential difference between the signal source and the device ground, shown as V_{cm} in the figure.

AI+ and AI- must both remain within ± 11 V of AI GND.

When to Use Non-Referenced Single-Ended (NRSE) Connections with Ground-Referenced Signal Sources

Only use NRSE connections if the input signal meets the following conditions:

- The input signal is high-level (greater than 1 V).
- The leads connecting the signal to the device are less than 3 m (10 ft).
- The input signal can share a common reference point with other signals.

DIFF input connections are recommended for greater signal integrity for any input signal that does not meet the preceding conditions.

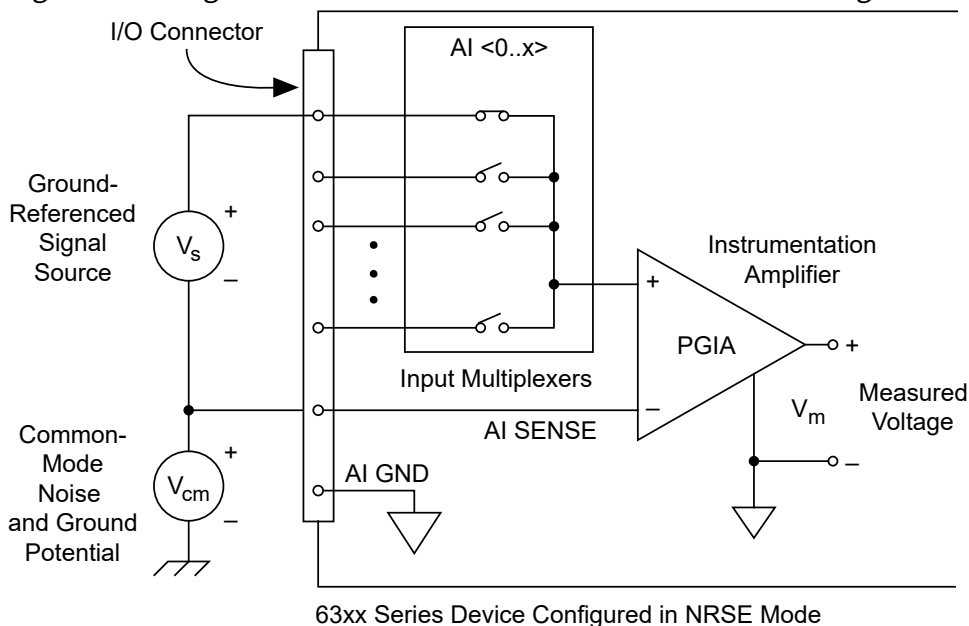
In the single-ended modes, more electrostatic and magnetic noise couples into the signal connections than in DIFF configurations. The coupling is the result of differences in the signal path. Magnetic coupling is proportional to the area between the two signal conductors. Electrical coupling is a function of how much the electric field differs between the two conductors.

With this type of connection, the NI-PGIA rejects both the common-mode noise in the signal and the ground potential difference between the signal source and the device ground.

Using Non-Referenced Single-Ended (NRSE) Connections for Ground-Referenced Signal Sources

The following figure shows how to connect ground-reference signal sources in NRSE mode.

Figure 105. Single-Ended Connections for Ground-Referenced Signal Sources (NRSE Configuration)



Notice AI <0..31> and AI SENSE must both remain within ± 11 V of AI GND.

To measure a single-ended, ground-referenced signal source, you must use the NRSE ground-reference setting. Use the following table to determine how to correctly connect your AI signal.

Table 20. NI 63xx AI Signal Connections

Signal	Ground-Reference
AI <0..15>	AI SENSE
AI <16..79>	AI SENSE 2
AI <80..143>	AI SENSE 3
AI <144..207>	AI SENSE 4

AI SENSE is internally connected to the negative input of the NI-PGIA. Therefore, the ground point of the signal connects to the negative input of the NI-PGIA.

Any potential difference between the device ground and the signal ground appears as a common-mode signal at both the positive and negative inputs of the NI-PGIA, and this difference is rejected by the amplifier. If the input circuitry of a device were referenced to ground, as it is in the RSE ground-reference setting, this difference in ground potentials would appear as an error in the measured voltage.

Using the DAQ Assistant, you can configure the channels for RSE or NRSE input modes. Refer to the *Configuring AI Ground-Reference Settings in Software* topic for more information about the DAQ Assistant.

When to Use Referenced Single-Ended (RSE) Connections with Ground-Referenced Signal Sources

Do not use RSE connections with ground-referenced signal sources. Use NRSE or DIFF connections instead.

As illustrated in the *Connecting Analog Input Signals* topic, there can be a potential difference between AI GND and the ground of the sensor. In RSE mode, this ground loop causes measurement errors.

Related reference:

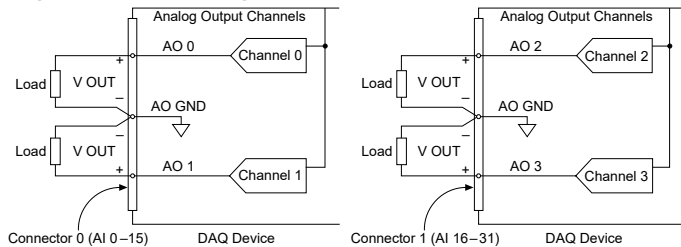
- [Configuring AI Ground-Reference Settings in Software](#)

Connecting Analog Output Signals

AO <0..3> are the voltage output signals for analog output channels 0, 1, 2, and 3. AO GND is the ground reference for AO <0..3>.

The following figure shows how to make analog output connections to the device.

Figure 106. Analog Output Connections



Related reference:

- [Analog Output for PCIe-6353 and PXIe-6353 Devices](#)

Field Wiring Considerations

Environmental noise can seriously affect the measurement accuracy of the device if you do not take proper care when running signal wires between signal sources and the device. The following recommendations apply mainly to AI signal routing to the device, although they also apply to signal routing in general.

Minimize noise pickup and maximize measurement accuracy by taking the following precautions:

- Use differential analog input connections to reject common-mode noise.
- Use individually shielded, twisted-pair wires to connect AI signals to the device. With this type of wire, the signals attached to the positive and negative input channels are twisted together and then covered with a shield. You then connect this shield only at one point to the signal source ground. This kind of connection is required for signals traveling through areas with large magnetic fields or high electromagnetic interference.

Refer to the *Field Wiring and Noise Considerations for Analog Signals* document for more information. To access this document, go to ni.com/info and enter the Info Code

rdfwn3.

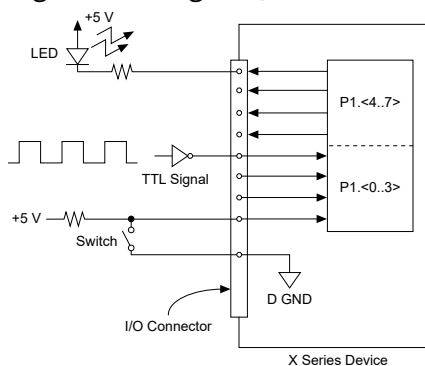
Related information:

- [Field Wiring and Noise Considerations for Analog Signals](#)

Connecting Digital I/O Signals

The DIO signals, P0.<0..31>, P1.<0..7>, and P2.<0..7> are referenced to D GND. You can individually program each line as an input or output. The following figure shows P1.<0..3> configured for digital input and P1.<4..7> configured for digital output. The following figure shows the switch receiving TTL signals and sensing external device states and shows the LED sending TTL signals and driving external devices.

Figure 107. Digital I/O Connections



Exceeding the maximum input voltage ratings, which are listed in each X Series device specifications, can damage the DAQ device and the computer. NI is *not* liable for any damage resulting from such signal connections.

Related reference:

- [Digital I/O for PCIe-6353 and PXIe-6353 Devices](#)

Connecting to Counter Signals

NI-DAQmx routes the counter/timer inputs and outputs to the PFI pins by default.

The following table lists the default NI-DAQmx counter/timer outputs for the PCIe-6353 and PXIe-6353. You can use these defaults or select other sources and destinations for the counter/timer signals in NI-DAQmx.

Refer to the NI-DAQmx topic *X Series Signal Connections for Counters* for more information about how to connect your signals for common counter measurements and generations. PCIe-6353 and PXIe-6353 default PFI lines for counter functions are listed in the NI-DAQmx topic *X Series Physical Channels*.

Table 21. Default Counter/Timer Terminals

Counter/Timer Signal	Default PFI Terminal
CTR 0 SRC	PFI 8
CTR 0 GATE	PFI 9
CTR 0 AUX	PFI 10
CTR 0 OUT	PFI 12
CTR 0 A	PFI 8
CTR 0 Z	PFI 9
CTR 0 B	PFI 10
CTR 1 SRC	PFI 3
CTR 1 GATE	PFI 4
CTR 1 AUX	PFI 11
CTR 1 OUT	PFI 13
CTR 1 A	PFI 3
CTR 1 Z	PFI 4
CTR 1 B	PFI 11
CTR 2 SRC	PFI 0
CTR 2 GATE	PFI 1
CTR 2 AUX	PFI 2
CTR 2 OUT	PFI 14
CTR 2 A	PFI 0
CTR 2 Z	PFI 1
CTR 2 B	PFI 2
CTR 3 SRC	PFI 5

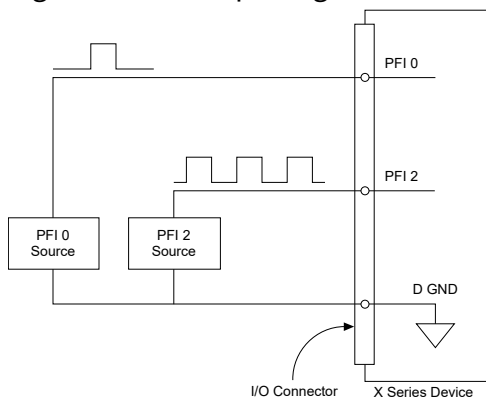
Counter/Timer Signal	Default PFI Terminal
CTR 3 GATE	PFI 6
CTR 3 AUX	PFI 7
CTR 3 OUT	PFI 15
CTR 3 A	PFI 5
CTR 3 Z	PFI 6
CTR 3 B	PFI 7
FREQ OUT	PFI 14

Connecting to PFI Input Signals

All PFI input connections are referenced to D GND.

The following figure shows PFI input connections referenced to D GND and how to connect an external PFI 0 source and an external PFI 2 source to two PFI terminals.

Figure 108. PFI Input Signal Connections



Connecting to the +5 V Power Source

The +5 V terminals on the I/O connector supply +5 V referenced to D GND. Use these terminals to power external circuitry.



Notice To avoid damage, never connect the +5 V power terminals to analog or digital ground or to any other voltage source on this device or any other

device. NI is not liable for damage resulting from such a connection. Refer to the device specifications to obtain the exact power rating.

Device Routing in MAX

To find the device routing table for your device, launch MAX and select **Devices and Interfaces**. Click a device to open a tabbed window in the middle pane. Click the Device Routes tab at the bottom of the pane to display the device routing table.



Note MAX does not display the device routing table for SCXI chassis, SCC connector blocks, or RTSI cable devices.

Each cell in the table is an index with the valid source and destination terminal for the device. These are the same terminal names you can find in the Terminal Name I/O control in LabVIEW.

If a route is possible between a source and destination terminal, the intersecting cell is colored green or yellow. A green cell indicates the route can be made without consuming any important resource of your device. A yellow cell indicates that although the route is possible, something important must be consumed to create the route. Placing the cursor over a yellow square reveals the resource used in the **subsystem used** indicator. Usually, the sacrificed resource is a counter.

When you display the device routing table for a cDAQ chassis or a C Series device, the table contains all of the terminals for the chassis and all devices installed in the chassis.

Related reference:

- [AI Start Trigger Signal](#)
- [AI Reference Trigger Signal](#)
- [AI Pause Trigger Signal](#)
- [AO Sample Clock Signal](#)
- [AO Start Trigger Signal](#)
- [AO Pause Trigger Signal](#)
- [DI Sample Clock Signal](#)

- [DI Start Trigger Signal](#)
- [DI Reference Trigger Signal](#)
- [DI Pause Trigger Signal](#)
- [DO Sample Clock Signal](#)
- [DO Start Trigger Signal](#)
- [DO Pause Trigger Signal](#)
- [Using an Internal Source](#)

Calibration

Calibration maximizes the accuracy of the PCIe-6353 and PXIe-6353.

To ensure accuracy, you must perform calibration. The PCIe-6353 and PXIe-6353 has calibration circuitry to correct gain and offset errors. You can calibrate the device to minimize AO (for AO-capable devices) and AI errors caused by time and temperature drift at run time. No external circuitry is necessary; an internal reference ensures high accuracy and stability over time and temperature changes.

The PCIe-6353 and PXIe-6353 supports two types of calibration: external (factory) calibration and internal self-calibration.

Factory-calibration constants are permanently stored in an onboard EEPROM and cannot be modified. When you self-calibrate the PCIe-6353 and PXIe-6353, software stores new constants in a user-modifiable section of the EEPROM.

Table 22. Calibration Options

Calibration Option	Application	Frequency
External calibration	- All ranges in differential mode - Recalibrates for time drift of onboard calibration constants	Every two years
Self-calibration	Adjusts the scaling constants for analog input and analog output with respect to the onboard reference	After installation and when ambient temperature changes more than 1 °C from the last self-calibration.

All NI measurement devices receive NIST-traceable calibration before shipping.

External Calibration

For information on externally calibrating the PCIe-6353 and PXIe-6353, refer to the *DAQ Multifunction I/O (MIO) and Simultaneous Multifunction I/O (SMIO) Devices Calibration Procedure* document.

Related information:

- [DAQ Multifunction I/O \(MIO\) and Simultaneous Multifunction I/O \(SMIO\) Devices Calibration Procedure](#)

Self-Calibration

Use the self-calibration function to reduce errors caused by usage time and temperature drift. Self-calibration recalculates certain internal reference values, gains, and offsets to improve accuracy significantly over the full operating temperature range of the device.

Every measurement instrument performs within its specifications over a finite temperature range and time. If the excursions in temperature and time exceed those specified and you need performance close to the specifications, then you must recalibrate the instrument at the new temperature or after a certain amount of time. The NI-DAQmx self-calibration function allows you to perform a calibration at the temperature you are making your measurements.

Self-calibration fully recalibrates the analog input and analog output calibration constants for all ranges.



Note DAQmx Self Calibrate and `DAQmxSelfCal` calls device reset.

The PCIe-6353 and PXIe-6353 incorporates a temperature sensor that you can read to determine how much temperature has drifted from the previous calibration. You can also read the previous self-calibration time and date.

During self-calibration, the internal circuitry is automatically disconnected from the input. Therefore, for the majority of applications, you do not need to disconnect the input signals during self-calibration. However, excessive signal levels may generate a self-calibration error. Disconnecting the input signals during a self-calibration prevents the error from occurring.

Cleaning the PCIe-6353 and PXIe-6353

NI recommends the following to clean and maintain your instrument:

- Clean the hardware with a soft, nonmetallic brush. Make sure that the hardware is completely dry and free from contaminants before returning it to service.

Maintaining Forced-Air Cooling for PCIe Devices

Inadequate air circulation can cause the temperature within a PC chassis/case to rise above the maximum recommended operating temperature for your device. Overheating can cause a thermal shutdown or damage to the device.

To maintain optimal forced-air cooling for the NI PCIe-6353 follow these guidelines.

- Install all chassis/case filler panels after installing the device. Missing filler panels disrupt the necessary air circulation in the chassis.
- Allow plenty of space around the chassis/case fan intake vents and exhaust vents. Do not block the fan vents. Blocking the fan vents impedes the airflow that is needed for cooling.
- Maintain proper airflow within the chassis or case.

For devices with an onboard fan:

- Ensure that the onboard fan is not obstructed.
- Leave the slot adjacent to the fan side of the NI PCIe-6353 empty. If you must use the adjacent slot, install a card that allows for the maximum amount of clearance between the fan and the adjacent device (for example, low-profile cards).

For devices without an onboard fan:

- Ensure that the PC chassis/case has active cooling that provides airflow across the card cage.
- Leave the slots adjacent to the NI PCIe-6353 empty. If you must use the adjacent slot, install a card that allows for the maximum amount of clearance between each device (for example, low-profile cards).



Note Refer to your PC chassis/case documentation for more information about air circulation paths, fan settings, space allowances, and cleaning procedures.

Maintaining Forced-Air Cooling for PXIe Devices

Inadequate air circulation can cause the temperature within a PXI chassis to rise above the maximum recommended operating temperature for your PXI/PXI Express module. Overheating can cause a thermal shutdown or damage to the PXI/PXI Express module.

Ensure the ambient temperature of your PXI system is within the specifications for all of the system components. Ensure the ambient temperature provides adequate cooling clearances for your chassis to achieve the required airflow through the chassis. For specific guidelines and practices to maintain forced-air cooling, refer to your chassis documentation.



Note The ambient temperature of a PXI system is defined as the temperature at the chassis fan inlet (air intake).